

NÍVEL ASSOCIATE

DVA-C02

AWS CERTIFIED DEVELOPER – ASSOCIATE

Guia de revisão

Lambda, APIs, DynamoDB, mensageria, segurança, CI/CD, observabilidade e desenvolvimento com IA para a revisão da certificação. Inclui a transição para o DVA-C03.

Erik Nathan eriknathan.me

DOMÍNIOS DO EXAME • PESOS OFICIAIS

32%

Development with AWS Services
Desenvolvimento com serviços AWS

26%

Security
Segurança

24%

Deployment
Implantação

18%

Troubleshooting and
Optimization
Solução de problemas e
otimização

Sumário

COMO USAR ESTE GUIA

Números, limites e atualizações	6	Formato do exame DVA-C02	7
Método para questões do Developer	7	Domínios do exame DVA-C02 (pesos oficiais)	7

CAPÍTULO 1 – PADRÕES DE DESENVOLVIMENTO, SDK E RESILIÊNCIA

1.1	Padrões de arquitetura para desenvolvedores	8	Assinatura, regiões e respostas	10
	Padrões cobrados no guia	8	1.3 Resiliência em código	10
	Stateful vs. stateless	8	Retries, backoff e jitter	10
	Acoplamento forte vs. fraco	9	Idempotência	10
	Síncrono vs. assíncrono	9	Integrações com serviços de terceiros	10
1.2	SDK, CLI e chamadas autenticadas	9	1.4 Decisão rápida — Padrões, SDK e resiliência	11
	Cadeia de provedores de credenciais	9		

CAPÍTULO 2 – AWS LAMBDA

2.1	Fundamentos e configuração	12	Tipos de concorrência	14
	Modelo de execução	12	Cold start e desempenho	15
	Parâmetros de configuração	12	2.4 Versões, aliases e implantação	15
	Pacotes de implantação	13	Versões e aliases	15
2.2	Modelos de invocação e tratamento de erros	13	Lambda em VPC	15
	Invocação síncrona	13	Outras formas de expor e executar funções	16
	Invocação assíncrona	13	Integração e processamento em tempo quase real	16
	Event source mapping (poll-based)	14	Números do Lambda	17
	Números de invocação e erros	14	Padrões e pegadinhas de Lambda	17
2.3	Concorrência e escala	14	2.5 Decisão rápida — Lambda	17

CAPÍTULO 3 – AMAZON API GATEWAY

3.1	Tipos de API e integrações	19	Autorização no API Gateway	21
	REST vs. HTTP vs. WebSocket	19	Throttling e usage plans	21
	Tipos de integração	19	Cache de respostas (REST)	21
	Transformações, validação e códigos de status	20	3.4 Erros, timeouts e observabilidade	22
3.2	Stages, implantação e ambientes	20	Códigos de erro mais cobrados	22
	Stages, deployments e stage variables	20	Logs, métricas e tracing	22
	Custom domains	20	Números do API Gateway	22
3.3	Segurança, throttling e cache	21	3.5 Decisão rápida — API Gateway	23

CAPÍTULO 4 – AMAZON DYNAMODB

4.1	Modelo de dados e chaves	24	Query vs. Scan	26
	Tabelas, itens e chaves primárias	24	4.3 Operações da API	26
	Partition keys de alta cardinalidade	24	Operações de item, condições e contadores	26
	Índices secundários	24	Operações em lote e transações	27
4.2	Leitura, escrita e consistência	25	4.4 Recursos de ciclo de vida e integração	27
	Modelos de consistência	25	TTL, Streams, backups e exportação	27
	Cálculo de capacidade	25	DAX e cache	27
	Modos de capacidade	25		

4.5	Serialização e persistência	28	Números do DynamoDB	28
	Formatos e mapeamento de objetos	28	Padrões e pegadinhas de DynamoDB	28
4.6	Números e pegadinhas do DynamoDB	28	4.7	Decisão rápida — DynamoDB
				29

CAPÍTULO 5 – ARMAZENAMENTO, BANCOS RELACIONAIS E CACHE

5.1	Amazon S3 para desenvolvedores	30	Amazon ElastiCache e estratégias de cache	31
	Operações e recursos mais cobrados	30	5.4	Armazenamentos especializados
	Criptografia no S3	30		OpenSearch, Athena, EFS e EBS
5.2	Bancos relacionais	31		Gestão do ciclo de vida dos dados
	Amazon RDS e Aurora para desenvolvedores	31	5.5	Decisão rápida — Armazenamento e cache
5.3	Cache de dados	31		33

CAPÍTULO 6 – MENSAGERIA, EVENTOS, STREAMING E ORQUESTRAÇÃO

6.1	Amazon SQS	34	Amazon Kinesis Data Streams	36
	Standard vs. FIFO	34	Amazon Data Firehose	37
	Ciclo de vida da mensagem	34	6.5	Orquestração e APIs de dados
6.2	Amazon SNS	35		AWS Step Functions
	Tópicos, assinaturas e fan-out	35		AWS AppSync
6.3	Amazon EventBridge	35	6.6	Números e comparação de mensageria
	Barramentos, regras e destinos	35		Números de mensageria e streaming
	Padrões orientados a eventos com EventBridge	36		Qual serviço escolher
6.4	Streaming de dados	36	6.7	Decisão rápida — Mensageria e orquestração
				39

CAPÍTULO 7 – AUTENTICAÇÃO E AUTORIZAÇÃO

7.1	IAM para desenvolvedores	41	Tokens, fluxos e recursos do User Pool	43
	Políticas e avaliação	41	7.4	Autorização na aplicação e entre serviços
	Roles para cada tipo de computação	41		Bearer tokens
7.2	AWS STS e acesso programático	42		Autorização de granularidade fina
	Operações do STS	42		Autenticação entre serviços em microserviços
7.3	Amazon Cognito	43		Aplicações multi-tenant
	User Pools vs. Identity Pools	43	7.5	Decisão rápida — Autenticação e autorização
				45

CAPÍTULO 8 – CRIPTOGRAFIA E DADOS SENSÍVEIS

8.1	Criptografia em repouso e em trânsito	46	ACM, AWS Private CA e chaves SSH	48
	Conceitos	46	8.4	Gestão de segredos e configuração sensível
8.2	AWS KMS	46		Secrets Manager vs. Parameter Store
	Tipos de chave	46		Variáveis de ambiente com dados sensíveis
	Operações e envelope encryption	47	8.5	Classificação, sanitização e mascaramento
	Key policies, grants e uso entre contas	47		Classificação de dados
	Rotação de chaves	48		Sanitizar e mascarar dados na aplicação
8.3	Certificados e chaves para desenvolvimento	48	8.6	Decisão rápida — Criptografia e dados sensíveis
				50

CAPÍTULO 9 – EMPACOTAMENTO, INFRAESTRUTURA COMO CÓDIGO E AMBIENTES

9.1	Preparar artefatos de implantação	52	Template e recursos do SAM	53
	Dependências, estrutura e requisitos	52	SAM CLI	53
9.2	AWS SAM	53	9.3	AWS CloudFormation
				54

Anatomia do template	54	9.5	Configuração por ambiente	56
Stacks, mudanças e proteção	54		AWS AppConfig	56
Recursos avançados de CloudFormation	55		Gerenciar ambientes em cada serviço	56
9.4 AWS CDK	55	9.6	Decisão rápida — Empacotamento, IaC e ambientes	57
Constructs e comandos	55			

CAPÍTULO 10 – TESTES EM DESENVOLVIMENTO E AUTOMAÇÃO

10.1	Testes unitários e locais	58		Integração com dependências reais e simuladas	59
	Testes unitários com AWS SAM	58		Testar aplicações orientadas a eventos	59
	Eventos de teste	58		Testes gerados com IA	59
10.2	Testes de integração e ambientes de teste	59	10.3	Decisão rápida — Testes	59

CAPÍTULO 11 – CI/CD E ESTRATÉGIAS DE IMPLANTAÇÃO

11.1	AWS CodeBuild	61		Versões, labels e branches	63
	buildspec.yml	61	11.4	AWS Elastic Beanstalk	64
11.2	AWS CodeDeploy	62		Políticas de implantação	64
	Plataformas e configurações de implantação	62	11.5	Contêineres	65
	appspec.yml e hooks	62		ECS, ECR e EKS para desenvolvedores	65
11.3	AWS CodePipeline e artefatos	63	11.6	Estratégias de implantação e rollback	65
	Estrutura do pipeline	63		Comparação das estratégias	65
	AWS CodeArtifact	63	11.7	Decisão rápida — CI/CD e implantação	66

CAPÍTULO 12 – OBSERVABILIDADE E ANÁLISE DE CAUSA RAIZ

12.1	Logging, monitoramento e observabilidade	68	12.3	Tracing distribuído	71
	Diferenças	68		AWS X-Ray e OpenTelemetry	71
	Estratégia de logging e logs estruturados	68	12.4	Troubleshooting e causa raiz	72
12.2	Amazon CloudWatch	68		Erros comuns e o que significam	72
	CloudWatch Logs e Logs Insights	68		Depurar código e falhas de implantação	72
	Métricas, métricas personalizadas e EMF	69		Depurar integrações entre serviços	73
	Alarmes e notificações	70		Health checks e readiness probes	73
	Dashboards e insights	70	12.5	Decisão rápida — Observabilidade e troubleshooting	74

CAPÍTULO 13 – OTIMIZAÇÃO DE APLICAÇÕES

13.1	Concorrência e capacidade	75		Cache com base em headers	76
	Definir e dimensionar concorrência	75		Cache na aplicação	76
	Memória e computação mínimas	75	13.4	Otimizar mensageria e uso de recursos	77
13.2	Perfilamento e análise de desempenho	75		Filtros de assinatura e eficiência na mensageria	77
	Encontrar gargalos	75		Otimizar o uso de recursos	77
13.3	Cache em cada camada	76	13.5	Decisão rápida — Otimização	77

CAPÍTULO 14 – DESENVOLVIMENTO ASSISTIDO POR IA E O DVA-C03

14.1	Ferramentas de IA para desenvolvedores	79		Controles ao integrar serviços de IA	80
	Amazon Q Developer e Kiro	79	14.3	DVA-C02 ou DVA-C03	80
	IA no CI/CD, nos testes e no troubleshooting	79		O que muda e como decidir	80
14.2	Segurança em aplicações que usam IA	80	14.4	Decisão rápida — IA no desenvolvimento	81

CAPÍTULO 15 – PADRÕES RECORRENTES E PALAVRAS-CHAVE

15.1	Palavras-chave do enunciado	82	15.3	Afirmações falsas que aparecem como alternativa	84
15.2	Pares que mais se confundem	83			

CAPÍTULO 16 – MAPA DE DOMÍNIOS DO EXAME

16.1	Domínio 1 — Development with AWS Services (32%)	85	16.4	Domínio 4 — Troubleshooting and Optimization (18%)	86
16.2	Domínio 2 — Security (26%)	85	16.5	Observação sobre o modelo de pontuação	86
16.3	Domínio 3 — Deployment (24%)	85			

CAPÍTULO 17 – AUTOTESTE – FLASHCARDS DE REVISÃO RÁPIDA

17.1	Padrões e SDK	87	17.8	Criptografia e segredos	102
17.2	Lambda	88	17.9	Empacotamento, IaC e testes	105
17.3	API Gateway	92	17.10	CI/CD e implantação	107
17.4	DynamoDB	94	17.11	Observabilidade e otimização	109
17.5	Armazenamento e cache	96	17.12	IA no desenvolvimento	111
17.6	Mensageria e orquestração	97	17.13	Pegadinhas Duplas (confusões mais recorrentes)	112
17.7	Autenticação e autorização	100			

CAPÍTULO 18 – MAPA DE COBERTURA DO GUIA OFICIAL E QUESTÕES DE MÚLTIPLA RESPOSTA

18.1	Como resolver múltipla resposta	113
------	---------------------------------	-----

Como usar este guia

- Use os capítulos 1 a 14 para revisão de conteúdo. Eles seguem a lógica do dia a dia do desenvolvedor: padrões e SDK, Lambda, API Gateway, dados, mensageria, segurança, empacotamento, testes, CI/CD, observabilidade, otimização e desenvolvimento com IA.
- Use a tabela **Decisão rápida** ao final de cada capítulo (1 a 14) como cheat sheet de véspera de prova: cada uma reúne cenários típicos, a resposta correta e os distratores clássicos daquele tema.
- Use as tabelas **Números de...** dentro dos capítulos para revisar limites. O Developer - Associate cobra números com mais frequência que as outras provas (timeout do Lambda, tamanho de item do DynamoDB, visibility timeout do SQS, cálculo de RCU e WCU).
- Use os **Padrões recorrentes e palavras-chave** (seção 15) para treinar a leitura do enunciado. Expressões como "sem alterar o código", "menor esforço operacional", "credenciais temporárias" ou "sem perder mensagens" costumam decidir a questão.
- Use o **Mapa de Domínios** (seção 16) para priorizar a revisão conforme o peso de cada domínio na nota final.
- Use o **Autoteste** (seção 17) para revisão ativa: cada flashcard esconde a resposta até você clicar.
- Use o **Mapa de cobertura e questões de múltipla resposta** (seção 18) para conferir as 13 tarefas oficiais e praticar questões com mais de uma resposta.
- Diferente do Cloud Practitioner, o Developer cobra **como usar** os serviços: nomes de APIs, parâmetros, arquivos de configuração (`buildspec.yml`, `appspec.yml`, `template.yml`), códigos de erro e o comportamento em falhas.

Números, limites e atualizações

Os limites e comportamentos deste guia foram conferidos na documentação oficial da AWS (docs.aws.amazon.com e aws.amazon.com) em **setembro de 2026**. São uma referência datada: confirme cotas na documentação antes de depender de um valor exato. Vários limites clássicos de material de estudo **mudaram entre 2024 e 2026**; quando for o caso, o guia mostra o valor atual e o antigo.

Mudanças recentes que afetam o Developer - Associate e ainda não aparecem na maior parte dos cursos e simulados:

- **DVA-C03 a caminho:** a AWS anunciou uma nova versão do exame. O **último dia para fazer o DVA-C02 é 30/nov/2026**; o **DVA-C03** abre inscrições em 27/out/2026 e começa em **1º/dez/2026**. Os fundamentos são os mesmos; o C03 acrescenta desenvolvimento assistido por IA e segurança de aplicações com IA. Veja a seção 14.
- **Payloads maiores:** Lambda assíncrono passou de 256 KB para **1 MB**; SQS e EventBridge passaram para **1 MB/1 MiB**; SNS aceita até **1 MiB** com o atributo `MaximumMessageSize`; Kinesis Data Streams aceita registros de até **10 MiB**; o Lambda faz **streaming de respostas de até 200 MB**.
- **API Gateway:** o timeout de integração de APIs REST regionais e privadas pode passar de 29 segundos (com cota), e as APIs REST agora fazem **response streaming** por até 15 minutos.
- **X-Ray:** os SDKs e o daemon do X-Ray entraram em **modo de manutenção em 25/fev/2026** (fim do suporte em 25/fev/2027). A recomendação é instrumentar com **OpenTelemetry / AWS Distro for OpenTelemetry (ADOT)** e enviar os traces ao X-Ray.
- **Amazon Q Developer:** não aceita novas assinaturas desde 15/mai/2026; o sucessor é o **Kiro**. O guia do DVA-C02 ainda cita o Q Developer (habilidades 1.1.11 e 3.3.6).
- **AWS CodeCommit** voltou à disponibilidade geral em nov/2025, depois de ter sido fechado para novos clientes em 2024.
- **Amazon ECS** passou a fazer implantações **blue/green, linear e canary nativas**, sem precisar do CodeDeploy.

Método para questões do Developer

- 1. Identifique o serviço e a operação:** a questão é sobre invocação, permissão, configuração, implantação ou depuração? Muitas alternativas usam o serviço certo com a operação errada.
- 2. Procure a restrição decisiva:** "sem alterar o código da aplicação", "com o menor esforço operacional", "sem armazenar credenciais", "garantir a ordem", "processar exatamente uma vez", "sem tempo de inatividade".
- 3. Prefira o recurso nativo:** um recurso já existente do serviço (DLQ, Destinations, retries do SDK, stage variables, aliases, TTL) quase sempre vence código personalizado.
- 4. Pense no modo de falha:** o que acontece se a função falhar, estourar o timeout ou for estrangulada (throttled)? A resposta certa costuma tratar o erro, não apenas o caminho feliz.
- 5. Credenciais sempre temporárias:** roles do IAM, STS, Cognito Identity Pools. Access keys no código, em variáveis de ambiente ou na AMI são quase sempre a alternativa errada.
- 6. Questões com várias respostas:** confira quantas opções o enunciado pede e avalie cada uma isoladamente.

Formato do exame DVA-C02

Item	Valor
Código	DVA-C02 (guia versão 2.1). Disponível até 30/nov/2026
Questões	65 no total: 50 pontuadas e 15 não pontuadas (não identificadas)
Duração	130 minutos
Tipos de questão	Múltipla escolha (1 correta entre 4) e múltipla resposta (2 ou mais corretas entre 5 ou mais)
Pontuação	Escala de 100 a 1.000; mínimo de 720 para aprovação
Modelo de pontuação	Compensatório: não é preciso passar em cada domínio, só na prova como um todo
Chute	Questão em branco conta como errada e não há penalidade por errar
Público-alvo	1 ano ou mais de experiência desenvolvendo e mantendo aplicações com serviços AWS
Fora do escopo	Projetar arquiteturas e esquemas de banco, projetar pipelines de CI/CD, administrar usuários e grupos do IAM, administrar servidores e SO, projetar redes (VPC, Direct Connect)
Tópicos emergentes	Questões não pontuadas sobre ferramentas de IA para gerar, revisar e testar código, segurança ao integrar serviços de IA e IA no CI/CD e no troubleshooting
Valor	US\$ 150

Domínios do exame DVA-C02 (pesos oficiais)

Domínio	Peso no DVA-C02	Peso anunciado para o DVA-C03
Development with AWS Services (Desenvolvimento com serviços AWS)	32%	30%
Security (Segurança)	26%	26%
Deployment (Implantação) — no C03, Testing and Deployment	24%	22%
Troubleshooting and Optimization (Troubleshooting e otimização)	18%	22%

1. Padrões de Desenvolvimento, SDK e Resiliência

Regra de ouro da prova: aplicações na AWS devem ser **sem estado** (estado fora da instância ou função), **fracamente acopladas** (filas e eventos entre componentes), **assíncronas quando possível** e **resilientes a falhas de dependências** (timeouts, retries com backoff exponencial e jitter, idempotência). O SDK já faz retries e assinatura de requisições; o desenvolvedor configura, não reimplementa.

Padrões de arquitetura para desenvolvedores

Padrões cobrados no guia

Padrão	Como funciona	Serviços típicos
Monolítico	Uma única aplicação implantada como uma unidade; simples de começar, difícil de escalar partes isoladas	EC2, Elastic Beanstalk
Microserviços	Serviços pequenos e independentes, cada um com seus dados, comunicando-se por APIs ou eventos	Lambda, ECS/EKS, API Gateway, DynamoDB por serviço
Orientado a eventos (event-driven)	Produtores emitem eventos sem saber quem consome; consumidores reagem	EventBridge, SNS, SQS, DynamoDB Streams, S3 Event Notifications
Coreografia (choreography)	Cada serviço reage a eventos e publica novos eventos; não há coordenador central	EventBridge, SNS
Orquestração (orchestration)	Um coordenador central chama cada etapa, controla a ordem, os erros e as compensações	AWS Step Functions
Fan-out	Uma mensagem é entregue a vários consumidores em paralelo	SNS → várias filas SQS; EventBridge com vários destinos
Nivelamento de carga por fila (queue-based load leveling)	A fila absorve picos e os consumidores processam no próprio ritmo	SQS + Lambda/ECS
Saga	Transação distribuída em etapas, com ações de compensação quando uma etapa falha	Step Functions com Catch
Claim check	Guarda o payload grande no S3 e envia só a referência na mensagem	S3 + SQS/SNS/EventBridge
Strangler fig	Migra um monólito aos poucos, roteando partes para novos serviços	API Gateway, ALB com regras de roteamento

- **Coreografia vs. orquestração:** coreografia dá mais independência e menos acoplamento, mas é difícil enxergar o fluxo completo. Orquestração dá visibilidade, controle de erros e compensação em um só lugar. Na prova, "fluxo com várias etapas, tratamento de erros e visibilidade do estado" leva a **Step Functions**; "serviços independentes reagindo a eventos de negócio" leva a **EventBridge**.

Stateful vs. stateless

- **Stateless:** cada requisição traz tudo o que é preciso; qualquer instância pode atendê-la. Permite escalar horizontalmente e substituir instâncias. Funções Lambda devem ser tratadas como stateless.
- **Stateful:** o servidor guarda contexto entre requisições (sessão em memória, arquivos locais). Dificulta escalar e perde dados quando a instância é substituída.

- **Como tornar stateless:** guardar sessão no **ElastiCache** ou **DynamoDB**, arquivos no **S3** ou **EFS**, estado de fluxos no **Step Functions**, tokens no cliente (JWT). **Sticky sessions** do ALB mantêm o usuário na mesma instância, mas são um paliativo: a sessão se perde se a instância cair.

Acoplamento forte vs. fraco

- **Fortemente acoplado:** o componente A chama B diretamente e depende de B estar disponível e rápido. Uma falha em B derruba A.
- **Fracamente acoplado:** A publica em uma fila ou barramento e segue; B consome quando puder. Falhas e picos ficam isolados.
- Ferramentas de desacoplamento: **SQS** (fila), **SNS** (pub/sub), **EventBridge** (barramento de eventos), **Kinesis** (stream), **Step Functions** (orquestração com estado).

Síncrono vs. assíncrono

Aspecto	Síncrono	Assíncrono
Comportamento	O chamador espera a resposta	O chamador não espera ; recebe só um aceite
Exemplos	API Gateway → Lambda, ALB → Lambda, Invoke com RequestResponse	S3 → Lambda, SNS → Lambda, EventBridge → Lambda, Invoke com Event, mensagens em SQS
Tratamento de erro	O cliente recebe o erro e decide tentar de novo	O serviço faz retries e envia falhas a uma DLQ ou destino
Quando usar	O usuário precisa do resultado agora	Processamento longo, picos, integrações que podem falhar

- Para tarefas longas atrás de uma API: a API grava o pedido em uma fila (ou inicia um Step Functions) e **devolve 202 Accepted** com um ID; o cliente consulta o status depois ou recebe um webhook/notificação (WebSocket, AppSync subscription).

SDK, CLI e chamadas autenticadas

Cadeia de provedores de credenciais

Os SDKs e a CLI procuram credenciais em uma ordem padrão (os detalhes variam um pouco por linguagem):

1. **Parâmetros explícitos no código** (evite).
2. **Variáveis de ambiente:** `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, `AWS_PROFILE`, `AWS_REGION`.
3. **Arquivos compartilhados** `~/.aws/credentials` e `~/.aws/config` (perfis, `role_arn` para assumir role, `sso_session` para IAM Identity Center).
4. **Credenciais do contêiner** (task role do ECS, EKS Pod Identity).
5. **Metadados da instância EC2** (IMDSv2, perfil de instância).

- Em Lambda, a **execution role** já é exposta como variáveis de ambiente de credenciais temporárias. **Nunca** coloque access keys em código, variáveis de ambiente de função ou imagens.
- Localmente, prefira `aws configure sso` (IAM Identity Center) ou perfis que assumem roles, em vez de access keys de longo prazo.

Assinatura, regiões e respostas

- **Signature Version 4 (SigV4):** toda chamada às APIs AWS é assinada com as credenciais; o SDK e a CLI fazem isso automaticamente. Para chamar um API Gateway com autorização IAM ou uma Lambda Function URL com `AWS_IAM`, a requisição HTTP também precisa ser assinada com SigV4.
- **Relógio fora de sincronia** gera erros de assinatura (`SignatureDoesNotMatch`, `RequestTimeTooSkewed`).
- **Região:** defina por variável `AWS_REGION`, perfil ou código. Serviço chamado na região errada retorna "recurso não encontrado".
- **Paginação:** listas longas vêm em páginas (`NextToken`, `LastEvaluatedKey`, `Marker`). Use os **paginators** do SDK; a CLI pagina automaticamente (`--max-items`, `--starting-token`, `--no-paginate`).
- **Waiters:** funções do SDK que esperam um recurso atingir um estado (ex.: `bucket_exists`, `stack_create_complete`), evitando loops manuais.
- **CLI úteis na prova:** `--query` (filtro JMESPath no cliente), `--filter` (filtro no servidor, quando existe), `--output json|text|table|yaml`, `--dry-run` (EC2: testa permissão sem executar), `--profile`, `--region`, `--debug`.
- **Mensagens de autorização codificadas:** algumas APIs (como EC2) retornam uma mensagem de erro codificada; decodifique com `aws sts decode-authorization-message` (exige a permissão `sts:DecodeAuthorizationMessage`).
- **"Quem sou eu?":** `aws sts get-caller-identity` mostra conta, ARN e usuário/role das credenciais em uso.
- **AWS CloudShell:** terminal no navegador, já autenticado com a identidade do console, com CLI e SDKs instalados e 1 GB de armazenamento persistente por Região.

Resiliência em código

Retries, backoff e jitter

- Erros **transitórios** (`throttling 429` / `ThrottlingException` / `ProvisionedThroughputExceededException`, `5xx`, timeouts de rede) devem ser **tentados de novo**. Erros de cliente (`400`, `403` por falta de permissão, `ValidationException`) **não** devem.
- **Backoff exponencial:** esperar 1 s, 2 s, 4 s, 8 s... entre tentativas, com um teto.
- **Jitter:** somar aleatoriedade à espera para evitar que muitos clientes tentem ao mesmo tempo (thundering herd).
- **Os SDKs já fazem isso.** Modos de retry: `legacy`, `standard` (padrão atual, 3 tentativas, backoff com jitter) e `adaptive` (acrescenta limitação de taxa no cliente). Configure por `AWS_RETRY_MODE`, `AWS_MAX_ATTEMPTS` ou no código.
- **Na prova:** "a aplicação recebe `ProvisionedThroughputExceededException` esporadicamente" → **retries com backoff exponencial** (o SDK já faz) e, se persistir, aumentar a capacidade ou corrigir a chave quente.

Idempotência

- Uma operação é **idempotente** quando executá-la várias vezes produz o mesmo efeito que uma vez. É obrigatória em sistemas com retries e entrega **ao menos uma vez** (SQS Standard, SNS, EventBridge, Lambda assíncrono).
- **Como implementar: chave de idempotência** (ID do pedido, `MessageDeduplicationId`, `ClientToken`) guardada no DynamoDB com **escrita condicional** (`attribute_not_exists`) e TTL; o **Powertools for AWS Lambda** tem um utilitário de idempotência pronto.
- Muitas APIs AWS aceitam `ClientToken` / `ClientRequestToken` para tornar criações idempotentes.

Integrações com serviços de terceiros

- **Timeouts** curtos e explícitos em toda chamada externa (nunca herde o timeout inteiro da função).

- **Retries** limitados com backoff exponencial e jitter, só para erros transitórios.
- **Circuit breaker**: depois de N falhas seguidas, "abre o circuito" e para de chamar o serviço por um tempo, devolvendo erro rápido ou resposta alternativa (fallback); depois testa de novo ("meio aberto"). O estado do circuito pode ficar no **DynamoDB** ou **ElastiCache**, ou ser implementado com **Step Functions**.
- **Fallback**: resposta em cache, valor padrão ou degradação elegante.
- **Bulkhead**: isolar recursos (ex.: **concorrência reservada** do Lambda por função) para que uma dependência lenta não consuma tudo.
- **Desacoplar com fila e DLQ**: enviar a chamada ao terceiro por SQS; mensagens que falharem repetidamente vão para a **DLQ** para análise e reprocessamento.
- **Limitação de taxa** (rate limiting): respeitar os limites do terceiro, por exemplo com a concorrência máxima do consumidor SQS ou o `MaximumConcurrency` do event source mapping.
- **Segredos** do terceiro (chaves de API) no **Secrets Manager**, nunca no código.

Decisão rápida — Padrões, SDK e resiliência

Cenário / requisito	Resposta correta	Distratores clássicos
Fluxo com várias etapas, compensação e visibilidade do estado	Orquestração com Step Functions	Coreografia com SNS, Lambda chamando Lambda
Serviços independentes reagindo a eventos de negócio	Coreografia com EventBridge	Step Functions central
Entregar a mesma mensagem a vários consumidores	Fan-out SNS → várias SQS (ou EventBridge com vários destinos)	Uma fila SQS com vários consumidores
Absorver picos sem perder pedidos	SQS entre a API e o processamento	Aumentar o timeout da API
Sessão do usuário sobrevivendo à troca de instâncias	ElastiCache ou DynamoDB para sessão	Sticky sessions, disco local
Throttling esporádico em chamadas ao SDK	Retries com backoff exponencial e jitter (já no SDK)	Loop de retry imediato
Processar a mesma mensagem duas vezes sem efeito duplicado	Idempotência (chave + escrita condicional)	Confiar em entrega única do SQS Standard
Dependência externa instável derrubando a aplicação	Circuit breaker + timeout + fallback	Retries infinitos
Descobrir qual identidade o código está usando	<code>aws sts get-caller-identity</code>	<code>aws iam list-users</code>
Ler uma mensagem de erro de autorização codificada	<code>aws sts decode-authorization-message</code>	CloudTrail sozinho
Filtrar a saída da CLI no cliente	<code>--query</code> (JMESPath)	<code>--filter</code>
Testar se tem permissão para lançar EC2 sem lançar	<code>--dry-run</code>	Policy simulator sozinho
Tarefa longa atrás de uma API síncrona	Fila/Step Functions + 202 Accepted + consulta de status	Aumentar timeout até o limite
Payload grande em mensagem	Claim check: S3 + referência na mensagem	Dividir manualmente em várias mensagens

2. AWS Lambda

Regra de ouro da prova: primeiro identifique **como a função é invocada**. **Síncrona** (API Gateway, ALB, SDK RequestResponse): o chamador recebe o erro e faz o retry. **Assíncrona** (S3, SNS, EventBridge, Event): o Lambda faz **2 retries** e envia a falha a um **destino** ou **DLQ**. **Event source mapping** (SQS, Kinesis, DynamoDB Streams, MSK): o Lambda **busca** os registros em lotes, e o comportamento de erro depende da fonte. Quase toda questão de Lambda se resolve por aí.

Fundamentos e configuração

Modelo de execução

- A função roda em um **ambiente de execução** (microVM Firecracker) criado sob demanda. Na primeira requisição há um **cold start**: baixar o código, iniciar o runtime e executar o **código de inicialização** (fora do handler). Requisições seguintes reutilizam o ambiente (**warm start**).
- Handler**: a função chamada a cada invocação, com o **evento** (payload JSON) e o **contexto** (ID da requisição, tempo restante com `getRemainingTimeInMillis()`, nome da função, memória).
- Coloque fora do handler** tudo o que pode ser reutilizado: clientes do SDK, conexões com bancos, leitura de configuração e segredos. Isso reduz a latência dos warm starts.
- /tmp**: armazenamento efêmero de 512 MB a 10.240 MB, reaproveitado enquanto o ambiente existir. Não é persistente nem compartilhado entre ambientes.
- Uma requisição por vez por ambiente**: 100 requisições simultâneas exigem 100 ambientes (exceto em Lambda Managed Instances, que aceita várias requisições por ambiente).

Parâmetros de configuração

Parâmetro	O que controla	Detalhes para a prova
Memória	128 MB a 10.240 MB	CPU e rede são proporcionais à memória ; com 1.769 MB a função tem o equivalente a 1 vCPU. Aumentar memória pode reduzir o custo total se a função ficar mais rápida
Timeout	Até 900 s (15 min)	Padrão de 3 s. Estourou → erro <code>Task timed out</code> . Processos mais longos: Step Functions, Fargate, Batch ou durable functions
Runtime	Node.js, Python, Java, .NET, Ruby, <code>provided.al2023</code> (runtime personalizado: Go, Rust etc.) ou imagem de contêiner	Runtimes antigos são descontinuados; atualize antes da data de fim de suporte
Handler	Arquivo e função de entrada	Ex.: <code>app.lambda_handler</code> (Python), <code>index.handler</code> (Node.js)
Arquitetura	<code>x86_64</code> ou <code>arm64</code> (Graviton)	<code>arm64</code> costuma dar melhor preço-desempenho
Variáveis de ambiente	Configuração por função, até 4 KB no total	Criptografadas em repouso com KMS; para segredos use Secrets Manager/Parameter Store (seção 8)
Layers	Até 5 layers por função	Compartilhar bibliotecas, runtimes e dependências entre funções; entram no limite de 250 MB descompactado
Extensions	Processos que rodam junto da função (via layers)	Monitoramento, segurança, cache de segredos e configuração (ex.: AWS Parameters and Secrets Lambda Extension, AppConfig)
Armazenamento efêmero	<code>/tmp</code> de 512 MB a 10.240 MB	Paga-se o excedente a 512 MB

Parâmetro	O que controla	Detalhes para a prova
Concorrência	Reservada ou provisionada	Veja "Concorrência e escala"
Triggers	Fontes que invocam a função	S3, SNS, EventBridge, API Gateway, SQS, Kinesis, DynamoDB Streams etc.
Destinations	Para onde vai o resultado de invocações assíncronas e de algumas event source mappings	Sucesso e falha: SQS, SNS, Lambda, EventBridge; falha também para S3
Execution role	Role do IAM que a função assume	Permissões da função para chamar outros serviços
Resource-based policy	Quem pode invocar a função	Necessária para S3, SNS, API Gateway, EventBridge e outras contas chamarem a função (<code>lambda:InvokeFunction</code>)

Pacotes de implantação

- **Arquivo .zip**: até **50 MB compactado** no upload direto (maior que isso, envie pelo S3) e **250 MB descompactado**, incluindo layers.
- **Imagem de contêiner**: até **10 GB**, armazenada no **Amazon ECR**. A imagem precisa implementar a **Lambda Runtime API** (use as imagens base da AWS ou o Runtime Interface Client). Para testar localmente, use o **Runtime Interface Emulator**.
- **Layers**: .zip com bibliotecas extraídas em `/opt`; versionadas e compartilháveis entre contas.
- **Quando usar contêiner**: dependências grandes (bibliotecas de ML), ferramentas já baseadas em Docker, padronização com o resto da empresa.

Modelos de invocação e tratamento de erros

Invocação síncrona

- O chamador espera a resposta: `aws lambda invoke --invocation-type RequestResponse`, API Gateway, ALB, Lambda Function URLs, Cognito triggers, CloudFront (Lambda@Edge).
- **Erros voltam para o chamador**, que decide se tenta de novo. O Lambda **não** faz retry automático.
- Payload de **6 MB** na requisição e na resposta; com **response streaming**, respostas de até **200 MB**.
- Throttling devolve `429 TooManyRequestsException`.

Invocação assíncrona

- O Lambda coloca o evento em uma **fila interna** e responde **202** na hora: S3, SNS, EventBridge, SES, CloudFormation, CodeCommit, `--invocation-type Event`.
- **Retries automáticos**: até **2 novas tentativas** (3 execuções no total), com espera entre elas. Configurável de 0 a 2 (`MaximumRetryAttempts`).
- **Idade máxima do evento**: de 60 s a **6 horas** (`MaximumEventAgeInSeconds`). Eventos estrangulados (throttled) ou com erro de sistema ficam na fila e são tentados até esse limite.
- **Para onde vão as falhas**:
 - **Destinations** (recomendado): destino de **sucesso** e de **falha** (SQS, SNS, Lambda, EventBridge; falha também para S3). O registro inclui **o evento, a resposta/erro e metadados** da invocação.
 - **Dead-letter queue (DLQ)**: SQS ou SNS configurados na função; recebe **só o evento** que falhou, sem detalhes do erro.
- **Idempotência é obrigatória**: o mesmo evento pode ser processado mais de uma vez.
- Payload assíncrono de até **1 MB** (era 256 KB até 2025).

Event source mapping (poll-based)

O **Lambda busca** os registros e invoca a função com **lotes**. Fontes: **SQS**, **Kinesis Data Streams**, **DynamoDB Streams**, **Amazon MSK / Kafka autogerenciado**, **Amazon MQ**, **DocumentDB**.

Aspecto	SQS	Kinesis e DynamoDB Streams
Ordem	Standard: sem ordem; FIFO: por message group	Ordem por shard
Lote	Até 10.000 mensagens (Standard) com janela de lote (<code>MaximumBatchingWindowInSeconds</code>); FIFO até 10	Até 10.000 registros
O que acontece no erro	Mensagens voltam à fila quando o visibility timeout expira e são tentadas de novo; após <code>maxReceiveCount</code> vão para a DLQ da fila	O lote é tentado de novo até expirar ou até <code>MaximumRetryAttempts</code> / <code>MaximumRecordAgeInSeconds</code> , bloqueando o shard
Evitar reprocessar o lote todo	<code>ReportBatchItemFailures</code> (resposta parcial do lote com os IDs que falharam)	<code>ReportBatchItemFailures</code> , <code>BisectBatchOnFunctionError</code> (divide o lote ao meio para isolar o registro ruim)
Destino de falhas	DLQ da fila SQS (configurada na fila, não na função)	On-failure destination do mapping (SQS, SNS ou S3)
Escala	Até 1.000 execuções simultâneas por fila (limitável com <code>MaximumConcurrency</code> , 2 a 1.000)	Uma invocação por shard por vez; <code>ParallelizationFactor</code> de 1 a 10 lotes simultâneos por shard
Dica de configuração	Visibility timeout da fila $\geq 6 \times$ timeout da função	Monitore a métrica <code>IteratorAge</code> : crescendo significa que a função não acompanha o stream

- **Filtros de eventos:** o event source mapping aceita **filter criteria** para invocar a função só com registros que casam com um padrão, reduzindo custo e código.
- **Kinesis/DynamoDB Streams:** `StartingPosition` (`TRIM_HORIZON` desde o mais antigo, `LATEST` só os novos, `AT_TIMESTAMP`) e **tumbling windows** para agregações por janela de tempo.

Números de invocação e erros

Item	Valor
Timeout máximo	900 s (15 min)
Payload síncrono (requisição e resposta)	6 MB cada
Resposta com streaming	até 200 MB
Payload assíncrono	1 MB (era 256 KB)
Retries em invocação assíncrona	2 (configurável 0–2)
Idade máxima de evento assíncrono	60 s a 6 h
Lote máximo SQS Standard / FIFO	10.000 / 10
Parallelization factor (Kinesis/DynamoDB Streams)	1 a 10

Concorrência e escala

Tipos de concorrência

- **Concorrência** = número de requisições sendo processadas **ao mesmo tempo**. Estimativa: **requisições por segundo $\times$ duração média em segundos**. Ex.: $200 \text{ req/s} \times 0,5 \text{ s} = 100$ execuções simultâneas.

- **Cota da conta: 1.000** execuções simultâneas por Região (padrão, aumentável), compartilhadas por todas as funções.
- **Escala:** cada função pode adicionar até **1.000 ambientes de execução a cada 10 segundos**.
- **Concorrência reservada** (reserved concurrency):
 - **Garante** um número de execuções para a função **e limita** o máximo dela. Protege outras funções e dependências (ex.: um banco que aguenta 50 conexões).
 - Sem custo adicional.
 - **Reservada = 0** desliga a função (todas as invocações são estranguladas).
- **Concorrência provisionada** (provisioned concurrency):
 - Mantém ambientes **pré-inicializados**, eliminando cold starts. Tem custo.
 - Configurada em uma **versão publicada ou alias** (não em `$LATEST`) e pode escalar com Application Auto Scaling (agendado ou por utilização).
- **Throttling:** quando não há concorrência disponível, invocações síncronas recebem **429**; assíncronas são retidas e tentadas por até 6 horas; event source mappings reduzem o ritmo de leitura.

Cold start e desempenho

- **Causas de cold start:** primeiro uso, escala para novos ambientes, nova versão implantada, ambiente reciclado.
- **Como reduzir:**
 - Pacote menor e menos dependências.
 - Inicialização leve.
 - Mais memória (mais CPU).
 - **Provisioned concurrency.**
 - **SnapStart:** tira um snapshot do ambiente já inicializado e restaura a partir dele. Disponível para **Java, Python e .NET**; publicado por versão. Cuidado com dados que precisam ser únicos (conexões, números aleatórios): use os **runtime hooks** para recriá-los após a restauração.
- **Init Duration** aparece na linha `REPORT` do log apenas nos cold starts.

Versões, aliases e implantação

Versões e aliases

- `$LATEST`: versão editável. **Publicar uma versão** cria um snapshot **imutável** (código + configuração) com número sequencial (1, 2, 3...).
- **ARN qualificado** (com `:versão` ou `:alias`) vs. **não qualificado** (aponta para `$LATEST`).
- **Alias:** ponteiro com nome (`prod`, `dev`) para uma versão. Pode ser atualizado sem mudar quem chama.
- **Alias com pesos** (weighted alias / routing config): divide o tráfego entre **duas versões** (ex.: 90% v5, 10% v6), base para implantações **canary** e **linear** com o **CodeDeploy** ou o **SAM** (`DeploymentPreference`).
- **Stage variables do API Gateway** podem apontar para um alias (`function:minha-funcao:${stageVariables.alias}`); é preciso dar permissão de invocação para **cada alias**.
- Triggers, provisioned concurrency e permissões podem ser configurados por alias.

Lambda em VPC

- Configurar sub-redes e security groups faz a função criar **ENIs** (Hyperplane, compartilhadas) na VPC para acessar recursos privados: **RDS, ElastiCache**, instâncias EC2, APIs internas.

- A execution role precisa de permissões para gerenciar ENIs (política gerenciada `AWSLambdaVPCLambdaAccessExecutionRole`).
- **Uma função em VPC não tem acesso à internet** por padrão, mesmo em sub-rede pública. Para sair: sub-rede **privada** com rota para um **NAT Gateway**. Para serviços AWS (S3, DynamoDB, Secrets Manager, SQS), prefira **VPC endpoints**.
- **Banco relacional + Lambda**: use o **RDS Proxy** para agrupar e reutilizar conexões e evitar esgotar o limite de conexões com muitas execuções simultâneas; ele também suporta autenticação IAM e segredos do Secrets Manager.
- **Amazon EFS** pode ser montado na função (exige VPC) para arquivos compartilhados e grandes.

Outras formas de expor e executar funções

- **Lambda Function URLs**: endpoint HTTPS dedicado (`https://<id>.lambda-url.<região>.on.aws`) sem API Gateway. Autenticação `AWS_IAM` (SigV4) ou `NONE` (pública), com CORS configurável e suporte a response streaming. Não tem throttling por cliente, API keys nem transformações; para isso, use o API Gateway.
- **Lambda@Edge vs. CloudFront Functions**:

Aspecto	CloudFront Functions	Lambda@Edge
Linguagem	JavaScript	Node.js e Python
Eventos	Viewer request e viewer response	Viewer e origin request/response
Tempo	Submilissegundo	Até 5 s (viewer) ou 30 s (origin)
Acesso à rede e ao corpo da requisição	Não	Sim
Uso típico	Reescrever URLs, headers, redirecionamentos, validar tokens simples, em altíssima escala	Lógica mais pesada, chamar outros serviços, alterar respostas da origem

- **Durable functions**: funções que coordenam **fluxos de várias etapas de longa duração** em código, com checkpoints e espera sem pagar pelo tempo parado. Alternativa em código ao Step Functions.
- **Lambda Managed Instances**: executa funções em instâncias EC2 gerenciadas pelo Lambda (várias requisições por ambiente, opções de preço do EC2), para cargas estáveis e de alto volume.
- **Deteção de loop recursivo**: o Lambda interrompe automaticamente loops entre Lambda, SQS, SNS e S3 (ex.: função que grava no mesmo bucket que a dispara).

Integração e processamento em tempo quase real

- **S3 → Lambda**: processar uploads (miniaturas, validação). Use **prefixos/sufixos diferentes** ou outro bucket para a saída e evitar loops.
- **DynamoDB Streams → Lambda**: reagir a mudanças em itens (auditoria, sincronizar índices de busca, enviar notificações).
- **Kinesis → Lambda**: transformar e enriquecer streams (cliques, telemetria) em lotes.
- **Amazon Data Firehose + Lambda**: transformar registros antes da entrega ao S3/OpenSearch. A função recebe registros em `base64` e devolve cada um com `recordId`, `result` (`Ok`, `Dropped`, `ProcessingFailed`) e `data`.
- **SQS → Lambda**: processar filas com escala automática e resposta parcial de lote.
- **EventBridge (regras e Scheduler) → Lambda**: tarefas agendadas (substitui o cron em servidor) e reações a eventos de serviços AWS.

Números do Lambda

Métrica	Valor	Observação
Memória	128 MB a 10.240 MB	CPU proporcional; 1.769 MB ≈ 1 vCPU
Timeout	900 s	Padrão de 3 s
Concorrência padrão da conta	1.000 por Região	Aumentável
Escala por função	1.000 ambientes a cada 10 s	
Variáveis de ambiente	4 KB no total	
Layers	5	
Pacote .zip	50 MB compactado (upload direto) / 250 MB descompactado	Maior: via S3 ou contêiner
Imagem de contêiner	10 GB	No ECR
/tmp	512 MB a 10.240 MB	
Payload síncrono	6 MB	Streaming: 200 MB
Payload assíncrono	1 MB	⚠ Era 256 KB; muitos simulados ainda usam o valor antigo
Política baseada em recurso	20 KB	

Padrões e pegadinhas de Lambda

- "Função precisa de 20 minutos": **não** cabe no Lambda (limite de 15 min). Step Functions, ECS/Fargate, Batch ou durable functions.
- "Evento assíncrono falhou e precisamos do erro e do evento": **Destination de falha**; DLQ guarda só o evento.
- "Mensagens do SQS processadas mais de uma vez": visibility timeout **menor** que a duração da função → aumente para pelo menos 6× o timeout e torne o processamento idempotente.
- "Um registro ruim trava o processamento do Kinesis": `BisectBatchOnFunctionError`, `MaximumRetryAttempts`, `MaximumRecordAgeInSeconds` e on-failure destination.
- "Lote de 10 mensagens, 1 falhou, as 9 foram reprocessadas": `ReportBatchItemFailures`.
- "Função em VPC não acessa a internet": **NAT Gateway** em sub-rede pública e função em sub-rede privada (ou VPC endpoint para serviços AWS).
- "Muitas conexões abertas no RDS": **RDS Proxy**.
- "Cold start alto em Java": **SnapStart** ou **provisioned concurrency**.
- "Uma função consome toda a concorrência da conta": **reserved concurrency** nela (e nas críticas).
- "S3 não consegue invocar a função": falta a **resource-based policy** (`lambda:InvokeFunction` para `s3.amazonaws.com`); a execution role não resolve isso.
- "Função não consegue ler o bucket": falta permissão na **execution role**.
- "Trocar versão em produção sem mudar a configuração da API": **alias** e atualizar o alias.

Decisão rápida — Lambda

Cenário / requisito	Resposta correta	Distratores clássicos
Capturar evento e erro de invocações assíncronas com falha	Lambda Destination (on-failure)	DLQ, CloudWatch Logs

Cenário / requisito	Resposta correta	Distratores clássicos
Evitar reprocessar o lote inteiro do SQS	ReportBatchItemFailures	Reduzir batch size para 1
Registro inválido bloqueando o shard do Kinesis	Bisect batch + max retries + on-failure destination	Aumentar shards
Eliminar cold start em função crítica	Provisioned concurrency (ou SnapStart)	Aumentar timeout
Limitar uma função a 50 execuções simultâneas	Reserved concurrency = 50	Provisioned concurrency
Desligar uma função imediatamente	Reserved concurrency = 0	Apagar a função
Função precisa acessar RDS privado	Configurar VPC (sub-redes e SG) + RDS Proxy	Tornar o RDS público
Função em VPC precisa chamar API na internet	Sub-rede privada + NAT Gateway	Internet Gateway direto na função
Dividir 10% do tráfego para a nova versão	Alias com pesos (canary via CodeDeploy/SAM)	Duas funções separadas
Endpoint HTTPS simples para uma função sem API Gateway	Lambda Function URL	ALB
Dependências de 3 GB (bibliotecas de ML)	Imagem de contêiner no ECR	Layers
Compartilhar bibliotecas entre funções	Layers	Copiar em cada pacote
Processamento de 30 minutos	Step Functions / Fargate / Batch	Lambda com timeout máximo
Reescrever URLs na borda com latência mínima	CloudFront Functions	Lambda@Edge
Alterar a resposta da origem na borda chamando outro serviço	Lambda@Edge	CloudFront Functions
Muitas conexões simultâneas esgotando o banco	RDS Proxy	Aumentar memória
Transformar registros no Firehose antes de gravar no S3	Lambda de transformação do Firehose	Glue job

3. Amazon API Gateway

Regra de ouro da prova: **REST API** tem todos os recursos (cache, usage plans e API keys, validação, transformações, WAF, endpoint privado, canary); **HTTP API** é mais simples, mais barata e mais rápida (JWT nativo, sem cache nem usage plans); **WebSocket API** mantém conexões bidirecionais. **Stages** separam ambientes; **stage variables** tornam a configuração dinâmica; **deployments** publicam mudanças. Um erro **502** quase sempre é resposta malformada da Lambda proxy; **504** é timeout de integração; **429** é throttling.

Tipos de API e integrações

REST vs. HTTP vs. WebSocket

Recurso	REST API	HTTP API	WebSocket API
Custo e latência	Maior	Menor	Por mensagem e minuto de conexão
Autorização	IAM, Cognito User Pools, Lambda authorizer, resource policy	IAM, JWT authorizer (Cognito ou OIDC), Lambda authorizer	IAM, Lambda authorizer no \$connect
API keys e usage plans	Sim	Não	Não
Cache de respostas	Sim	Não	Não
Validação de requisição e mapping templates	Sim	Não (só mapeamento de parâmetros)	Sim (rotas)
AWS WAF	Sim	Não	Não
Endpoint privado	Sim	Não	Não
Canary release no stage	Sim	Não	Não
Response streaming	Sim	—	—
Uso típico	APIs públicas com controle de consumo e proteção	Proxy simples para Lambda ou HTTP, APIs internas econômicas	Chat, dashboards em tempo real, notificações

- **Tipos de endpoint (REST):** **edge-optimized** (entrada por CloudFront, clientes globais), **regional** (clientes na mesma Região ou com seu próprio CloudFront) e **private** (só acessível de uma VPC via interface endpoint `execute-api`).
- **WebSocket:** rotas `$connect`, `$disconnect`, `$default` e rotas personalizadas pela chave de roteamento; o backend envia mensagens ao cliente pela **callback URL** (`@connections`) usando o `connectionId`.

Tipos de integração

Integração	O que faz	Quando usar
Lambda proxy (<code>AWS_PROXY</code>)	Repassa a requisição inteira à função; a função devolve <code>statusCode</code> , <code>headers</code> e <code>body</code>	Padrão para APIs com Lambda; a lógica fica no código
Lambda não proxy (<code>AWS_CUSTOM</code>)	Usa mapping templates (VTL) para transformar requisição e resposta	Adaptar formatos sem mudar a função
HTTP proxy / HTTP	Encaminha a um endpoint HTTP (com ou sem transformação)	Expor um backend existente

Integração	O que faz	Quando usar
AWS service	Chama uma API da AWS diretamente (ex.: <code>SendMessage</code> no SQS, <code>PutItem</code> no DynamoDB, <code>StartExecution</code> no Step Functions)	Eliminar Lambdas "cola"; exige role de execução para a API
Mock	Devolve uma resposta fixa sem backend	Desenvolver o frontend antes do backend, testes, respostas de CORS
VPC Link / private integration	Alcança recursos privados (NLB, ALB, Cloud Map) na VPC	Backends em ECS/EC2 privados

Transformações, validação e códigos de status

- **Mapping templates** (Velocity Template Language, VTL) em integrações não proxy transformam o corpo e os parâmetros: `$input.json('$')`, `$input.path('$.campo')`, `$context.requestId`, `$context.identity.sourceIp`, `$stageVariables.nome`.
- **Validação de requisição: request validators** verificam parâmetros obrigatórios (query, headers, path) e o corpo contra um **model** (JSON Schema) **antes** de chamar o backend, devolvendo **400** sem custo de invocação.
- **Sobrescrever códigos de status:**
 - Em integração não proxy, use **integration responses** com **regex de seleção** sobre a mensagem de erro para mapear em `400`, `404` etc.
 - Em proxy, a própria função devolve o `statusCode`.
 - **Gateway responses** personalizam os erros gerados pelo próprio API Gateway (ex.: `UNAUTHORIZED`, `THROTTLED`, `DEFAULT_4XX`), inclusive para adicionar headers de CORS.
- **Formato exigido na Lambda proxy:** `{"statusCode": 200, "headers": {...}, "body": "string"}`. O body precisa ser **string** (JSON serializado). Formato errado → **502 Bad Gateway** ("Malformed Lambda proxy response").
- **CORS:** habilite no API Gateway (REST: método `OPTIONS` com mock; HTTP API: configuração de CORS). Em **Lambda proxy**, a **função também precisa devolver** os headers `Access-Control-Allow-Origin` nas respostas.

Stages, implantação e ambientes

Stages, deployments e stage variables

- **Deployment:** snapshot da configuração da API. **Mudanças só entram em vigor depois de implantar** (REST) em um stage. HTTP API pode ter **auto-deploy**.
- **Stage:** referência nomeada a um deployment (`dev`, `test`, `prod`), com URL própria (`https://{api-id}.execute-api.{região}.amazonaws.com/{stage}`), configurações de throttling, cache, logs e X-Ray.
- **Stage variables:** pares chave-valor por stage, usados em integrações, mapping templates e ARNs. Exemplo clássico: a integração aponta para `minha-funcao:${stageVariables.lambdaAlias}`; no stage `dev` a variável vale `dev` e em `prod` vale `prod`. É o que o guia chama de **"implantações dinâmicas com configurações de runtime existentes"**. Na Lambda, o valor chega em `event.stageVariables`.
- **Canary release** (REST): um stage envia uma **porcentagem do tráfego** a um deployment canary, com stage variables e logs próprios; depois se **promove** o canary.
- **Separar ambientes:** stages no mesmo API **ou** APIs/contas separadas por ambiente (mais isolamento, recomendado para produção).

Custom domains

- Nome de domínio próprio (`api.exemplo.com`) com certificado do **ACM**:

- **Edge-optimized:** certificado na **us-east-1** (N. Virgínia).
- **Regional:** certificado na **mesma Região** da API.
- **Base path mappings** (ou API mappings) ligam caminhos do domínio a APIs e stages (ex.: `/v1` → API A stage prod, `/v2` → API B).
- Registro no **Route 53** (alias) apontando para o domínio do API Gateway.
- **mTLS** pode ser habilitado em custom domains regionais para autenticar clientes por certificado.

Segurança, throttling e cache

Autorização no API Gateway

Mecanismo	Como funciona	Quando usar
IAM (SigV4)	O cliente assina a requisição; política do IAM concede <code>execute-api:Invoke</code>	Clientes AWS, serviços internos, usuários com credenciais temporárias (Cognito Identity Pools)
Cognito User Pool authorizer (REST)	Valida o token JWT do User Pool no header <code>Authorization</code>	Usuários do seu app autenticados no Cognito, sem código
JWT authorizer (HTTP API)	Valida JWT de qualquer provedor OIDC/OAuth 2.0 (Cognito, Auth0, Okta), incluindo escopos	APIs HTTP com provedor de identidade padrão
Lambda authorizer	Função que recebe o token (TOKEN) ou vários dados da requisição (REQUEST : headers, query, stage variables, contexto) e devolve uma política IAM e um <code>context</code>	Tokens personalizados, provedores próprios, regras de negócio na autorização; resultado em cache (padrão 300 s, até 3.600 s)
Resource policy (REST)	Política no próprio API: permitir contas, IPs, VPCs ou endpoints específicos	Restringir por IP, permitir outra conta, APIs privadas
API keys + usage plans	Identificam o cliente para throttling e cota	Não são mecanismo de autenticação por si só

- **Cache do Lambda authorizer:** se a política devolvida cobre só o recurso chamado e o cache está ativo, outras rotas com o mesmo token podem ser negadas; devolva políticas que cubram os recursos necessários ou use a identidade correta como chave de cache.

Throttling e usage plans

- **Limite da conta por Região: 10.000 requisições por segundo** com burst de **5.000** (algoritmo token bucket), compartilhado entre todas as APIs. Aumentável.
- Níveis de throttling: conta → stage/método → **usage plan por API key** (rate, burst e **cota** diária/semanal/mensal).
- Excedeu → **429 Too Many Requests**. O cliente deve fazer retry com backoff.
- **Usage plans** são a resposta para "planos Básico e Premium com limites diferentes por cliente".

Cache de respostas (REST)

- Cache por **stage**, com tamanho de 0,5 GB a 237 GB e TTL padrão de **300 s** (0 a 3.600 s; 0 desliga).
- Pode ser ajustado **por método**; as **chaves de cache** podem incluir headers, query strings e parâmetros de caminho.
- **Invalidação:** pelo console ou pelo cliente, com o header `Cache-Control: max-age=0` (exige a permissão `execute-api:InvalidCache`; configure para exigir autorização e não deixar qualquer cliente invalidar).
- Métricas `CacheHitCount` e `CacheMissCount` no CloudWatch.
- Cobrança por hora conforme o tamanho do cache.

Erros, timeouts e observabilidade

Códigos de erro mais cobrados

Código	Causa típica
400 Bad Request	Falha na validação da requisição (model ou parâmetros obrigatórios)
403 Forbidden	Autorização negada, Missing Authentication Token (rota ou método inexistente, ou URL do stage errada), WAF bloqueou, resource policy negou
429 Too Many Requests	Throttling (conta, stage, método ou usage plan) ou cota excedida
502 Bad Gateway	Resposta malformada da Lambda proxy , erro não tratado na integração, resposta incompatível
503 Service Unavailable	Backend indisponível
504 Gateway Timeout	Timeout de integração (padrão de 29 s em REST)

- **Timeout de integração:** 50 ms a 29 s por padrão. Em APIs REST **regionais e privadas**, pode ser aumentado além de 29 s por cota (possivelmente reduzindo o throttling da conta). HTTP API: até 30 s.
- **Response streaming** (REST, integrações `AWS_PROXY` e `HTTP_PROXY`): envia a resposta aos poucos, melhora o tempo até o primeiro byte, permite respostas maiores que 10 MB e integrações de até **15 minutos**.
- **Payload** máximo de **10 MB** sem streaming.

Logs, métricas e tracing

- **Execution logs** (CloudWatch Logs): detalhes de cada etapa da requisição, níveis `ERROR` ou `INFO`, com opção de logar o corpo completo. Úteis para depurar mapping templates e autorizadores. Exige uma role do API Gateway com permissão para gravar logs, configurada na conta.
- **Access logs:** uma linha por requisição em formato personalizável (JSON, CLF) com variáveis `$context` (IP, usuário, status, latência).
- **Métricas:** `Count`, `4XXError`, `5XXError`, `Latency` (tempo total no API Gateway) e `IntegrationLatency` (tempo do backend). Se `Latency` é alta mas `IntegrationLatency` é baixa, o gargalo está no próprio gateway (autorizador, transformação).
- **X-Ray:** ative o tracing no stage para ver o caminho até a Lambda e outros serviços.

Números do API Gateway

Métrica	Valor	Observação
Throttling da conta	10.000 req/s, burst 5.000	Por Região; aumentável
Timeout de integração (REST)	29 s	Aumentável em APIs regionais e privadas; streaming até 15 min
Timeout de integração (HTTP API)	30 s	
Payload	10 MB	Maior com response streaming
TTL do cache	300 s padrão, até 3.600 s	Só REST
Cache do Lambda authorizer	300 s padrão, até 3.600 s	
Certificado de domínio edge-optimized	ACM na us-east-1	Regional: mesma Região

Decisão rápida — API Gateway

Cenário / requisito	Resposta correta	Distratores clássicos
Limites e cotas diferentes por cliente pagante	Usage plans + API keys (REST)	Lambda authorizer, WAF
API simples e barata com JWT do Cognito	HTTP API + JWT authorizer	REST API com cache
Cache de respostas da API	REST API com cache no stage	HTTP API
Validar corpo da requisição sem invocar a Lambda	Request validator + model (JSON Schema)	Validar na função
Mesmo código apontando para aliases diferentes por ambiente	Stage variables com alias da Lambda	Uma API por função
Testar nova versão com 10% do tráfego da API	Canary release no stage	Criar outro domínio
API só acessível de dentro da VPC	Private REST API + interface endpoint <code>execute-api</code>	Resource policy em API regional pública
Autorização com token personalizado de terceiro	Lambda authorizer (TOKEN ou REQUEST)	API keys
Serviços AWS chamando a API com credenciais temporárias	Autorização IAM (SigV4)	API keys
Erro 502 após nova versão da Lambda proxy	Corrigir formato da resposta (<code>statusCode</code> , <code>body</code> <code>string</code>)	Aumentar timeout
Erro 504 em processamento de 45 s	Assíncrono (fila/Step Functions), aumentar cota de timeout (REST regional) ou streaming	Aumentar memória da API
Erro 429 para um cliente específico	Throttling/cota do usage plan	Erro de IAM
Frontend precisa de respostas antes de o backend existir	Integração mock	Lambda que devolve fixo
Gravar direto no SQS a partir da API sem Lambda	Integração AWS service	Lambda proxy
Domínio próprio para API edge-optimized	Custom domain + certificado ACM na us-east-1	Certificado na Região da API
Falta header CORS em resposta de Lambda proxy	A função deve retornar <code>Access-Control-Allow-Origin</code>	Só habilitar CORS no console
Ver tempo gasto só no backend	Métrica <code>IntegrationLatency</code>	<code>Latency</code>

4. Amazon DynamoDB

Regra de ouro da prova: no DynamoDB, **o acesso define o modelo**. Escolha uma **partition key de alta cardinalidade**, use **Query** (pela chave) e nunca **Scan** em caminho crítico, crie **GSIs** para outros padrões de acesso e calcule a capacidade: **1 RCU = 1 leitura fortemente consistente de até 4 KB/s** (ou 2 eventualmente consistentes); **1 WCU = 1 escrita de até 1 KB/s**; transações custam o **dobro**.

Modelo de dados e chaves

Tabelas, itens e chaves primárias

- **Tabela** → **itens** (até **400 KB** cada, incluindo nomes de atributos) → **atributos** (tipos: String, Number, Binary, Boolean, Null, List, Map, String Set, Number Set, Binary Set).
- Sem esquema fixo além da chave primária.
- **Chave primária simples:** só **partition key** (hash key). Cada valor identifica um item.
- **Chave primária composta:** **partition key + sort key** (range key). Vários itens compartilham a partition key e são ordenados pela sort key, permitindo consultas por intervalo (`begins_with`, `between`, `>`, `<`).
- **Partições:** a partition key passa por uma função hash que decide a partição física. Cada partição suporta até **3.000 RCU e 1.000 WCU** e cerca de 10 GB.

Partition keys de alta cardinalidade

- **Alta cardinalidade** = muitos valores distintos, acessados de forma **uniforme**. Exemplos bons: `userId`, `orderId`, `deviceId`.
- **Baixa cardinalidade** gera **partições quentes** (hot partitions) e throttling mesmo com capacidade total sobrando. Exemplos ruins: `status` ("ativo"/"inativo"), `data` do dia, `país`.
- **Técnicas para espalhar escrita:** acrescentar um **sufixo aleatório ou calculado** à chave (write sharding, ex.: `2026-09-29#7`), usar chaves compostas, e ler em paralelo os fragmentos.
- **Adaptive capacity** redistribui capacidade para partições mais acessadas, mas não resolve uma chave extremamente quente.

Índices secundários

Característica	Local Secondary Index (LSI)	Global Secondary Index (GSI)
Chave	Mesma partition key , sort key diferente	Qualquer partition key e sort key
Quando criar	Somente na criação da tabela	A qualquer momento
Quantidade	Até 5 por tabela	Até 20 por tabela (padrão)
Consistência	Eventual ou forte	Somente eventual
Capacidade	Usa a capacidade da tabela	Capacidade própria (em modo provisionado); se o GSI for estrangulado, as escritas na tabela também são
Limite de tamanho	Coleção de itens por partition key até 10 GB	Sem limite de coleção
Projeção	<code>KEYS_ONLY</code> , <code>INCLUDE</code> ou <code>ALL</code>	<code>KEYS_ONLY</code> , <code>INCLUDE</code> ou <code>ALL</code>

- **Projeção:** só os atributos projetados ficam no índice. Consultar atributos não projetados em um GSI exige outra leitura na tabela. Projete o necessário para o padrão de acesso.
- **Índice esperso:** um GSI só contém itens que **têm** o atributo da chave do índice. É útil para consultar um subconjunto (ex.: só pedidos com `pendente=true`).
- **Índices vetoriais:** o DynamoDB também passou a suportar **índices vetoriais** para busca por similaridade (até 5 por tabela), útil em aplicações de IA.

Leitura, escrita e consistência

Modelos de consistência

- **Eventualmente consistente** (padrão): pode não refletir uma escrita recém-concluída; custa **metade** de uma leitura forte.
- **Fortemente consistente** (`ConsistentRead=true`): reflete todas as escritas concluídas antes da leitura. Não disponível em GSIs nem em global tables com consistência eventual entre Regiões.
- **Transacional:** `TransactGetItems` / `TransactWriteItems` com isolamento ACID entre até **100 itens** (em uma ou várias tabelas da mesma conta e Região), com tamanho agregado de até **4 MB**; custa **2×** a capacidade.
- **Global tables:** replicação multi-Região ativa-ativa. Modo **MREC** (consistência eventual entre Regiões, "última escrita vence") ou **MRSC** (consistência **forte** entre Regiões, RPO zero).

Cálculo de capacidade

Operação	Unidade	Regra
Leitura fortemente consistente	1 RCU	Item de até 4 KB , 1 por segundo
Leitura eventualmente consistente	0,5 RCU	Item de até 4 KB (2 leituras por RCU)
Leitura transacional	2 RCU	Item de até 4 KB
Escrita padrão	1 WCU	Item de até 1 KB , 1 por segundo
Escrita transacional	2 WCU	Item de até 1 KB

- **Sempre arredonde o tamanho para cima** (blocos de 4 KB na leitura e de 1 KB na escrita) **antes** de multiplicar.
- **Exemplos:**
 - 10 leituras fortemente consistentes/s de itens de **6 KB** → 6 KB vira 2 blocos de 4 KB → **20 RCU**.
 - As mesmas 10 leituras eventualmente consistentes → **10 RCU**.
 - 10 leituras transacionais de 6 KB → **40 RCU**.
 - 5 escritas/s de itens de **2,5 KB** → 3 blocos de 1 KB → **15 WCU**.
 - 5 escritas transacionais de 2,5 KB → **30 WCU**.
 - 100 leituras eventualmente consistentes/s de 3 KB → 1 bloco × 100 ÷ 2 → **50 RCU**.
- **Query/Scan:** a capacidade consumida é calculada pelo **total de dados lidos** (soma dos itens, arredondada para 4 KB), não por item, e **antes** do filtro.

Modos de capacidade

- **On-demand:** paga por requisição, **sem planejar capacidade**, escala instantânea até os limites da tabela. Ideal para tráfego **imprevisível**, novas aplicações e cargas intermitentes. Pode definir um throughput máximo.

- **Provisionado**: define RCU e WCU (com **Auto Scaling** por utilização-alvo). Mais barato para tráfego **previsível e constante**; pode usar **capacidade reservada**.
- **Burst capacity**: capacidade não usada de até 5 minutos pode ser consumida em picos curtos.
- Pode-se trocar de modo periodicamente (há limites de trocas).

Query vs. Scan

Aspecto	Query	Scan
Como encontra	Pela partition key (obrigatória), opcionalmente com condição na sort key	Lê a tabela ou índice inteiro
Eficiência	Lê só os itens daquela chave	Consome capacidade de todos os itens lidos
Filtro (<code>FilterExpression</code>)	Aplicado depois da leitura; não reduz a capacidade consumida	Idem
Ordem	Pela sort key (<code>ScanIndexForward=false</code> inverte)	Sem ordem
Página	Até 1 MB por chamada; continue com <code>LastEvaluatedKey</code> → <code>ExclusiveStartKey</code>	Idem
Paralelismo	—	Parallel scan com <code>Segment</code> e <code>TotalSegments</code> para acelerar tabelas grandes

- **Reduza o custo de Scan**: `Limit` menor, `ProjectionExpression` (só os atributos necessários, reduz o tráfego mas não a capacidade), executar fora do horário de pico, ou trocar por Query com um GSI adequado.
- `FilterExpression` **não é índice**: se precisa filtrar sempre pelo mesmo atributo, crie um GSI.

Operações da API

Operações de item, condições e contadores

- `PutItem`: cria ou **substitui** o item inteiro.
- `UpdateItem`: altera atributos específicos (ou cria se não existir) com `UpdateExpression` (`SET`, `REMOVE`, `ADD`, `DELETE`).
- `GetItem`: lê um item pela chave primária completa.
- `DeleteItem`: remove um item.
- **Escritas condicionais** (`ConditionExpression`): a operação só acontece se a condição for verdadeira; caso contrário, `ConditionalCheckFailedException`. Exemplos:
 - `attribute_not_exists(pk)`: não sobrescrever item existente (idempotência).
 - `saldo >= :valor`: validar regra de negócio.
- **Bloqueio otimista** (optimistic locking): guardar um atributo `versao`; cada escrita exige `versao = :versaoLida` e incrementa. Se outro processo gravou antes, a condição falha e a aplicação relê e tenta de novo. Os mapeadores dos SDKs (como o `@DynamoDbVersionAttribute` do Enhanced Client em Java) automatizam isso.
- **Contador atômico**: `UpdateItem` com `ADD contador :1` ou `SET contador = contador + :1`, sem ler antes. Não é idempotente (um retry conta duas vezes).
- `ReturnValues` (`ALL_OLD`, `UPDATED_NEW`...) devolve o item antes ou depois; `ReturnConsumedCapacity` mostra a capacidade consumida.

Operações em lote e transações

Operação	Limites	Comportamento
BatchGetItem	Até 100 itens e 16 MB	Leituras em paralelo; itens não processados em <code>UnprocessedKeys</code>
BatchWriteItem	Até 25 itens (put ou delete) e 16 MB	Não suporta update nem condições; itens não processados em <code>UnprocessedItems</code> → reenviar com backoff
TransactWriteItems	Até 100 ações , 4 MB	Tudo ou nada; aceita condições; <code>ClientRequestToken</code> para idempotência (10 minutos)
TransactGetItems	Até 100 itens , 4 MB	Leitura consistente e isolada de vários itens

- **Lote não é transação:** parte pode falhar e parte não.
- **PartiQL:** sintaxe parecida com SQL (`SELECT` , `INSERT` , `UPDATE` , `DELETE`) para DynamoDB, com `ExecuteStatement` , `BatchExecuteStatement` e `ExecuteTransaction` . Um `SELECT` sem a chave vira **Scan**.

Recursos de ciclo de vida e integração

TTL, Streams, backups e exportação

- **TTL (Time to Live):** um atributo do tipo **Number** com o **timestamp Unix em segundos** marca a expiração. O DynamoDB apaga itens expirados em segundo plano, normalmente **em poucos dias, sem consumir WCU**. Itens expirados ainda podem aparecer em leituras até serem apagados; filtre-os. As exclusões por TTL aparecem no **Stream** como exclusões de serviço. Ideal para sessões, tokens, dados temporários e **gestão do ciclo de vida dos dados**.
- **DynamoDB Streams:** registro ordenado de mudanças por item, retido por **24 horas**. Tipos de visão: `KEYS_ONLY` , `NEW_IMAGE` , `OLD_IMAGE` , `NEW_AND_OLD_IMAGES` . Consumido por **Lambda** (event source mapping) para auditoria, replicação, notificações e atualização de índices de busca. Até 2 leitores simultâneos por shard.
- **Kinesis Data Streams for DynamoDB:** alternativa aos Streams com retenção maior e mais consumidores.
- **Backups: on-demand** (retidos até serem apagados) e **PITR** (point-in-time recovery), com período de recuperação configurável de **1 a 35 dias**. A restauração cria **uma nova tabela**.
- **Exportação para o S3** (completa ou incremental, sem consumir capacidade) e **importação do S3**, para analytics com Athena.

DAX e cache

- **DynamoDB Accelerator (DAX):** cache em memória **compatível com a API do DynamoDB** (mudança mínima no código: trocar o cliente), com latência de **microsegundos**.
 - Cache de itens (`GetItem` , `BatchGetItem`) e cache de queries (`Query` , `Scan`).
 - Write-through.
 - **Só leituras eventualmente consistentes** são servidas do cache; fortemente consistentes vão direto à tabela.
 - Roda em cluster dentro da VPC.
- **DAX vs. ElastiCache:** DAX para acelerar leituras do DynamoDB sem mudar a lógica; ElastiCache para cachear resultados agregados, respostas de APIs, sessões ou dados de várias fontes.

Serialização e persistência

Formatos e mapeamento de objetos

- **Formato de baixo nível** (DynamoDB JSON): cada valor vem com o tipo, por exemplo `{ "nome": { "S": "Ana" }, "idade": { "N": "30" } }`. Números trafegam como **string**.
- **Cientes de alto nível** convertem automaticamente entre objetos da linguagem e o formato do DynamoDB (**marshall/unmarshall**):
 - **Document Client** (`@aws-sdk/lib-dynamodb`) no JavaScript.
 - `boto3.resource('dynamodb').Table` e `TypeSerializer / TypeDeserializer` no Python.
 - **Enhanced Client** (e o antigo `DynamoDBMapper`) no Java, com anotações.
- **Boas práticas de serialização**:
 - Datas como **string ISO 8601** (ordenáveis na sort key) ou **número epoch** (obrigatório para TTL).
 - Dados binários em base64.
 - Objetos grandes no **S3** com a referência (chave) no item, respeitando o limite de 400 KB.
 - Comprimir atributos grandes quando fizer sentido.
- **Serialização em geral**: converter objetos em JSON (ou outro formato) para gravar em S3, filas e bancos, e reconstruí-los na leitura, validando o esquema.

Números e pegadinhas do DynamoDB

Números do DynamoDB

Métrica	Valor	Observação
Tamanho máximo do item	400 KB	Inclui nomes e valores dos atributos
RCU	1 leitura forte/s ou 2 eventuais/s de até 4 KB	Transacional: 2 RCU
WCU	1 escrita/s de até 1 KB	Transacional: 2 WCU
Por partição	3.000 RCU e 1.000 WCU	Chaves quentes estouram antes do total
LSI	5 por tabela, só na criação	Coleção de itens ≤ 10 GB
GSI	20 por tabela (padrão)	Só consistência eventual
Query/Scan	1 MB por página	<code>LastEvaluatedKey</code> para continuar
BatchGetItem	100 itens / 16 MB	
BatchWriteItem	25 itens / 16 MB	Sem update nem condição
Transações	100 itens / 4 MB	2× capacidade
Streams	Retenção de 24 h	
PITR	1 a 35 dias (configurável)	Restauração cria nova tabela
TTL	Epoch em segundos (Number)	Exclusão em poucos dias, sem custo de WCU

Padrões e pegadinhas de DynamoDB

- "Throttling em uma tabela com capacidade sobrando": **partição quente** (partition key de baixa cardinalidade) ou **GSI estrangulado**.

- "Consulta por um atributo que não é chave": **GSI**; Scan com filtro é a alternativa cara.
- "Leitura precisa refletir a última escrita": `ConsistentRead=true` na tabela ou LSI (não em GSI).
- "Dois usuários editam o mesmo item e um sobrescreve o outro": **bloqueio otimista** com atributo de versão e `ConditionExpression`.
- "Evitar criar duplicado": `PutItem` com `attribute_not_exists`.
- "Transferir saldo entre duas contas de forma atômica": `TransactWriteItems`.
- "Itens temporários apagados automaticamente sem custo": **TTL**.
- "Reagir a cada mudança na tabela": **DynamoDB Streams + Lambda**.
- "`BatchWriteItem` retornou itens": reenvie os `UnprocessedItems` com backoff exponencial.
- "Leitura em microssegundos sem reescrever a aplicação": **DAX**.
- "Criar um LSI em tabela existente": não é possível; crie um GSI ou recrie a tabela.

Decisão rápida — DynamoDB

Cenário / requisito	Resposta correta	Distratores clássicos
Tráfego imprevisível, sem planejar capacidade	Modo on-demand	Provisionado sem Auto Scaling
Tráfego estável e previsível com menor custo	Provisionado + Auto Scaling (e capacidade reservada)	On-demand
Nova consulta por outro atributo em tabela existente	Global Secondary Index	LSI, Scan com filtro
Consulta alternativa com a mesma partition key e leitura forte	LSI (definido na criação)	GSI
Evitar sobrescrita concorrente	Bloqueio otimista (versão + condição)	Transação para tudo
Operação tudo-ou-nada em vários itens	<code>TransactWriteItems</code>	<code>BatchWriteItem</code>
Gravar 25 itens em uma chamada	<code>BatchWriteItem</code>	<code>TransactWriteItems</code>
Expirar sessões automaticamente	TTL (epoch em segundos)	Lambda agendada apagando itens
Auditoria de cada alteração	DynamoDB Streams + Lambda	Scan periódico
Cache de leitura compatível com a API do DynamoDB	DAX	ElastiCache
Throttling em uma chave muito acessada	Melhorar cardinalidade / write sharding	Aumentar RCU da tabela
Continuar uma Query com mais de 1 MB	<code>LastEvaluatedKey</code> → <code>ExclusiveStartKey</code>	Aumentar Limit
Scan de tabela grande mais rápido	Parallel scan (<code>Segment</code> / <code>TotalSegments</code>)	Query sem chave
Recuperar a tabela ao estado de 3 dias atrás	PITR (restaura em nova tabela)	Streams
Replicação multi-Região com RPO zero	Global tables com MRSC	Global tables MREC, backups

5. Armazenamento, Bancos Relacionais e Cache

Regra de ouro da prova: S3 para objetos (uploads diretos do cliente com **presigned URLs**, arquivos grandes com **multipart upload**, eventos com **Event Notifications**); **RDS/Aurora** para relacional (com **RDS Proxy** para funções e **Secrets Manager** para credenciais); **ElastiCache** para cache (**lazy loading** ou **write-through + TTL**); **OpenSearch** para busca de texto e análise de logs.

Amazon S3 para desenvolvedores

Operações e recursos mais cobrados

- **Consistência: leitura forte após escrita** (read-after-write) para PUT e DELETE, inclusive em listagens.
- **Presigned URLs**: URL temporária assinada que permite a **quem não tem credenciais AWS** fazer **GET** (download) ou **PUT** (upload) de um objeto específico, **com as permissões de quem gerou a URL**. Validade de até **7 dias** quando gerada com credenciais de usuário IAM; limitada à duração da sessão quando gerada com credenciais temporárias (roles). Padrão para uploads **direto do navegador ou app mobile** para o S3, sem passar pelo backend.
- **Multipart upload**: divide o objeto em partes enviadas em paralelo (e reenviáveis individualmente).
 - **Recomendado acima de 100 MB; obrigatório acima de 5 GB** (limite do PUT único). Objeto máximo de 5 TB.
 - Partes de 5 MB a 5 GB, até 10.000 partes.
 - Uploads incompletos cobram armazenamento: limpe com **lifecycle rule** (`AbortIncompleteMultipartUpload`).
- **Byte-range fetches**: baixar faixas de bytes em paralelo (`Range: bytes=0-1048575`) para acelerar downloads ou ler só o cabeçalho de um arquivo.
- **S3 Transfer Acceleration**: uploads de longa distância pelas edge locations (endpoint `bucket.s3-accelerate.amazonaws.com`).
- **Desempenho**: pelo menos **3.500 PUT/COPY/POST/DELETE** e **5.500 GET/HEAD por segundo por prefixo**. Mais prefixos = mais paralelismo.
- **Event Notifications**: eventos (`s3:ObjectCreated:*`, `s3:ObjectRemoved:*` ...) com filtro de prefixo/sufixo para **SNS, SQS, Lambda** ou **EventBridge** (este último com filtros avançados, vários destinos e replay).
- **Versionamento, lifecycle** (transição de classe e expiração), **replicação** e **Object Lock** (seção de ciclo de vida abaixo).
- **CORS**: configure regras de CORS no bucket quando uma página de outro domínio acessa objetos diretamente pelo navegador.
- **Metadados e tags**: metadados de sistema (`Content-Type`, `Cache-Control`) e definidos pelo usuário (`x-amz-meta-*`); tags de objeto para lifecycle, permissões e custo.

Criptografia no S3

Opção	Quem gerencia a chave	Detalhes
SSE-S3	AWS (chave do S3)	Padrão em todo bucket desde 2023; AES-256; header <code>x-amz-server-side-encryption: AES256</code>

Opção	Quem gerencia a chave	Detalhes
SSE-KMS	Chave no AWS KMS (AWS managed aws/s3 ou customer managed)	Controle de acesso à chave, auditoria no CloudTrail , rotação; cada objeto gera chamadas ao KMS (sujeitas a cotas) → use S3 Bucket Keys para reduzir chamadas e custo
DSSE-KMS	KMS, com duas camadas de criptografia	Para requisitos regulatórios de dupla criptografia
SSE-C	O cliente fornece a chave em cada requisição	O S3 não guarda a chave; exige HTTPS
Client-side encryption	O cliente criptografa antes de enviar	S3 só guarda o texto cifrado; use o AWS Encryption SDK ou o S3 Encryption Client

- **Forçar HTTPS:** bucket policy negando requisições com `aws:SecureTransport = false`.
- **Forçar um tipo de criptografia:** bucket policy com condição em `s3:x-amz-server-side-encryption` ou `s3:x-amz-server-side-encryption-aws-kms-key-id`.
- **Erro de KMS ao baixar objeto:** quem lê precisa de permissão `kms:Decrypt` na chave, além de `s3:GetObject`.

Bancos relacionais

Amazon RDS e Aurora para desenvolvedores

- **Endpoints:**
 - RDS tem endpoint da instância primária e de cada read replica.
 - O Aurora tem **cluster endpoint** (escritor), **reader endpoint** (balanceia entre réplicas de leitura) e **custom endpoints**.
 - A aplicação deve **ler do reader endpoint** e **gravar no writer**.
- **Multi-AZ** (alta disponibilidade, failover automático de DNS) vs. **read replicas** (escala de leitura, replicação assíncrona). Após failover, a aplicação deve reconectar; **cache de DNS** longo atrasa a recuperação.
- **Conexões:** cada conexão consome memória no banco. Aplicações com muitos clientes curtos (Lambda) devem usar **RDS Proxy** (pool de conexões, failover mais rápido, IAM auth, segredos do Secrets Manager).
- **Autenticação IAM no banco** (MySQL e PostgreSQL): a aplicação gera um **token de autenticação** temporário (15 minutos) com o SDK, usando a role, sem senha no código.
- **Credenciais:** guarde usuário e senha no **Secrets Manager** com **rotação automática**.
- **Aurora Serverless v2:** capacidade que escala automaticamente; **RDS Data API** (Aurora) permite executar SQL por **HTTPS** sem gerenciar conexões, útil em Lambda.
- **Criptografia:** em repouso com KMS (definida na criação) e em trânsito com **SSL/TLS** (baixe o certificado CA do RDS e force SSL por parâmetro do banco).

Cache de dados

Amazon ElastiCache e estratégias de cache

- **Engines:** **Valkey**, **Redis OSS** (estruturas de dados, persistência, replicação, pub/sub, sorted sets para rankings, alta disponibilidade Multi-AZ) e **Memcached** (cache simples, multithread, sem persistência nem replicação). Modo **serverless** disponível.

Estratégia	Como funciona	Vantagens	Desvantagens
------------	---------------	-----------	--------------

Estratégia	Como funciona	Vantagens	Desvantagens
Lazy loading (cache-aside)	A aplicação lê do cache; em miss , lê do banco e grava no cache	Só guarda o que é lido; falha do cache não derruba a aplicação	Primeira leitura lenta (miss penalty); dados podem ficar desatualizados
Write-through	Toda escrita no banco também atualiza o cache	Cache sempre atualizado	Grava dados que talvez nunca sejam lidos; escrita mais lenta
TTL	Cada chave expira após um tempo	Limita dados desatualizados e o uso de memória	Pode gerar picos de misses ao expirar muitas chaves juntas

- **Combinação recomendada:** lazy loading + write-through + TTL (com um pouco de aleatoriedade no TTL para evitar expiração simultânea).
- **Invalidação:** apagar a chave ao atualizar o dado.
- **Sessões:** guardar sessões de usuário no ElastiCache (ou DynamoDB) torna a aplicação stateless.
- **Cache em memória na aplicação:** variáveis fora do handler da Lambda guardam dados entre invocações do mesmo ambiente (configuração, segredos com TTL curto), sem serviço adicional.

Armazenamentos especializados

OpenSearch, Athena, EFS e EBS

- **Amazon OpenSearch Service:** **busca de texto completo** (relevância, fuzzy, autocompletar), **análise de logs** e dashboards (OpenSearch Dashboards), busca vetorial. Padrão típico: **DynamoDB (fonte da verdade)** → **Streams** → **Lambda** → **OpenSearch** para busca, ou ingestão zero-ETL.
- **Amazon Athena:** SQL serverless sobre dados no **S3** (logs, exportações do DynamoDB), pago por dados escaneados; use formatos colunares e partições.
- **Amazon EFS:** sistema de arquivos NFS compartilhado entre instâncias, contêineres e funções Lambda.
- **Amazon EBS:** volume de bloco para uma instância EC2.

Padrão de acesso	Armazenamento indicado
Chave-valor com latência de milissegundos em qualquer escala	DynamoDB
Relacional com joins e transações complexas	RDS / Aurora
Busca de texto livre e relevância	OpenSearch Service
Objetos, arquivos, mídia, data lake	S3
Consultas SQL ad hoc sobre arquivos no S3	Athena
Cache e sessões com latência de microssegundos	ElastiCache
Relacionamentos complexos (grafos)	Neptune
Arquivos compartilhados entre computação	EFS

Gestão do ciclo de vida dos dados

- **S3 Lifecycle:** mover para classes mais baratas e expirar objetos e versões antigas; abortar multipart uploads incompletos.
- **DynamoDB TTL:** expirar itens.
- **CloudWatch Logs:** definir **período de retenção** por log group (o padrão é nunca expirar).
- **ECR lifecycle policies:** apagar imagens antigas ou sem tag.

- **Versões de Lambda e versões de aplicação do Elastic Beanstalk:** limpar as antigas (há limites de armazenamento e de versões).
- **Backups:** AWS Backup, snapshots do RDS e PITR do DynamoDB, com retenção definida.

Decisão rápida — Armazenamento e cache

Cenário / requisito	Resposta correta	Distratores clássicos
Usuário do app faz upload direto para o S3 sem credenciais AWS	Presigned URL (PUT)	Access key no app, bucket público
Upload de arquivo de 8 GB	Multipart upload	PUT único
Upload mais rápido a partir de outro continente	S3 Transfer Acceleration	CloudFront para download
Processar cada objeto enviado	S3 Event Notification → Lambda/SQS/EventBridge	Lambda agendada listando o bucket
Auditado uso da chave de criptografia dos objetos	SSE-KMS	SSE-S3
Reduzir chamadas e custo de KMS com SSE-KMS	S3 Bucket Keys	Trocar para SSE-C
Criptografar antes de enviar ao S3	Client-side encryption	SSE-S3
Chave fornecida pelo cliente a cada requisição	SSE-C (com HTTPS)	SSE-KMS
Muitas funções Lambda esgotam conexões do RDS	RDS Proxy	Aumentar a instância
Conectar ao banco sem senha no código	Autenticação IAM do RDS ou Secrets Manager	Senha em variável de ambiente
Distribuir leituras entre réplicas do Aurora	Reader endpoint	Cluster endpoint
Cache que só guarda o que é lido	Lazy loading	Write-through
Cache sempre atualizado após escrita	Write-through	Lazy loading
Limitar dados desatualizados no cache	TTL	Aumentar o cache
Sessões compartilhadas entre instâncias	ElastiCache (ou DynamoDB)	Sticky sessions
Busca de texto com relevância sobre dados do DynamoDB	Streams → Lambda → OpenSearch	Scan com <code>contains</code>
SQL ad hoc sobre logs no S3	Athena	Carregar no RDS

6. Mensageria, Eventos, Streaming e Orquestração

Regra de ouro da prova: **SQS** = fila, o consumidor **puxa** e apaga; **SNS** = pub/sub, **empurra** para vários assinantes; **EventBridge** = barramento de eventos com **regras e padrões**, integração com SaaS, arquivo e replay; **Kinesis Data Streams** = stream **ordenado por shard**, com **vários consumidores** e **replay**; **Step Functions** = **orquestração** de etapas com retries, tratamento de erros e estado visível.

Amazon SQS

Standard vs. FIFO

Característica	Standard	FIFO
Entrega	Ao menos uma vez (pode duplicar)	Exatamente uma vez no processamento (deduplicação)
Ordem	Melhor esforço	Garantida por message group (<code>MessageGroupId</code>)
Throughput	Praticamente ilimitado	300 transações/s por ação (3.000 mensagens/s com lotes de 10); modo de alto throughput muito maior
Deduplicação	Não	<code>MessageDeduplicationId</code> ou deduplicação por conteúdo (hash do corpo), janela de 5 minutos
Nome	Qualquer	Precisa terminar em <code>.fifo</code>
Uso	Máximo volume, consumidores idempotentes	Ordem importa (transações financeiras, comandos em sequência)

- **Message group:** mensagens do mesmo grupo são processadas em ordem, uma de cada vez; grupos diferentes são processados em paralelo. Use o ID do cliente ou do pedido como grupo para paralelizar mantendo a ordem por entidade.

Ciclo de vida da mensagem

1. O produtor chama `SendMessage` (ou `SendMessageBatch`, até 10 mensagens).
2. O consumidor chama `ReceiveMessage` (até 10 mensagens por chamada) e recebe um **receipt handle**.
3. A mensagem fica **invisível** pelo **visibility timeout** enquanto é processada.
4. Ao terminar, o consumidor chama `DeleteMessage` com o receipt handle. **Se não apagar**, a mensagem volta a ficar visível e é entregue de novo.

Configuração	Valor	Detalhes
Visibility timeout	Padrão 30 s, de 0 s a 12 h	Deve ser maior que o tempo de processamento. Estenda durante o processamento com <code>ChangeMessageVisibility</code>
Retenção	Padrão 4 dias , de 60 s a 14 dias	Mensagens mais antigas são apagadas
Tamanho da mensagem	Até 1 MiB	⚠ Era 256 KB até ago/2025. Maior: Extended Client Library (payload no S3, até 2 GB)
Long polling	<code>WaitTimeSeconds</code> de 1 a 20 s	Reduz respostas vazias e custo; configure na fila (<code>ReceiveMessageWaitTimeSeconds</code>) ou na chamada

Configuração	Valor	Detalhes
Delay queue	0 a 15 minutos	Atrasa a entrega de todas as mensagens da fila; message timers atrasam mensagens individuais (Standard)
Dead-letter queue (DLQ)	maxReceiveCount na redrive policy	Após N recebimentos sem exclusão, a mensagem vai para a DLQ. DLQ de fila FIFO precisa ser FIFO. Use redrive para devolver mensagens à fila de origem depois de corrigir o problema
Mensagens em processamento	Cerca de 120.000 (Standard) e 20.000 (FIFO)	Limite de mensagens recebidas e não apagadas

- **Short polling** (padrão, `WaitTimeSeconds=0`) consulta só alguns servidores e pode devolver vazio mesmo com mensagens; **long polling** consulta todos e espera.
- **Retenção da DLQ** deve ser **maior** que a da fila de origem, porque o timestamp original da mensagem é mantido.
- **Criptografia**: SSE-SQS (padrão) ou SSE-KMS; **política de acesso** da fila para permitir SNS, S3 ou outras contas enviarem mensagens.
- **Escalar consumidores**: Auto Scaling de EC2/ECS pela métrica `ApproximateNumberOfMessagesVisible` (ou backlog por instância); Lambda escala automaticamente. `ApproximateAgeOfOldestMessage` alto indica consumidores lentos.

Amazon SNS

Tópicos, assinaturas e fan-out

- **Tópico**: canal onde publicadores enviam mensagens. **Assinaturas**: SQS, Lambda, HTTP/HTTPS, e-mail, SMS, push mobile, Amazon Data Firehose.
- **Fan-out SNS** → **SQS**: publicar uma vez e entregar a **várias filas**, cada uma com seu consumidor, retries e DLQ. É o padrão clássico para processar o mesmo evento de formas diferentes (ex.: pedido criado → fila de faturamento, fila de estoque, fila de e-mail). As filas precisam de **política de acesso** permitindo `sqs:SendMessage` do tópico.
- **Filter policies**: cada assinatura recebe **só as mensagens que casam com o filtro**, aplicado aos **message attributes** (padrão) ou ao **corpo** da mensagem (`FilterPolicyScope: MessageBody`). Evita que consumidores recebam e descartem mensagens, reduzindo custo e código. É o que o guia chama de "usar políticas de filtro de assinatura para otimizar a mensageria".
- **Tópicos FIFO**: ordem e deduplicação, entregando a filas SQS; nome terminado em `.fifo`.
- **Entrega e falhas**: políticas de retry por protocolo (HTTP/S configurável); **DLQ por assinatura** (fila SQS) para mensagens que não puderam ser entregues.
- **Tamanho**: até **1 MiB** configurando o atributo `MaximumMessageSize` do tópico (antes 256 KiB). Para payloads maiores, use o **Extended Client Library** com S3.
- **SNS vs. SQS**: SNS não guarda mensagens para consumo posterior; se o assinante está fora do ar, depende de retries e DLQ. Para garantir processamento, coloque uma fila SQS como assinante.

Amazon EventBridge

Barramentos, regras e destinos

- **Event bus**: **default** (eventos de serviços AWS), **custom** (eventos da sua aplicação, enviados com `PutEvents`) e **partner** (eventos de SaaS como Zendesk, Datadog, Shopify).

- **Estrutura do evento:** `source`, `detail-type`, `detail` (JSON do negócio), `account`, `region`, `time`, `resources`. Tamanho máximo de **1 MB** (era 256 KB até jan/2026).
- **Regras** com **event patterns** que filtram por qualquer campo (valores exatos, prefixo, sufixo, números, `anything-but`, `exists`, wildcard). Cada regra pode ter até **5 destinos**: Lambda, SQS, SNS, Step Functions, Kinesis, API Gateway, ECS tasks, outro event bus (inclusive de outra conta ou Região), **API destinations** (endpoints HTTP de terceiros com autenticação e limite de taxa).
- **Input transformer:** reformata o evento antes de entregá-lo ao destino.
- **Retries e DLQ:** por padrão, o EventBridge tenta entregar por **até 24 horas e 185 tentativas** com backoff; falhas definitivas vão para uma **DLQ** (SQS) configurada no destino.
- **Archive e replay:** arquivar eventos e **reproduzi-los** depois, para reprocessar após corrigir um bug ou para testar.
- **Schema registry:** descobre os esquemas dos eventos e gera **bindings de código** (classes tipadas) para Java, Python, TypeScript e Go.
- **EventBridge Pipes:** integração ponto a ponto **fonte** → **filtro** → **enriquecimento** → **destino** (ex.: SQS/DynamoDB Streams/Kinesis → filtrar → chamar Lambda/API para enriquecer → Step Functions), sem código de cola.
- **EventBridge Scheduler:** agendamentos únicos ou recorrentes (cron e rate) com fuso horário, janela flexível, retries e DLQ, em escala de milhões de agendamentos. Substitui cron em servidores.

Padrões orientados a eventos com EventBridge

- **Coreografia:** cada microsserviço publica eventos de domínio (`PedidoCriado`, `PagamentoAprovado`) em um barramento custom e assina os de interesse, sem conhecer os demais.
- **Integração entre contas:** enviar eventos a barramentos de outras contas com política de recurso no barramento de destino.
- **Reagir a serviços AWS:** mudança de estado do EC2, conclusão de build do CodeBuild, eventos do S3, chamadas de API registradas pelo CloudTrail.
- **EventBridge vs. SNS:** EventBridge tem filtragem mais rica pelo conteúdo, mais destinos, integração com SaaS, schema registry, archive e replay; SNS tem maior fan-out por tópico, menor latência e entrega a e-mail, SMS e push mobile.

Streaming de dados

Amazon Kinesis Data Streams

- **Shards:** unidade de capacidade. Cada shard aceita **1 MB/s ou 1.000 registros/s de escrita** e fornece **2 MB/s de leitura** (compartilhados entre consumidores padrão, com até 5 `GetRecords` por segundo).
- **Partition key:** define o shard do registro. **Ordem garantida dentro do shard**. Chave de baixa cardinalidade gera **hot shard** e `ProvisionedThroughputExceededException`.
- **Modos:** **provisionado** (você define os shards e faz resharding: split/merge) ou **on-demand** (escala automaticamente).
- **Retenção:** padrão de **24 horas**, até **365 dias**. Consumidores podem **reler** (replay) desde qualquer ponto dentro da retenção.
- **Tamanho do registro:** até **10 MiB** (era 1 MB até out/2025; muitos simulados usam o valor antigo). Os dados chegam em **base64** no evento da Lambda.
- **Produtores:** SDK (`PutRecord`, `PutRecords` em lote), **Kinesis Producer Library (KPL)** com agregação e retries, Kinesis Agent.
- **Consumidores:**
 - **Lambda** (event source mapping).

- **Kinesis Client Library (KCL)**, que usa uma tabela do **DynamoDB** para coordenar leases e checkpoints; no máximo **um worker por shard**.
- **Enhanced fan-out**: cada consumidor recebe **2 MB/s por shard** dedicados, via push (`SubscribeToShard`), com menor latência.
- **Amazon Data Firehose** e **Managed Service for Apache Flink**.
- **Kinesis vs. SQS**: Kinesis quando é preciso **ordem, vários consumidores** do mesmo dado e **replay**; SQS quando cada mensagem é processada por **um** consumidor e depois apagada.

Amazon Data Firehose

- Entrega gerenciada e serverless de streams para **S3, Redshift, OpenSearch, Splunk, endpoints HTTP** e outros, com buffer por tamanho ou tempo (quase tempo real), conversão de formato (JSON → Parquet), compressão e **transformação com Lambda**.
- **Não guarda nem reprocessa** dados: para replay, coloque o Kinesis Data Streams antes.

Orquestração e APIs de dados

AWS Step Functions

- **Máquina de estados** definida em **Amazon States Language** (JSON), com fluxo visual (Workflow Studio).
- **Tipos de estado**:
 - **Task**: executa trabalho (Lambda, chamada a serviço AWS, atividade).
 - **Choice**: desvio condicional.
 - **Parallel**: ramos em paralelo.
 - **Map**: itera sobre itens; o **Distributed Map** processa milhões de objetos do S3 com até 10.000 execuções filhas em paralelo.
 - **Wait**: espera um tempo ou até uma data.
 - **Pass, Succeed e Fail**.

Característica	Standard	Express
Duração máxima	1 ano	5 minutos
Semântica	Exatamente uma vez	Ao menos uma vez (assíncrono) ou no máximo uma vez (síncrono)
Histórico	Completo e auditável no console (90 dias)	No CloudWatch Logs
Preço	Por transição de estado	Por número de execuções, duração e memória
Uso	Processos longos, auditáveis, com espera humana	Alto volume e curta duração (ingestão de eventos, IoT, processamento de streams)

- **Tratamento de erros**:
 - `Retry`: por tipo de erro (`ErrorEquals`), com `IntervalSeconds`, `MaxAttempts`, `BackoffRate`, `MaxDelaySeconds` e `JitterStrategy`.
 - `Catch`: desvia para um estado de tratamento ou **compensação** (padrão **saga**), guardando o erro em `ResultPath`.
 - **Erros predefinidos**: `States.ALL`, `States.Timeout`, `States.TaskFailed`, `States.Permissions`.
 - `TimeoutSeconds` e `HeartbeatSeconds` evitam tarefas penduradas.

- **Padrões de integração com serviços:**
 - **Request-response** (padrão): chama o serviço e segue sem esperar o fim do trabalho.
 - **Run a job** (`.sync`): espera o trabalho terminar (ex.: job do Glue, task do ECS, outra execução do Step Functions).
 - **Wait for callback** (`.waitForTaskToken`): pausa até alguém chamar `SendTaskSuccess` ou `SendTaskFailure` com o **task token**. Usado para **aprovação humana**, sistemas externos ou filas.
- **Integrações otimizadas e AWS SDK integrations:** chamar mais de 200 serviços **diretamente**, sem Lambda de cola (DynamoDB `PutItem`, SQS `SendMessage`, SNS `Publish`, Bedrock `InvokeModel`).
- **Processamento de dados:** `JSONPath` (`InputPath`, `Parameters`, `ResultSelector`, `ResultPath`, `OutputPath`) ou **JSONata** com **variáveis** (`Assign`), que simplificam as transformações.
- **Payload** entre estados de até **256 KB**; para dados maiores, guarde no S3 e passe a referência.
- **Testes:** **TestState API** testa um estado isoladamente, sem implantar a máquina.

AWS AppSync

- Serviço gerenciado de **APIs GraphQL** (e **AppSync Events**, APIs pub/sub em WebSocket) que combina várias fontes de dados em uma única API.
- **Data sources:** DynamoDB, Lambda, HTTP, Aurora/RDS, OpenSearch, EventBridge e Bedrock. **Resolvers** em JavaScript (ou VTL) mapeiam campos às fontes; **pipeline resolvers** encadeiam funções.
- **Subscriptions em tempo real** via WebSocket: clientes recebem atualizações quando uma mutation acontece (chat, placares, dashboards).
- **Autorização:** API key, IAM, **Cognito User Pools**, OIDC e Lambda, combináveis por campo.
- **Cache** no servidor e suporte offline em apps com o Amplify.
- **Quando usar:** o cliente precisa buscar **exatamente os campos** de que precisa, de **várias fontes**, em uma chamada, ou precisa de **atualizações em tempo real**.

Números e comparação de mensageria

Números de mensageria e streaming

Serviço	Métrica	Valor
SQS	Tamanho da mensagem	1 MiB (Extended Client: até 2 GB via S3)
SQS	Retenção	60 s a 14 dias (padrão 4 dias)
SQS	Visibility timeout	0 s a 12 h (padrão 30 s)
SQS	Long polling	Até 20 s
SQS	Delay	Até 15 min
SQS	Lote de envio/recebimento	10 mensagens
SQS FIFO	Throughput	300 TPS por ação (3.000 msg/s com lotes); mais em alto throughput
SQS FIFO	Janela de deduplicação	5 min
SNS	Tamanho da mensagem	Até 1 MiB (atributo <code>MaximumMessageSize</code>)
EventBridge	Tamanho do evento	1 MB
EventBridge	Retries padrão	24 h / 185 tentativas
EventBridge	Destinos por regra	5

Serviço	Métrica	Valor
Kinesis	Escrita por shard	1 MB/s ou 1.000 registros/s
Kinesis	Leitura por shard	2 MB/s (compartilhado) ou 2 MB/s por consumidor com enhanced fan-out
Kinesis	Retenção	24 h a 365 dias
Kinesis	Tamanho do registro	Até 10 MiB
Step Functions	Duração Standard / Express	1 ano / 5 min
Step Functions	Payload entre estados	256 KB

Qual serviço escolher

Requisito	Serviço
Um consumidor por mensagem, desacoplamento, picos	SQS Standard
Ordem e processamento exatamente uma vez	SQS FIFO
Mesma mensagem para vários sistemas	SNS (fan-out para SQS)
Roteamento por conteúdo, SaaS, replay de eventos	EventBridge
Tarefas agendadas	EventBridge Scheduler
Stream ordenado, vários consumidores e replay	Kinesis Data Streams
Entregar stream no S3/Redshift/OpenSearch sem código	Amazon Data Firehose
Fluxo com várias etapas, erros e compensação	Step Functions
API GraphQL com dados de várias fontes e tempo real	AppSync
Migrar aplicação que usa JMS, AMQP ou MQTT	Amazon MQ

Decisão rápida — Mensageria e orquestração

Cenário / requisito	Resposta correta	Distratores clássicos
Mensagem processada duas vezes porque o processamento demora	Aumentar visibility timeout (ou <code>ChangeMessageVisibility</code>)	Aumentar retenção
Reduzir respostas vazias e custo de polling	Long polling (<code>WaitTimeSeconds</code> até 20)	Short polling mais frequente
Mensagens que sempre falham travando o consumidor	DLQ com <code>maxReceiveCount</code>	Aumentar retenção
Garantir ordem por cliente com paralelismo entre clientes	SQS FIFO com <code>MessageGroupId</code> = ID do cliente	SQS Standard
Evitar duplicados enviados em 5 minutos	FIFO com <code>MessageDeduplicationId</code> ou deduplicação por conteúdo	Standard
Mensagem de 5 MB no SQS	Extended Client Library (payload no S3)	Aumentar o limite da fila
Atrasar o processamento de cada mensagem em 10 min	Delay queue (ou message timer)	Visibility timeout
Assinante só deve receber pedidos acima de R\$ 1.000	SNS filter policy (ou regra do EventBridge)	Filtrar no consumidor
Eventos de SaaS disparando fluxos na AWS	EventBridge partner event bus	SNS
Reprocessar eventos da semana passada após corrigir bug	EventBridge archive e replay (ou Kinesis dentro da retenção)	SQS

Cenário / requisito	Resposta correta	Distratores clássicos
Classes tipadas para os eventos	EventBridge schema registry (code bindings)	Escrever à mão
Fonte → filtro → enriquecimento → destino sem código	EventBridge Pipes	Lambda de cola
Vários consumidores leem o mesmo stream com baixa latência	Kinesis enhanced fan-out	Mais shards
ProvisionedThroughputExceededException no Kinesis	Melhorar a partition key, adicionar shards ou on-demand, retry com backoff	Aumentar retenção
Aprovação humana no meio do fluxo	Step Functions <code>.waitForTaskToken</code>	Lambda em loop
Fluxo de alto volume com duração de segundos	Step Functions Express	Standard
Processo de negócio de dias, auditável	Step Functions Standard	Express
Chamar DynamoDB direto do fluxo sem Lambda	Integração AWS SDK do Step Functions	Lambda de cola
Atualizações em tempo real no app via GraphQL	AppSync subscriptions	Polling na API REST

7. Autenticação e Autorização

Regra de ouro da prova: **autenticação** responde "quem é você?"; **autorização** responde "o que você pode fazer?". Para **usuários do seu aplicativo**, use **Cognito User Pools** (login e tokens JWT). Para dar a esses usuários **credenciais AWS temporárias**, use **Cognito Identity Pools**. Para **código rodando na AWS**, use **roles do IAM** (execution role, task role, instance profile). Para **outra conta**, **assuma uma role** com STS.

IAM para desenvolvedores

Políticas e avaliação

- **Estrutura de uma declaração:** `Effect` (`Allow` / `Deny`), `Action` (ex.: `dynamodb:GetItem`), `Resource` (ARN), `Condition` (opcional) e, em políticas de recurso, `Principal`.
- **Políticas baseadas em identidade** (anexadas a usuário, grupo ou role) vs. **baseadas em recurso** (anexadas ao recurso: bucket policy, política de fila SQS, resource policy da Lambda, key policy do KMS, política de segredo, resource policy do API Gateway).
- **Lógica de avaliação** dentro da mesma conta:
 1. Tudo começa **negado** (deny implícito).
 2. Um **Deny** **explícito** em qualquer política vence.
 3. Um **Allow** em política de identidade **ou** de recurso concede o acesso.
 4. SCPs, **permission boundaries** e políticas de sessão só **limitam**.
- **Entre contas:** é preciso **permissão dos dois lados**, na política de identidade de quem chama e na política de recurso (ou na trust policy da role) de quem é chamado.
- **Permission boundary:** define o **máximo** de permissões que uma identidade pode ter; útil para deixar desenvolvedores criarem roles sem escalar privilégios.
- **Condições comuns:** `aws:SourceIp`, `aws:SecureTransport`, `aws:PrincipalTag/...`, `aws:RequestedRegion`, `aws:SourceArn` / `aws:SourceAccount` (evitar o problema do *confused deputy* em permissões dadas a serviços), `dynamodb:LeadingKeys`, `s3:prefix`, `kms:ViaService`.
- **Variáveis de política:** `${aws:username}`, `${aws:PrincipalTag/tenant}`, `${cognito-identity.amazonaws.com:sub}`, que permitem uma única política servir a muitos usuários (cada um acessa só o próprio prefixo ou item).
- **Ferramentas:** **IAM Policy Simulator** (testar se uma ação é permitida), **IAM Access Analyzer** (gerar política de menor privilégio a partir do CloudTrail, validar políticas, encontrar acessos externos), `aws sts decode-authorization-message`.

Roles para cada tipo de computação

Onde o código roda	Como recebe credenciais
EC2	Instance profile com uma role; credenciais pelo IMDS (use IMDSv2)
Lambda	Execution role (o que a função pode fazer) + resource-based policy (quem pode invocá-la)
ECS	Task role (permissões da aplicação) e task execution role (permissões do agente do ECS : puxar imagem do ECR, enviar logs, buscar segredos para injetar no contêiner)
EKS	EKS Pod Identity ou IAM Roles for Service Accounts (IRSA)

Onde o código roda	Como recebe credenciais
CodeBuild / CodePipeline / CodeDeploy	Service roles de cada serviço
Elastic Beanstalk	Service role + instance profile das instâncias
Máquina local do desenvolvedor	IAM Identity Center (<code>aws configure sso</code>) ou perfil que assume role
On-premises	IAM Roles Anywhere (certificados X.509)

AWS STS e acesso programático

Operações do STS

API	Para quê
<code>AssumeRole</code>	Assumir uma role na mesma conta ou em outra conta ; devolve access key, secret key e session token temporários. Duração de 15 min até o <code>MaxSessionDuration</code> da role (máx. 12 h); role chaining limita a 1 h
<code>AssumeRoleWithWebIdentity</code>	Trocar um token OIDC (Google, Facebook, GitHub Actions, Kubernetes) por credenciais AWS. Para apps, prefira Cognito
<code>AssumeRoleWithSAML</code>	Trocar uma asserção SAML de um IdP corporativo por credenciais
<code>GetSessionToken</code>	Credenciais temporárias para um usuário IAM, geralmente com MFA , para chamar APIs que exigem MFA (<code>aws:MultiFactorAuthPresent</code>)
<code>GetFederationToken</code>	Credenciais temporárias para usuários federados por um broker próprio
<code>GetCallerIdentity</code>	Descobrir conta e ARN da identidade atual
<code>DecodeAuthorizationMessage</code>	Decodificar mensagens de falha de autorização

- **Acesso entre contas:**

1. A conta B cria uma role com **trust policy** confiando na conta A (e, para terceiros, exigindo um `sts:ExternalId`).
2. A identidade da conta A recebe permissão `sts:AssumeRole` nessa role.
3. O código chama `AssumeRole` e usa as credenciais devolvidas.

- **Session tags e políticas de sessão:** passar tags (`sts:TagSession`) ou uma política inline ao assumir a role restringe ou contextualiza a sessão, por exemplo com o ID do tenant.

- **Configurar acesso programático:**

- Para pessoas: **IAM Identity Center** com credenciais temporárias na CLI.
- Para cargas na AWS: **roles**.
- Access keys de usuário IAM só quando não houver alternativa, com rotação e privilégio mínimo.

Amazon Cognito

User Pools vs. Identity Pools

Aspecto	User Pools	Identity Pools (Federated Identities)
Função	Diretório de usuários e autenticação: cadastro, login, MFA, recuperação de senha	Credenciais AWS temporárias para usuários (autenticados ou convidados)
Resultado	Tokens JWT (ID, access e refresh)	Access key, secret key e session token do STS
Federação	Login social (Google, Apple, Facebook, Amazon), SAML e OIDC	Aceita tokens do User Pool, provedores sociais, SAML, OIDC e identidades autenticadas pelo seu backend (developer-authenticated)
Uso típico	Proteger APIs (API Gateway, ALB, AppSync) e o próprio app	Acessar S3, DynamoDB e outros serviços direto do app
Convidados	Não	Sim (role para não autenticados)

- **Fluxo comum:** o usuário faz login no **User Pool** → recebe JWT → troca o ID token no **Identity Pool** → recebe credenciais temporárias de uma role → acessa o S3 diretamente, **limitado ao próprio prefixo** com a variável `${cognito-identity.amazonaws.com:sub}` na política.
- **Role mapping** no Identity Pool: escolher a role por regras (claims do token) ou pelo grupo do User Pool (`cognito:preferred_role`).

Tokens, fluxos e recursos do User Pool

- **ID token:** informações do **usuário** (claims como `email`, `cognito:groups`, atributos personalizados). Usado pelo **frontend** e pelo **Cognito User Pool authorizer** do API Gateway (REST).
- **Access token:** o que o portador **pode fazer** (`scope`, `cognito:groups`). Usado para **autorizar chamadas a APIs** (OAuth 2.0) e operações do próprio usuário no Cognito.
- **Refresh token:** obtém novos ID e access tokens sem novo login (validade configurável, padrão 30 dias).
- **Validação de JWT** no backend: verificar a **assinatura** com as chaves públicas do endpoint **JWKS** do User Pool, o emissor (`iss`), o público (`aud / client_id`), o tipo de token (`token_use`) e a **expiração** (`exp`).
- **Managed login / Hosted UI:** páginas de login prontas com domínio próprio, suporte a **OAuth 2.0**:
 - **Authorization code com PKCE:** apps web e mobile.
 - **Client credentials: máquina a máquina**, com resource servers e escopos personalizados.
- **Lambda triggers:** pre sign-up (validar ou autoconfirmar), post confirmation (criar perfil no banco), pre authentication, **pre token generation** (adicionar claims e escopos ao token, ex.: `tenantId`), custom message, migrate user (migrar usuários de outro diretório no primeiro login), custom auth challenge.
- **Grupos:** aparecem no claim `cognito:groups` e podem ter uma role do IAM associada.
- **MFA** (SMS, TOTP, e-mail) e proteção contra ameaças (senhas comprometidas, autenticação adaptativa).
- **Planos de recursos:** **Lite**, **Essentials** e **Plus** (o Plus inclui proteção contra ameaças).
- *O **Cognito Sync** (sincronização de dados do usuário) entrou em modo de manutenção em 30/jul/2026; para sincronizar dados entre dispositivos, use **AppSync**.*

Autorização na aplicação e entre serviços

Bearer tokens

- Um **bearer token** (geralmente um JWT OAuth 2.0) é enviado no header `Authorization: Bearer <token>`; **quem o possui** tem o acesso. Por isso:
 - Use sempre **HTTPS**.
 - Use tokens de **curta duração**, com refresh tokens.
 - Nunca registre tokens em logs.
 - Valide assinatura, emissor, público e expiração em cada requisição.
- **No API Gateway**: Cognito User Pool authorizer (REST), **JWT authorizer** (HTTP API) ou Lambda authorizer para tokens próprios.

Autorização de granularidade fina

- **No token**: escopos OAuth e grupos (`cognito:groups`) verificados pelo API Gateway ou pelo código.
- **Lambda authorizer**: devolve uma política IAM por usuário e um `context` (ex.: `tenantId`, papel) repassado à integração.
- **Amazon Verified Permissions**: serviço de autorização da aplicação baseado em políticas **Cedar** ("o usuário X pode editar o documento Y se for dono ou do mesmo departamento"), que separa as regras de permissão do código.
- **IAM com condições**, para acesso direto do cliente a serviços AWS:
 - `dynamodb:LeadingKeys`: o usuário só acessa itens cuja partition key é o seu ID.
 - `dynamodb:Attributes`: restringe as colunas acessíveis.
 - Prefixos do S3 com variáveis de política.
- **AppSync**: autorização por campo e filtros com a identidade do usuário.

Autenticação entre serviços em microsserviços

- **Roles do IAM + SigV4**: o serviço A assume sua role e chama o serviço B (API Gateway com autorização IAM, Lambda Function URL com `AWS_IAM`, chamada direta à Lambda) com uma política que permite exatamente essa chamada. É o padrão na AWS, porque não há segredos para gerenciar.
- **Políticas de recurso**: a função ou a API do serviço B lista quais roles ou contas podem chamá-la.
- **OAuth 2.0 client credentials** (Cognito): serviços obtêm um access token com escopos e o enviam ao outro serviço. Útil quando o chamador não está na AWS.
- **mTLS**: certificados de cliente (API Gateway custom domain, ALB) emitidos por uma **AWS Private CA**.
- **Propagação do contexto do usuário**: repassar o token ou claims do usuário original (e o ID de correlação do tracing) para que o serviço B aplique as regras daquele usuário.
- **Segredos compartilhados** (chaves de API entre serviços): guardados no **Secrets Manager** com rotação, nunca no código.

Aplicações multi-tenant

Modelo	Como isola	Prós e contras
Silo	Recursos (ou contas) separados por tenant	Isolamento forte e custo por tenant claro; mais caro e complexo de operar

Modelo	Como isola	Prós e contras
Pool	Recursos compartilhados , com o <code>tenantId</code> em cada item/objeto	Mais eficiente; exige isolamento rigoroso no código e no IAM
Bridge	Mistura: alguns recursos compartilhados, outros dedicados	Equilíbrio conforme o nível do cliente

• **Padrões de acesso a dados multi-tenant:**

- `tenantId` como **prefixo da partition key** no DynamoDB.
- Prefixo por tenant no S3.
- Credenciais de sessão com escopo de tenant: `AssumeRole` **com session tags** ou política de sessão e condições como `dynamodb:LeadingKeys = ${aws:PrincipalTag/tenantId}`.
- Claim `custom:tenantId` no token do Cognito (via pre token generation).
- **Usage plans por nível** de tenant no API Gateway.

- **Nunca confie** em um `tenantId` enviado pelo cliente no corpo da requisição: obtenha-o do **token validado**.

Decisão rápida — Autenticação e autorização

Cenário / requisito	Resposta correta	Distratores clássicos
Cadastro e login dos usuários do app com login social	Cognito User Pool	IAM users, Identity Pool sozinho
App mobile acessa o S3 diretamente com credenciais temporárias	Cognito Identity Pool	Access keys embutidas no app
Usuários convidados (sem login) acessam recursos limitados	Identity Pool com acesso não autenticado	User Pool
Cada usuário só acessa o próprio prefixo no S3	Política com <code>\${cognito-identity.amazonaws.com:sub}</code>	Um bucket por usuário
Cada usuário só acessa os próprios itens no DynamoDB	Condição <code>dynamodb:LeadingKeys</code>	Filtrar no código do cliente
Adicionar <code>tenantId</code> aos tokens	Lambda trigger pre token generation	Atributo no corpo da requisição
Serviço sem usuário (M2M) chamando uma API protegida por Cognito	OAuth 2.0 client credentials	Usuário fictício com senha
Código em EC2 precisa acessar DynamoDB	Instance profile (role)	Access keys no arquivo de configuração
Contêiner ECS precisa gravar no S3	Task role	Task execution role
ECS precisa puxar imagem do ECR e ler segredo para injetar	Task execution role	Task role
Acessar recursos de outra conta	<code>AssumeRole</code> com trust policy (e External ID para terceiros)	Criar usuário IAM na outra conta
Chamar API que exige MFA pela CLI	<code>GetSessionToken</code> com MFA	<code>AssumeRoleWithWebIdentity</code>
Workflow do GitHub Actions implanta na AWS sem chaves	OIDC + <code>AssumeRoleWithWebIdentity</code>	Access keys em secrets do repositório
Regras de permissão complexas por recurso na aplicação	Amazon Verified Permissions (Cedar)	IAM para cada usuário final
Validar JWT no backend	Verificar assinatura (JWKS), <code>iss</code> , <code>aud</code> , <code>token_use</code> , <code>exp</code>	Só decodificar o payload
Descobrir se uma política permite a ação	IAM Policy Simulator	Tentar em produção
Gerar política de menor privilégio pelo uso real	IAM Access Analyzer (a partir do CloudTrail)	AdministratorAccess

8. Criptografia e Dados Sensíveis

Regra de ouro da prova: dados **grandes** são criptografados com **envelope encryption**: `GenerateDataKey` devolve uma chave de dados em texto claro (usada localmente e descartada) e a mesma chave **criptografada** (guardada junto dos dados). A chamada `Encrypt` do KMS aceita **no máximo 4 KB**. **Segredos** vão para o **Secrets Manager** (rotação automática) ou **Parameter Store** (`SecureString`, sem rotação nativa), nunca no código.

Criptografia em repouso e em trânsito

Conceitos

- **Em repouso** (at rest): dados gravados em disco ou armazenamento (S3, EBS, RDS, DynamoDB, SQS, logs). Normalmente com chaves do **KMS**.
- **Em trânsito** (in transit): dados trafegando na rede, protegidos por **TLS** (HTTPS) ou VPN. Os endpoints das APIs AWS usam TLS; force HTTPS em buckets (`aws:SecureTransport`), APIs e load balancers.
- **Criptografia do lado do servidor** (server-side): o serviço AWS criptografa ao gravar e descriptografa ao ler, de forma transparente para quem tem permissão (SSE-S3, SSE-KMS, criptografia do DynamoDB e do RDS).
- **Criptografia do lado do cliente** (client-side): a **aplicação criptografa antes de enviar**; o serviço só vê o texto cifrado. Use o **AWS Encryption SDK** (envelope encryption pronto, com chaves do KMS e cache de chaves de dados) ou o **Amazon S3 Encryption Client** e o **DynamoDB Encryption Client** (AWS Database Encryption SDK).
- **Quando usar client-side**: requisito de que nem o serviço de armazenamento veja os dados em claro, ou criptografia de atributos específicos (ex.: CPF em um item do DynamoDB).

AWS KMS

Tipos de chave

Tipo	Quem gerencia	Detalhes
AWS owned keys	AWS, usadas por vários clientes	Invisíveis na conta; sem custo e sem controle
AWS managed keys (<code>aws/s3</code> , <code>aws/lambda</code> ...)	AWS, na sua conta	Visíveis e auditáveis, mas a key policy não pode ser alterada ; não servem para uso entre contas ; rotação automática anual
Customer managed keys	Você	Key policy, grants, rotação, habilitar/desabilitar e agendar exclusão (7 a 30 dias); necessárias para compartilhar entre contas

- **Simétricas** (AES-256, padrão; a chave nunca sai do KMS) vs. **assimétricas** (RSA/ECC para criptografia ou assinatura, com chave pública exportável) vs. **HMAC** (gerar e verificar códigos de autenticação de mensagens).
- **Multi-Region keys**: chaves com o mesmo material em várias Regiões, para descriptografar dados replicados sem chamar outra Região.
- **Aliases** (`alias/app-prod`) apontam para uma chave e permitem trocá-la sem mudar o código.

Operações e envelope encryption

API	O que faz
Encrypt	Criptografa até 4 KB de dados diretamente com a chave do KMS (senhas, pequenos segredos)
Decrypt	Descriptografa; o texto cifrado já indica qual chave usar (simétrica)
ReEncrypt	Troca a chave que protege um texto cifrado sem expor o texto claro
GenerateDataKey	Devolve uma chave de dados em texto claro e criptografada
GenerateDataKeyWithoutPlaintext	Devolve só a chave de dados criptografada (para uso futuro)
GenerateRandom	Gera bytes aleatórios criptograficamente seguros
Sign / Verify	Assinatura digital com chaves assimétricas

- **Envelope encryption**, passo a passo:
 1. Chame `GenerateDataKey` com a chave do KMS.
 2. Use a **chave de dados em claro** para criptografar os dados localmente (AES).
 3. **Descarte** a chave em claro da memória.
 4. Guarde a **chave de dados criptografada** junto com os dados.
 5. Para ler: `Decrypt` na chave de dados criptografada → descriptografe os dados localmente.
- **Por que envelope**: o KMS só aceita 4 KB em `Encrypt`, cada chamada tem latência e custo, e há **cotas de requisições por segundo** (compartilhadas na conta e Região). Com envelope, o KMS só é chamado uma vez por chave de dados.
- `ThrottlingException` **do KMS**: reduzir chamadas com **cache de chaves de dados** (AWS Encryption SDK), **S3 Bucket Keys**, retries com backoff ou pedir aumento de cota.
- **Encryption context**: pares chave-valor **não secretos** ligados ao texto cifrado (dados autenticados adicionais). O mesmo contexto precisa ser informado no `Decrypt`; aparece no **CloudTrail**, ajuda na auditoria e pode ser exigido em condições da key policy.

Key policies, grants e uso entre contas

- **Toda chave tem uma key policy**. Sem uma declaração permitindo a conta (a padrão permite o root da conta e, por consequência, as políticas do IAM), **nenhuma política do IAM funciona** para aquela chave.
- **Grants**: permissões temporárias e programáticas para um principal usar a chave (muito usadas por serviços como EBS e RDS); podem ser revogadas.
- **Condição** `kms:ViaService`: permite usar a chave só por meio de um serviço específico (ex.: só pelo S3).
- **Uso entre contas**:
 1. A **key policy** da conta A permite a conta (ou role) B.
 2. Uma **política do IAM na conta B** concede as ações (`kms:Decrypt`, `kms:GenerateDataKey` ...) na chave da conta A.
 3. Só com **customer managed keys**.
 - Exemplos: compartilhar snapshots criptografados, bucket com SSE-KMS acessado por outra conta, artefatos do CodePipeline entre contas.

Rotação de chaves

- **Customer managed keys simétricas: rotação automática** que pode ser habilitada e desabilitada, com período configurável de **90 a 2.560 dias** (padrão **365 dias**), além de **rotação sob demanda**. O KMS guarda o material antigo: dados antigos continuam descriptografáveis, e **o ID e o ARN da chave não mudam**.
- **AWS managed keys**: rotação automática **anual**, que não pode ser alterada.
- **Sem rotação automática**: chaves **assimétricas, HMAC**, com **material importado** e em custom key stores. Para rotacionar: criar uma nova chave e **apontar o alias** para ela (**rotação manual**), mantendo a antiga para descriptografar dados antigos.
- **Desabilitar a rotação** não afeta dados já criptografados.

Certificados e chaves para desenvolvimento

ACM, AWS Private CA e chaves SSH

- **AWS Certificate Manager (ACM)**: certificados **públicos** TLS gratuitos para ELB, CloudFront, API Gateway e App Runner, com **renovação automática** (validação por DNS recomendada). Para o CloudFront e APIs edge-optimized, o certificado precisa estar na **us-east-1**. Certificados **importados** não são renovados automaticamente (o ACM avisa pelo EventBridge antes de expirar).
- **AWS Private CA**: autoridade certificadora **privada** gerenciada, para emitir certificados internos: **mTLS** entre microsserviços, dispositivos IoT, TLS em serviços internos, assinatura de código. Os certificados privados podem ser gerenciados pelo ACM.
- **Certificados para desenvolvimento**: certificados **autoassinados** com OpenSSL (ex.: `openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 365`) servem para testes locais, mas **não** são confiáveis para clientes externos. Em ambientes de teste compartilhados, prefira o ACM ou a Private CA.
- **Chaves SSH**:
 - **Pares de chaves do EC2**, criados no console, com `aws ec2 create-key-pair` ou importados (`ssh-keygen -t ed25519` e `import-key-pair`). A AWS guarda só a chave pública.
 - **EC2 Instance Connect** envia uma chave pública temporária (60 s).
 - **Systems Manager Session Manager** dispensa chaves SSH e portas abertas.
- **Chaves de Git (CodeCommit)**: chave SSH pública associada ao usuário IAM, credenciais HTTPS Git ou o helper `git-remote-codecommit` com credenciais temporárias.

Gestão de segredos e configuração sensível

Secrets Manager vs. Parameter Store

Recurso	AWS Secrets Manager	SSM Parameter Store
Finalidade	Segredos : senhas de banco, chaves de API, tokens	Configuração e segredos simples
Rotação automática	Sim , com função Lambda (rotação gerenciada para RDS, Aurora, Redshift, DocumentDB)	Não nativa
Criptografia	Sempre, com KMS	<code>SecureString</code> com KMS (<code>String</code> e <code>StringList</code> em claro)
Tamanho	Até 64 KB	Standard 4 KB; Advanced 8 KB
Custo	Por segredo/mês e por chamadas de API	Standard gratuito (até 10.000 parâmetros); Advanced pago

Recurso	AWS Secrets Manager	SSM Parameter Store
Recursos extras	Replicação entre Regiões , compartilhamento entre contas por resource policy, geração de senhas aleatórias	Hierarquias (/app/prod/db/url) com <code>GetParametersByPath</code> , versões e labels , políticas de expiração (Advanced)
Integração	CloudFormation (<code>{{resolve:secretsmanager:...}}</code>), ECS, Lambda, RDS	CloudFormation (<code>{{resolve:ssm:...}}</code> , <code>{{resolve:ssm-secure:...}}</code>), ECS, CodeBuild, AppConfig

- **Rotação no Secrets Manager:** a função de rotação cria a nova versão (label `AWSPENDING`), atualiza o banco, testa e promove para `AWSCURRENT`; a anterior vira `AWSPREVIOUS`. A aplicação deve **ler o segredo em tempo de execução** (com cache) para pegar a versão atual.
- **Compartilhar segredo entre contas:** resource policy no segredo + **customer managed key** do KMS com permissão para a outra conta (a chave `aws/secretsmanager` não pode ser compartilhada).
- **Parameter Store referenciando o Secrets Manager:** `/aws/reference/secretsmanager/nome-do-segredo`.
- **Cache de segredos:**
 - Bibliotecas de cache do Secrets Manager (Java, Python, .NET, Go).
 - **AWS Parameters and Secrets Lambda Extension:** servidor HTTP local em `localhost:2773` que busca e guarda em cache parâmetros e segredos, reduzindo latência, custo e throttling.

Variáveis de ambiente com dados sensíveis

- As variáveis de ambiente do Lambda são **criptografadas em repouso** com uma chave AWS managed por padrão; é possível usar uma **customer managed key**.
- **Helpers de criptografia em trânsito:** o console pode criptografar o valor com uma chave do KMS **antes** de salvar; o código então chama `kms:Decrypt` na inicialização (a execution role precisa dessa permissão). O valor não aparece em claro no console nem na API `GetFunctionConfiguration`.
- **Melhor prática:** guardar no ambiente só o **nome/ARN** do segredo e buscar o valor no Secrets Manager ou Parameter Store (com cache), o que permite rotação sem reimplantar.
- **ECS:** injetar segredos no contêiner com `secrets` → `valueFrom` apontando para Secrets Manager ou Parameter Store na task definition (a **task execution role** precisa de permissão). **CodeBuild:** seções `parameter-store` e `secrets-manager` no `buildspec.yml`. **Elastic Beanstalk:** propriedades de ambiente, com segredos referenciados do Secrets Manager/Parameter Store.

Classificação, sanitização e mascaramento

Classificação de dados

- **PII** (Personally Identifiable Information): dados que identificam uma pessoa (nome, CPF, e-mail, telefone, endereço, IP).
- **PHI** (Protected Health Information): informações de saúde ligadas a uma pessoa (sujeitas a HIPAA nos EUA).
- **Outras categorias:** dados financeiros e de cartão (PCI DSS), credenciais, dados confidenciais do negócio.
- A classificação define os controles: criptografia, acesso, retenção, mascaramento e onde os dados podem aparecer (nunca em logs, URLs ou mensagens de erro).
- **Descoberta:** **Amazon Macie** encontra dados sensíveis no S3; **Amazon Comprehend** detecta PII em texto.

Sanitizar e mascarar dados na aplicação

- **Sanitização de entrada:** validar tipo, tamanho e formato, escapar ou rejeitar caracteres perigosos, usar consultas parametrizadas (contra SQL injection) e codificar saída (contra XSS). Validação no **API Gateway** (models) e no código.
- **Mascaramento na saída:** mostrar só parte do dado (`***.***.123-45`), remover campos sensíveis das respostas da API e dos eventos.
- **Logs sem dados sensíveis:**
 - Não registrar tokens, senhas, números de cartão nem o corpo inteiro das requisições.
 - Use logs estruturados com **lista de campos permitidos**.
 - **CloudWatch Logs data protection policies** detectam e **mascaram** PII nos log groups automaticamente (ver o dado original exige a permissão `logs:Unmask`).
 - O **Powertools for AWS Lambda** tem utilitário de **data masking** (apagar ou criptografar campos antes de registrar ou enviar).
- **Tokenização e criptografia de campo:** substituir o dado sensível por um token ou criptografá-lo no lado do cliente, para que só serviços autorizados vejam o valor real.
- **Pseudonimização** em ambientes de teste: nunca copie dados de produção com PII para desenvolvimento sem anonimizar.

Decisão rápida — Criptografia e dados sensíveis

Cenário / requisito	Resposta correta	Distratores clássicos
Criptografar um arquivo de 50 MB com o KMS	Envelope encryption com <code>GenerateDataKey</code>	<code>Encrypt</code> direto
Criptografar uma senha de 1 KB	<code>kms:Encrypt</code>	<code>GenerateDataKey</code> obrigatório
Trocar a chave de um texto cifrado sem vê-lo em claro	<code>ReEncrypt</code>	<code>Decrypt</code> + <code>Encrypt</code> no cliente
<code>ThrottlingException</code> do KMS em alto volume	Cache de chaves de dados / S3 Bucket Keys / retry com backoff	Criar mais chaves
Outra conta precisa descriptografar dados	Customer managed key: key policy + política IAM na outra conta	AWS managed key
Rotação automática de chave a cada 180 dias	Customer managed key com período de rotação de 180 dias	Rotação manual obrigatória
Rotacionar chave assimétrica ou com material importado	Rotação manual: nova chave e troca do alias	Habilitar rotação automática
Exigir que a chave só seja usada pelo S3	Condição <code>kms:ViaService</code> na key policy	SCP
Dados não podem ser vistos em claro pelo serviço de armazenamento	Client-side encryption (AWS Encryption SDK)	SSE-S3
Certificados para mTLS entre microserviços internos	AWS Private CA	ACM público
Certificado HTTPS com renovação automática para o ALB	ACM	Certificado autoassinado
Senha do banco com rotação automática	Secrets Manager	Parameter Store, variável de ambiente
Configurações hierárquicas por ambiente sem custo	Parameter Store Standard (<code>/app/prod/...</code>)	Secrets Manager
Reduzir latência e custo ao ler segredos em Lambda	Parameters and Secrets Lambda Extension (cache)	Ler a cada invocação sem cache
Segredo em variável de ambiente não pode aparecer em claro no console	Helpers de criptografia em trânsito + <code>kms:Decrypt</code> no código	Base64

Cenário / requisito	Resposta correta	Distratores clássicos
Injetar segredo em contêiner ECS	<code>secrets.valueFrom</code> na task definition + task execution role	Variável em texto na imagem
Mascarar CPF e e-mail que aparecem nos logs	CloudWatch Logs data protection policy	Apagar os logs
Encontrar PII em buckets S3	Amazon Macie	Inspector

9. Empacotamento, Infraestrutura como Código e Ambientes

Regra de ouro da prova: SAM é CloudFormation com atalhos para serverless (Transform: AWS::Serverless-2016-10-31), testes locais e implantação gradual (DeploymentPreference). **CloudFormation** é a base: templates, stacks, **change sets**, outputs e exports. **CDK** gera CloudFormation a partir de código. **AppConfig** muda configuração e feature flags **sem reimplantar** o código.

Preparar artefatos de implantação

Dependências, estrutura e requisitos

- **Dependências no pacote:**
 - Lambda .zip: bibliotecas na raiz do pacote ou em **layers**.
 - Lambda contêiner e ECS: no **Dockerfile**, com versões fixadas (requirements.txt, package-lock.json).
 - Evite incluir SDKs e bibliotecas de desenvolvimento desnecessárias; **sam build** resolve dependências por função.
- **Configuração fora do pacote:** variáveis de ambiente, Parameter Store, Secrets Manager e AppConfig. O mesmo artefato deve ser promovido de dev para prod mudando só a configuração ("build once, deploy many").
- **Estrutura de diretórios típica de um projeto SAM:**

```
meu-app/  
├── template.yaml      # infraestrutura (SAM)  
├── samconfig.toml     # parâmetros de deploy por ambiente  
├── src/  
│   ├── pedidos/app.py # código de cada função  
│   └── pedidos/requirements.txt  
├── events/            # eventos JSON de teste  
├── tests/unit/ e tests/integration/  
└── buildspect.yml     # build no CodeBuild
```

- **Elastic Beanstalk:** bundle .zip com o código na raiz, Procfile, pasta .ebextensions/ (configurações .config) e .platform/ (hooks e configuração do proxy).
- **CodeDeploy (EC2/on-premises):** appspec.yml **na raiz** do bundle, com os scripts dos hooks.
- **Contêineres:** imagem no **ECR** com **tags imutáveis** e versionadas (ex.: hash do commit), nunca só latest em produção.
- **Requisitos de recursos:**
 - Lambda: memória e timeout.
 - ECS: cpu e memory na task definition (Fargate exige combinações válidas), mais os limites por contêiner.
 - EC2 e Beanstalk: tipo de instância.
 - Declare-os no IaC para que sejam versionados junto com o código.
- **Repositórios de código:**
 - **CodeCommit** (novamente disponível para novos clientes desde nov/2025).

- **GitHub, GitLab e Bitbucket** conectados por **AWS CodeConnections** (antigo CodeStar Connections).
- O push em um branch dispara o pipeline.

AWS SAM

Template e recursos do SAM

- **Transform:** `AWS::Serverless-2016-10-31` transforma os recursos simplificados em CloudFormation na implantação.

Recurso SAM	O que cria
<code>AWS::Serverless::Function</code>	Função Lambda + role + event sources (<code>Events</code> : Api, HttpApi, S3, SQS, SNS, Schedule, DynamoDB, Kinesis, EventBridgeRule...)
<code>AWS::Serverless::Api</code> / <code>HttpApi</code>	API Gateway REST ou HTTP com stages, autorizadores e CORS
<code>AWS::Serverless::SimpleTable</code>	Tabela DynamoDB com chave primária simples
<code>AWS::Serverless::LayerVersion</code>	Layer do Lambda
<code>AWS::Serverless::StateMachine</code>	Máquina de estados do Step Functions
<code>AWS::Serverless::Application</code>	Aplicação aninhada (do Serverless Application Repository ou local)
<code>AWS::Serverless::Connector</code>	Permissões entre recursos (ex.: função → tabela) sem escrever IAM

- **Globals** : propriedades comuns a todas as funções (runtime, timeout, memória, variáveis de ambiente, tracing).
- **Policy templates**: permissões prontas de menor privilégio, como `DynamoDBCrudPolicy`, `S3ReadPolicy`, `SQSPollerPolicy`, `SNSPublishMessagePolicy`.
- **AutoPublishAlias** + **DeploymentPreference** : publica uma nova versão a cada deploy, atualiza o alias e usa o **CodeDeploy** para deslocar o tráfego:
 - Tipos: `Canary10Percent5Minutes`, `Canary10Percent10Minutes`, `Linear10PercentEvery1Minute`, `Linear10PercentEvery10Minutes`, `AllAtOnce`...
 - **Hooks** `PreTraffic` e `PostTraffic` (funções de validação).
 - **Alarmes** do CloudWatch que disparam **rollback automático**.

SAM CLI

Comando	Para quê
<code>sam init</code>	Criar um projeto a partir de um modelo
<code>sam build</code>	Resolver dependências e preparar os artefatos (inclusive com contêiner: <code>--use-container</code>)
<code>sam local invoke</code>	Executar uma função localmente (Docker) com um evento (<code>-e events/evento.json</code>)
<code>sam local start-api</code>	Subir a API localmente para testar pelo navegador ou curl
<code>sam local start-lambda</code>	Endpoint local para invocar funções pelo SDK/CLI em testes automatizados
<code>sam local generate-event</code>	Gerar eventos de exemplo (S3, SQS, API Gateway, DynamoDB...)

Comando	Para quê
<code>sam validate</code>	Validar o template (inclusive com <code>--lint</code>)
<code>sam deploy --guided</code>	Primeira implantação interativa; grava as respostas no <code>samconfig.toml</code>
<code>sam deploy --config-env prod</code>	Implantar usando a seção <code>prod</code> do <code>samconfig.toml</code> (outro ambiente/stack)
<code>sam sync --watch</code>	Sincronizar mudanças de código rapidamente em ambientes de desenvolvimento (não para produção)
<code>sam logs / sam traces</code>	Ver logs e traces da aplicação implantada
<code>sam remote invoke / sam remote test-event</code>	Invocar funções na nuvem e usar eventos de teste compartilháveis
<code>sam pipeline init --bootstrap</code>	Gerar um pipeline de CI/CD com estágios por ambiente
<code>sam package</code>	Enviar artefatos ao S3 e gerar o template final (o <code>sam deploy</code> já faz isso)

- **Mesmo template em vários ambientes:** parâmetros (`Parameters`) como `Stage`, `samconfig.toml` com seções por ambiente e **nomes de stack diferentes** (`app-dev`, `app-staging`, `app-prod`), cada um com seu conjunto de recursos. É a resposta para "implantar o template SAM em um ambiente de staging diferente".

AWS CloudFormation

Anatomia do template

Seção	Função
<code>AWSTemplateFormatVersion</code> , <code>Description</code>	Metadados
<code>Transform</code>	Macros como <code>AWS::Serverless-2016-10-31</code> (SAM) e <code>AWS::Include</code>
<code>Parameters</code>	Entradas na criação/atualização (tipos como <code>String</code> , <code>Number</code> , <code>AWS::EC2::KeyPair::KeyName</code> , <code>AWS::SSM::Parameter::Value<String></code> ; <code>NoEcho</code> para valores sensíveis)
<code>Mappings</code>	Tabelas fixas de valores (ex.: AMI por Região), lidas com <code>Fn::FindInMap</code>
<code>Conditions</code>	Criar recursos ou propriedades conforme condições (ex.: só em <code>prod</code>)
<code>Resources</code>	Única seção obrigatória: os recursos
<code>Outputs</code>	Valores de saída; com <code>Export</code> , podem ser importados por outras stacks
<code>Rules</code> , <code>Metadata</code>	Validações de parâmetros e metadados (ex.: <code>AWS::CloudFormation::Init</code>)

- **Funções intrínsecas:** `Ref` (parâmetro ou ID físico do recurso), `Fn::GetAtt` (atributo, ex.: ARN), `Fn::Sub` (substituir variáveis em string), `Fn::Join`, `Fn::Select`, `Fn::Split`, `Fn::FindInMap`, `Fn::ImportValue` (valor exportado por outra stack), `Fn::If`, `Fn::Equals`, `Fn::GetAZs`, `Fn::Base64` (user data).
- **Pseudoparâmetros:** `AWS::Region`, `AWS::AccountId`, `AWS::StackName`, `AWS::Partition`, `AWS::NoValue`.
- **Referências dinâmicas:** `{{resolve:ssm:/app/prod/url}}`, `{{resolve:ssm-secure:...}}` e `{{resolve:secretsmanager:MeuSegredo:SecretString:password}}`, para não colocar segredos no template.

Stacks, mudanças e proteção

- **Change sets:** mostram **o que vai mudar** (adicionar, modificar, **substituir**) antes de executar a atualização. Use-os para atualizar templates existentes com segurança.

- **Atualizações** podem ser sem interrupção, com interrupção ou com **substituição** (recurso novo com ID novo, por exemplo ao mudar o nome de uma tabela). Verifique o comportamento de cada propriedade na documentação.
- **Rollback**: falha na criação → a stack é apagada (padrão) ou mantida para depuração (`--disable-rollback` / preservar recursos com sucesso). Falha na atualização → volta ao estado anterior. `UPDATE_ROLLBACK_FAILED` exige **continue update rollback** (pulando recursos problemáticos).
- `DeletionPolicy: Retain` (manter o recurso ao apagar a stack), `Snapshot` (RDS, EBS, ElastiCache...), `Delete`. `UpdateReplacePolicy` faz o mesmo quando o recurso é substituído.
- **Stack policy**: impede atualizações acidentais em recursos críticos.
- **Termination protection**: impede apagar a stack.
- **Drift detection**: detecta mudanças manuais feitas fora do CloudFormation.
- **Cross-stack references** (`Export / ImportValue`) vs. **nested stacks** (`AWS::CloudFormation::Stack`, componentes reutilizáveis implantados juntos). Um export em uso **não pode ser alterado nem apagado**.
- **StackSets**: implantar a mesma stack em **várias contas e Regiões**.
- **Capacidades**: templates que criam recursos do IAM exigem `CAPABILITY_IAM` ou `CAPABILITY_NAMED_IAM` (nomes personalizados); macros e transforms (como SAM) exigem `CAPABILITY_AUTO_EXPAND`.
- **Empacotar código local**: `aws cloudformation package` envia artefatos ao S3 e reescreve o template; `aws cloudformation deploy` cria o change set e executa.

Recursos avançados de CloudFormation

- **Custom resources**: uma **função Lambda** (ou tópico SNS) chamada na criação, atualização e exclusão da stack, para lógica que o CloudFormation não suporta (ex.: esvaziar um bucket antes de apagar, buscar um valor externo). A função precisa **responder à URL pré-assinada** com `SUCCESS` ou `FAILED`; se não responder, a stack fica esperando até o timeout.
- **EC2 bootstrapping**:
 - `cfn-init` aplica a configuração declarada em `AWS::CloudFormation::Init` (pacotes, arquivos, serviços).
 - `cfn-signal` avisa sucesso ou falha, junto com `CreationPolicy` ou `WaitCondition` (a stack só conclui quando a instância sinaliza).
 - `cfn-hup` detecta mudanças nos metadados e reaplica a configuração.
- **Hooks do CloudFormation**: validações proativas (ex.: bloquear bucket sem criptografia) antes de criar recursos.
- **IaC generator**: gera templates a partir de recursos já existentes na conta.

AWS CDK

Constructs e comandos

- Escreva a infraestrutura em **TypeScript, Python, Java, C#/.NET ou Go**; o CDK **sintetiza** templates do CloudFormation.
- **Constructs**:
 - **L1** (`Cfn*`): mapeamento 1:1 com recursos do CloudFormation.
 - **L2**: recursos com padrões seguros e métodos úteis, como `bucket.grantRead(funcao)`, que gera a política de menor privilégio.
 - **L3 (patterns)**: arquiteturas completas, como `LambdaRestApi` e `ApplicationLoadBalancedFargateService`.
- **App** → **Stacks** → **Constructs**. Contexto e props permitem instanciar a mesma stack por ambiente.

Comando	Para quê
<code>cdk init</code>	Criar o projeto
<code>cdk bootstrap</code>	Criar os recursos de apoio (bucket, repositório ECR, roles) uma vez por conta e Região
<code>cdk synth</code>	Gerar o template do CloudFormation
<code>cdk diff</code>	Comparar com o que está implantado
<code>cdk deploy</code>	Implantar
<code>cdk destroy</code>	Remover a stack
<code>cdk watch</code> / <code>cdk deploy --hotswap</code>	Atualizações rápidas em desenvolvimento

- **Testes de infraestrutura:** módulo `assertions` para verificar o template gerado (ex.: "existe uma fila com criptografia").
- **CDK Pipelines:** pipeline autoatualizável que implanta a aplicação CDK em vários estágios e contas.

Configuração por ambiente

AWS AppConfig

- Parte do Systems Manager, para **gerenciar e implantar configuração e feature flags** com segurança, **sem reimplantar o código**.
- **Conceitos:**
 - **Aplicação** → **ambientes** (`dev`, `prod`).
 - **Perfis de configuração** (freeform, como JSON e YAML, ou **feature flags**).
 - Fontes: armazenamento hospedado do AppConfig, S3, Parameter Store, Secrets Manager, SSM Documents, CodePipeline.
- **Validators:** **JSON Schema** ou **função Lambda** que valida a configuração antes da implantação.
- **Estratégias de implantação:** liberar a nova configuração **gradualmente** (linear ou exponencial, ex.: `AppConfig.Linear50PercentEvery30Seconds`, `AppConfig.Canary10Percent20Minutes`, `AppConfig.AllAtOnce`), com **bake time** e **rollback automático** quando um **alarme do CloudWatch** dispara.
- **Consumo:** a aplicação busca a configuração (`StartConfigurationSession` e `GetLatestConfiguration`) com cache; em Lambda, a **AppConfig Lambda extension** faz polling e cache automaticamente.
- **Casos de uso:** feature flags, ajustar limites e parâmetros em tempo real, listas de permissão, desligar uma funcionalidade problemática instantaneamente.

Gerenciar ambientes em cada serviço

Serviço	Mecanismo de ambiente e versão aprovada
Lambda	Versões imutáveis e aliases (<code>dev</code> , <code>test</code> , <code>prod</code>) apontando para versões aprovadas
API Gateway	Stages (<code>dev</code> , <code>test</code> , <code>prod</code>) com stage variables ; ou APIs e contas separadas
Contêineres (ECR/ECS/EKS)	Tags de imagem imutáveis e aprovadas (ex.: <code>v1.4.2</code> , hash do commit); cada ambiente referencia uma tag
AWS Amplify	Branches conectadas a ambientes (<code>main</code> → produção, <code>dev</code> → desenvolvimento) e previews por pull request
AWS Copilot	CLI que cria environments separados para aplicações em contêiner (ECS/Fargate). Confirma a situação atual do projeto antes de adotá-lo

Serviço	Mecanismo de ambiente e versão aprovada
Elastic Beanstalk	Ambientes separados por aplicação e versões de aplicação implantadas em cada um
CloudFormation / SAM / CDK	Stacks separadas por ambiente, com parâmetros ou contexto
AppConfig	Ambientes e perfis de configuração

Decisão rápida — Empacotamento, IaC e ambientes

Cenário / requisito	Resposta correta	Distratores clássicos
Declarar funções, APIs e tabelas serverless com pouco código	AWS SAM	CloudFormation puro com todos os recursos
Testar a função localmente com um evento do S3	<code>sam local generate-event + sam local invoke</code>	Implantar para testar
Implantar o mesmo template SAM em staging	<code>sam deploy --config-env staging</code> (outra stack)	Editar o template para cada ambiente
Implantação canary automática de Lambda com rollback por alarme	SAM AutoPublishAlias + DeploymentPreference + alarms	Trocar o código manualmente
Ver o que uma atualização vai substituir antes de aplicar	Change set	Drift detection
Descobrir mudanças manuais fora do CloudFormation	Drift detection	Change set
Manter a tabela ao apagar a stack	<code>DeletionPolicy: Retain</code>	Stack policy
Usar o ARN de uma stack em outra	Output com <code>Export</code> + <code>Fn::ImportValue</code>	Copiar o ARN à mão
Senha do banco no template sem exibi-la	Referência dinâmica <code>{{resolve:secretsmanager:...}}</code>	Parameter com valor padrão
Template que cria roles com nomes personalizados	<code>CAPABILITY_NAMED_IAM</code>	<code>CAPABILITY_IAM</code>
Lógica que o CloudFormation não suporta	Custom resource com Lambda	Script manual depois do deploy
Stack deve esperar a instância terminar de configurar	<code>cfn-signal</code> + <code>CreationPolicy</code>	<code>DependsOn</code>
Implantar em 20 contas e 3 Regiões	CloudFormation StackSets	60 stacks manuais
Infraestrutura em TypeScript com padrões seguros	AWS CDK (constructs L2/L3)	SAM
Preparar conta para o CDK	<code>cdk bootstrap</code>	<code>cdk synth</code>
Ligar e desligar funcionalidade sem reimplantar	AppConfig feature flags	Variável de ambiente + deploy
Liberar nova configuração aos poucos com rollback por alarme	AppConfig deployment strategy + alarmes	Parameter Store
Previews de frontend por branch	Amplify branches	Stages do API Gateway

10. Testes em Desenvolvimento e Automação

Regra de ouro da prova: testes unitários rodam localmente e isolam dependências com **mocks** (SAM CLI, frameworks da linguagem); **testes de integração** rodam contra recursos reais em um ambiente de teste (stage do API Gateway, alias do Lambda, stack separada); **eventos de teste em JSON** reproduzem o que cada serviço envia; o pipeline executa tudo automaticamente antes de promover a versão.

Testes unitários e locais

Testes unitários com AWS SAM

- **Estrutura:** separar a **lógica de negócio** do handler (o handler só traduz o evento e chama funções puras). Isso permite testar a lógica sem AWS.
- **Frameworks:** pytest, Jest, JUnit etc., com **mocks** para chamadas ao SDK:
 - Bibliotecas de stub dos SDKs (`botocore.stub.Stubber`, `aws-sdk-client-mock`).
 - Simuladores como **moto** (Python).
 - **Injeção de dependência** dos clientes.
- **SAM CLI** para testes locais com Docker, próximos do ambiente real:
 - `sam local invoke Funcao -e events/evento.json` executa a função com um evento.
 - `sam local start-api` testa rotas HTTP.
 - `sam local start-lambda` expõe um endpoint local para testes automatizados via SDK.
 - `--env-vars env.json` define variáveis de ambiente locais.
 - `--docker-network` conecta a contêineres locais, como o **DynamoDB Local**.
- **DynamoDB Local:** versão local do DynamoDB para testes sem custo.
- **Step Functions: TestState API** para testar estados individuais (inclusive com respostas simuladas de serviços) e Step Functions Local para executar máquinas localmente.
- **Contêineres de Lambda: Runtime Interface Emulator** para invocar a imagem localmente.

Eventos de teste

- **JSON que reproduz o evento de cada fonte:** API Gateway (proxy: `httpMethod`, `path`, `headers`, `queryStringParameters`, `body` como string), S3 (`Records[]`, `s3.bucket.name`, `object.key`), SQS (`Records[]`, `body`, `messageId`), DynamoDB Streams (`NewImage` / `OldImage` no formato com tipos), Kinesis (`data` em **base64**), EventBridge (`source`, `detail-type`, `detail`).
- **Como obter:**
 - `sam local generate-event s3 put --bucket meu-bucket --key foto.jpg`.
 - Modelos de evento de teste no console do Lambda.
 - Eventos reais capturados nos logs (**sem dados sensíveis**).
- **Eventos de teste compartilháveis:** no console do Lambda (armazenados no schema registry do EventBridge) e com `sam remote test-event`, a equipe reutiliza os mesmos eventos. O console guarda até 10 eventos por função.
- **API Gateway:** o botão **Test** no console executa um método sem implantar, com logs detalhados da integração e da transformação.

Testes de integração e ambientes de teste

Integração com dependências reais e simuladas

- **Testes de integração** chamam os recursos implantados (API, fila, tabela) em uma **stack de teste** ou ambiente efêmero criado pelo pipeline e destruído depois.
- **Mock de APIs externas:**
 - **Integração mock do API Gateway** devolve respostas fixas sem backend.
 - Servidores de stub (WireMock, por exemplo).
 - Uma Lambda que simula o parceiro.
 - Isso permite testar sem depender da disponibilidade ou do custo do terceiro.
- **Endpoints de desenvolvimento: stages** do API Gateway (`dev` , `test`) apontando, via **stage variables**, para aliases de Lambda de teste; ambientes do Beanstalk; stacks por branch.
- **Ambientes com versões aprovadas:** aliases do Lambda, **tags de imagem** de contêiner, **branches** do Amplify e **environments** do Copilot garantem que o teste de integração use exatamente a versão candidata.
- **Contract tests:** verificar que produtor e consumidor concordam com o formato das mensagens e APIs (esquemas do EventBridge schema registry, OpenAPI do API Gateway).

Testar aplicações orientadas a eventos

- **Validar padrões de eventos** antes de implantar: API `TestEventPattern` do EventBridge (ou a sandbox do console).
- **Enviar eventos de teste** com `PutEvents` para um barramento de teste e verificar o efeito (ex.: item gravado, mensagem na fila).
- **Filas e destinos de teste:** assinar uma **fila SQS de teste** no tópico ou regra para capturar o que foi publicado e fazer asserções.
- **Archive e replay** do EventBridge para reproduzir eventos reais em um ambiente de teste.
- **Testar falhas:** forçar erros para verificar retries, **DLQ**, destinations e idempotência (processar o mesmo evento duas vezes).
- **Consistência eventual:** testes assíncronos precisam **aguardar com polling e timeout**, não um `sleep` fixo.
- **Rastreamento:** usar IDs de correlação e X-Ray/OpenTelemetry para seguir o evento pelos serviços durante o teste.

Testes gerados com IA

- O **Amazon Q Developer** gera testes unitários a partir do código (comando `/test` no IDE), sugere casos de borda e mocks. O guia do DVA-C02 cobra isso na habilidade 3.3.6.
- O **Kiro**, sucessor do Q Developer, gera testes a partir das **especificações** (requisitos → design → tarefas) e usa **agent hooks** para rodar testes automaticamente quando arquivos mudam.
- Revise sempre os testes gerados: eles podem testar o comportamento atual (inclusive bugs) em vez do comportamento esperado.

Decisão rápida — Testes

Cenário / requisito	Resposta correta	Distratores clássicos
---------------------	------------------	-----------------------

Cenário / requisito	Resposta correta	Distratores clássicos
Executar a função localmente com um evento do SQS	<code>sam local invoke -e evento.json</code> (evento via <code>sam local generate-event</code>)	Implantar em produção
Testar a API localmente	<code>sam local start-api</code>	Console do API Gateway
Testar código que usa DynamoDB sem custo	DynamoDB Local ou mocks do SDK	Tabela de produção
Frontend precisa testar sem o backend pronto	Integração mock do API Gateway	Lambda temporária
Validar se o padrão da regra casa com um evento	<code>TestEventPattern</code>	Implantar e esperar
Verificar o que foi publicado em um tópico durante o teste	Fila SQS de teste assinada no tópico	Ler logs do SNS
Testar um estado do Step Functions isoladamente	TestState API	Executar a máquina inteira
Reutilizar eventos de teste entre desenvolvedores	Eventos de teste compartilháveis (<code>console/ sam remote test-event</code>)	Enviar JSON por e-mail
Ambiente de integração com a versão candidata da função	Alias do Lambda apontando para a versão	<code>\$LATEST</code>
Gerar testes unitários com IA	Amazon Q Developer <code>/test</code> (ou Kiro)	CodeBuild reports

11. CI/CD e Estratégias de Implantação

Regra de ouro da prova: **CodePipeline** orquestra; **CodeBuild** compila e testa (`buildspec.yml`); **CodeDeploy** implanta em EC2/on-premises, Lambda e ECS (`appspec.yml`), com **canary**, **linear** ou **all-at-once** e **rollback automático**; **Elastic Beanstalk** implanta com políticas próprias (all at once, rolling, rolling com lote extra, **immutable**, traffic splitting) e blue/green por **troca de URL**.

AWS CodeBuild

buildspec.yml

- Arquivo na **raiz do repositório** (ou nome personalizado no projeto) que define o build:

```
version: 0.2
env:
  variables:
    AMBIENTE: "dev"
  parameter-store:
    DB_URL: /app/dev/db-url
  secrets-manager:
    API_KEY: meu-segreto:api_key
phases:
  install:
    runtime-versions:
      python: 3.12
    commands:
      - pip install -r requirements.txt
  pre_build:
    commands:
      - pytest tests/unit
      - aws ecr get-login-password | docker login --username AWS --password-stdin $REPO
  build:
    commands:
      - sam build
      - docker build -t $REPO:$CODEBUILD_RESOLVED_SOURCE_VERSION .
  post_build:
    commands:
      - docker push $REPO:$CODEBUILD_RESOLVED_SOURCE_VERSION
reports:
  testes-unitarios:
    files: ["reports/junit.xml"]
artifacts:
  files:
    - template-empacotado.yaml
cache:
  paths:
    - "/root/.cache/pip/**/*"
```

- Fases:** `install` → `pre_build` → `build` → `post_build`, cada uma com `commands` e `finally`. Se `build` falha, `post_build` ainda roda (útil para relatórios).

- **Variáveis:** texto (`variables`), **Parameter Store** e **Secrets Manager** (nunca coloque segredos em texto no `buildspec`), `exported-variables` para as próximas ações do pipeline. Há variáveis automáticas como `CODEBUILD_RESOLVED_SOURCE_VERSION` (`commit`).
- **Artefatos** enviados ao S3 (ou ao pipeline); **cache** em S3 ou local para acelerar builds.
- **Relatórios de testes e cobertura** (JUnit, Cucumber, JaCoCo...) visíveis no console.
- **Ambiente:** imagens gerenciadas ou personalizadas (Docker), tipos de computação (EC2 ou Lambda), **modo privilegiado** para `docker build`, execução **dentro da VPC** para acessar recursos privados.
- **Logs** no CloudWatch Logs e/ou S3. **Timeouts** configuráveis. Execução local com o **CodeBuild local agent** (Docker) para depurar o `buildspec`.
- **Permissões:** a **service role** do CodeBuild precisa acessar ECR, S3, Parameter Store, Secrets Manager, KMS etc.

AWS CodeDeploy

Plataformas e configurações de implantação

Plataforma	Tipos de implantação	Configurações predefinidas
EC2/on-premises	In-place (atualiza as instâncias existentes) ou blue/green (novas instâncias, troca no load balancer)	<code>CodeDeployDefault.AllAtOnce</code> , <code>HalfAtATime</code> , <code>OneAtATime</code> (ou personalizada por porcentagem ou número mínimo de hosts saudáveis)
AWS Lambda	Deslocamento de tráfego entre versões por alias	<code>Canary10Percent5Minutes</code> (10% por 5 min, depois 100%), <code>Linear10PercentEvery1Minute</code> (10% a cada minuto), <code>AllAtOnce</code> , entre outras
Amazon ECS	Blue/green com dois target groups no ALB/NLB	<code>ECSCanary10Percent5Minutes</code> , <code>ECSLinear10PercentEvery1Minutes</code> , <code>ECSAllAtOnce</code>

- **CodeDeploy agent:** obrigatório em instâncias EC2 e servidores on-premises (Lambda e ECS não precisam). Logs em `/var/log/aws/codedeploy-agent/` e nos logs de cada implantação.
- **Revisão:** bundle no **S3** ou no **GitHub**, com o `appspec.yml`.
- **Deployment group:** conjunto de destinos (tags de EC2, Auto Scaling groups, instâncias on-premises), configuração, alarmes e gatilhos (notificações SNS).
- **Rollback automático:** quando a implantação falha ou quando um **alarme do CloudWatch** dispara. O CodeDeploy **reimplanta a última revisão boa como uma nova implantação** (com novo ID).

appspec.yml e hooks

- **EC2/on-premises:** seções `files` (origem → destino), `permissions` e `hooks` com scripts. Ordem dos hooks em uma implantação **in-place**:
 1. `ApplicationStop`
 2. `DownloadBundle` (*executado pelo agente*)
 3. `BeforeInstall`
 4. `Install` (*executado pelo agente*)
 5. `AfterInstall`
 6. `ApplicationStart`
 7. `ValidateService`
 - Com load balancer, há também `BeforeBlockTraffic`, `BlockTraffic`, `AfterBlockTraffic` no início e `BeforeAllowTraffic`, `AllowTraffic`, `AfterAllowTraffic` no fim.

- **Lambda:** define a função, o alias e as versões atual e nova; hooks `BeforeAllowTraffic` e `AfterAllowTraffic` (funções Lambda de validação que chamam `PutLifecycleEventHookExecutionStatus`).
- **ECS:** define a task definition, o contêiner e a porta; hooks `BeforeInstall`, `AfterInstall`, `AfterAllowTestTraffic` (testes no listener de teste), `BeforeAllowTraffic`, `AfterAllowTraffic`.

AWS CodePipeline e artefatos

Estrutura do pipeline

- **Estágios** (stages) com **ações** em sequência ou em paralelo:
 - **Source:** CodeCommit, S3, ECR, GitHub/GitLab/Bitbucket via CodeConnections.
 - **Build:** CodeBuild, Jenkins.
 - **Test:** CodeBuild, Device Farm e ferramentas de terceiros.
 - **Deploy:** CodeDeploy, **CloudFormation** (criar/atualizar stack, criar e executar **change set**), ECS, Elastic Beanstalk, S3, AppConfig, Service Catalog.
 - **Approval:** manual, com notificação SNS e URL para revisão.
 - **Invoke:** Lambda, Step Functions.
- **Artefatos** passam entre ações por um **bucket S3** (artifact store), criptografados com KMS.
- **Pipelines V2:**
 - **Gatilhos** com filtros por **branch, tag Git e caminho de arquivo**.
 - **Variáveis** em nível de pipeline.
 - **Modos de execução:** `SUPERSEDED` (a mais nova substitui), `QUEUED` e `PARALLEL`.
 - **Condições e rollback de estágio**.
- **Commit que dispara o pipeline:** o push no branch configurado (via evento do EventBridge ou webhook do CodeConnections) inicia a execução: build, testes, implantação. Esse é o fluxo cobrado na habilidade 3.4.6.
- **Fluxos orquestrados para vários ambientes:** estágios `dev` → testes de integração → **aprovação manual** → `prod`, cada um implantando a mesma saída do build com parâmetros diferentes; entre contas, com roles cross-account e uma **customer managed key** do KMS no bucket de artefatos.
- **Monitoramento:** mudanças de estado do pipeline, estágio e ação geram **eventos no EventBridge** (ex.: notificar no Slack via SNS/Chatbot quando a implantação termina ou falha).

AWS CodeArtifact

- Repositório gerenciado de **pacotes** (npm, PyPI, Maven, NuGet, Gradle, Swift, Cargo...), organizado em **domínios e repositórios**, com **repositórios upstream** e conexões externas (npmjs, PyPI) que guardam cópias dos pacotes públicos.
- `aws codeartifact login --tool npm|pip|twine` configura o gerenciador de pacotes com um **token de autorização temporário** (12 horas por padrão).
- Útil para builds reproduzíveis, pacotes internos e controle sobre dependências externas.

Versões, labels e branches

- **Branches Git:** estratégias como trunk-based (branch principal sempre implantável) ou GitFlow; branches de feature disparam pipelines de teste e previews.
- **Tags Git e versionamento semântico** (`v1.4.2`) marcam releases; o pipeline pode ser disparado por tag.
- **Lambda:** versões numeradas e **aliases** como labels de ambiente.

- **Imagens de contêiner:** tags imutáveis por versão ou commit.
- **Beanstalk: version labels** das versões de aplicação.
- **AppConfig:** versões de configuração.
- **Rollback** passa a ser "reapontar para a versão anterior": alias antigo, tag anterior, versão anterior do Beanstalk.

AWS Elastic Beanstalk

Políticas de implantação

Política	Como funciona	Tempo de inatividade	Custo extra	Rollback
All at once	Implanta em todas as instâncias ao mesmo tempo	Sim	Não	Reimplantar a versão anterior
Rolling	Implanta em lotes ; cada lote sai de serviço durante a atualização	Não, mas com capacidade reduzida	Não	Reimplantação manual
Rolling with additional batch	Lança um lote extra antes, mantendo a capacidade total	Não	Pequeno (lote extra temporário)	Reimplantação manual
Immutable	Cria instâncias novas em um Auto Scaling group temporário; só depois de saudáveis substitui as antigas	Não	Maior (dobra temporariamente)	Rápido e seguro: apaga as novas
Traffic splitting	Instâncias novas recebem uma porcentagem do tráfego por um período de avaliação (canary)	Não	Temporário	Automático se as métricas de saúde falham
Blue/green (não é política nativa)	Clonar o ambiente, implantar no novo e trocar as URLs (Swap environment URLs, troca de CNAME)	Não	Dois ambientes	Trocar as URLs de volta

- **Configuração:**
 - `.ebextensions/*.config` (YAML/JSON): `option_settings`, `packages`, `files`, `commands` (antes de extrair a aplicação), `container_commands` (depois de extrair, antes de publicar; `leader_only: true` para rodar em uma instância só, como migrações de banco) e `Resources` (recursos CloudFormation adicionais).
 - `.platform/hooks/` (`prebuild`, `predeploy`, `postdeploy`) nas plataformas Amazon Linux 2/2023.
 - `Procfile` define os processos da aplicação.
- **Precedência das configurações:** aplicadas diretamente no ambiente (console/CLI) > configurações salvas > `.ebextensions` > valores padrão.
- **Tipos de ambiente:** **web server** (com ELB) e **worker**, que consome uma fila **SQS** por um daemon que faz POST localmente para a aplicação, com `cron.yaml` para tarefas periódicas.
- **Banco de dados:** um RDS criado **dentro** do ambiente é apagado junto com ele. Em produção, crie o RDS **separado** e passe a conexão por variáveis de ambiente.
- **Versões de aplicação:** há limite de versões por aplicação; configure a **lifecycle policy** para apagar as antigas.
- **EB CLI:** `eb init`, `eb create`, `eb deploy`, `eb logs`, `eb swap`, `eb health`.
- **Monitoramento:** **enhanced health reporting** com métricas de saúde por instância e por requisição.

Contêineres

ECS, ECR e EKS para desenvolvedores

- **ECR:**
 - Login com `aws ecr get-login-password | docker login ...`.
 - **Tag immutability** impede sobrescrever tags.
 - **Lifecycle policies** apagam imagens antigas.
 - **Scan de imagens** (básico ou avançado com Inspector).
 - Replicação entre Regiões e contas; **pull through cache** de registros públicos.
- **Task definition:**
 - Imagem, CPU/memória, portas.
 - Variáveis e `secrets` (Secrets Manager/Parameter Store).
 - `logConfiguration` com o driver `awslogs` (CloudWatch).
 - **health check** do contêiner.
 - **Task role** e **task execution role**.
 - Cada alteração cria uma **nova revisão**.
- **Service:** mantém N tasks, registra no target group do load balancer e faz **rolling update** conforme `minimumHealthyPercent` e `maximumPercent`. O **deployment circuit breaker** detecta falha e **faz rollback** automático.
- **Estratégias do ECS:** rolling update, **blue/green, linear e canary nativos** (desde 2025, com hooks de ciclo de vida em Lambda e bake time) ou blue/green via **CodeDeploy**.
- **Fargate:** sem gerenciar instâncias; defina CPU e memória por task.
- **EKS:** deployments do Kubernetes com rolling update e **readiness/liveness probes**; imagens no ECR; permissões com Pod Identity.
- **Pipeline típico de contêiner:** CodeBuild gera a imagem com tag do commit → push no ECR → gera `imagedefinitions.json` (ou `appspec` e `taskdef` para blue/green) → ação ECS do CodePipeline atualiza o service.

Estratégias de implantação e rollback

Comparação das estratégias

Estratégia	Como funciona	Vantagens	Riscos e custos
All-at-once	Troca tudo de uma vez	Rápida e simples	Tempo de inatividade; problema atinge 100% dos usuários
Rolling	Atualiza em lotes	Sem infraestrutura extra	Capacidade reduzida; duas versões convivem; rollback lento
Immutable	Sobe instâncias novas e só depois descarta as antigas	Rollback rápido, sem misturar versões na mesma instância	Custo temporário dobrado
Blue/green	Dois ambientes completos; troca o tráfego de uma vez (DNS, load balancer, alias)	Rollback instantâneo (voltar para o azul)	Dois ambientes; cuidado com banco e sessões
Canary	Pequena porcentagem do tráfego vai para a nova versão por um período, depois 100%	Detecta problemas com poucos usuários	Exige métricas e alarmes confiáveis

Estratégia	Como funciona	Vantagens	Riscos e custos
Linear	Aumenta o tráfego em passos iguais em intervalos regulares	Exposição gradual e previsível	Implantação mais lenta

- **Onde cada uma aparece:** Lambda (aliases com pesos + CodeDeploy/SAM: canary, linear, all-at-once); API Gateway (canary no stage); ECS (rolling, blue/green, canary, linear); EC2 (CodeDeploy in-place ou blue/green); Beanstalk (políticas acima); Route 53 (weighted routing para blue/green entre ambientes); AppConfig (implantação gradual de configuração).
- **Rollback:**
 - **Automático** por alarmes (CodeDeploy, SAM `DeploymentPreference`, circuit breaker do ECS, AppConfig, traffic splitting do Beanstalk).
 - **Manual:** reimplantar a versão anterior, reapontar o alias, trocar as URLs do Beanstalk ou voltar o peso do Route 53.

Decisão rápida – CI/CD e implantação

Cenário / requisito	Resposta correta	Distratores clássicos
Compilar, testar e gerar artefatos a cada commit	CodeBuild com <code>buildspec.yml</code>	CodeDeploy
Segredo necessário durante o build	<code>secrets-manager</code> / <code>parameter-store</code> no buildspec	Variável em texto no buildspec
Build precisa acessar um banco privado	CodeBuild configurado na VPC	Tornar o banco público
Acelerar builds que baixam as mesmas dependências	Cache do CodeBuild (S3 ou local)	Build maior
Push no branch <code>main</code> dispara build e deploy	CodePipeline com source no repositório	Cron no CodeBuild
Aprovação antes de produção	Ação de aprovação manual no CodePipeline	Lambda de aprovação sem intervenção humana
Implantar CloudFormation revisando as mudanças	Ação de criar e executar change set no pipeline	Atualização direta
Lambda: 10% do tráfego por 5 min e depois 100%	CodeDeploy <code>Canary10Percent5Minutes</code>	<code>Linear10PercentEvery1Minute</code>
Lambda: 10% a mais a cada minuto	<code>Linear10PercentEvery1Minute</code>	Canary
Rollback automático se a taxa de erro subir	Alarme do CloudWatch no deployment group / <code>DeploymentPreference</code>	Monitoramento manual
Validar a nova versão antes de receber tráfego (Lambda)	Hook <code>BeforeAllowTraffic</code>	<code>ValidateService</code>
Parar o serviço antes de copiar os arquivos (EC2)	Hook <code>ApplicationStop</code>	<code>AfterInstall</code>
Testar a nova task no listener de teste (ECS blue/green)	Hook <code>AfterAllowTestTraffic</code>	<code>BeforeInstall</code>
Beanstalk sem reduzir capacidade e com menor custo	Rolling with additional batch	Immutable
Beanstalk com rollback mais rápido e seguro	Immutable	Rolling
Beanstalk com parte do tráfego na nova versão	Traffic splitting	All at once
Beanstalk blue/green	Clonar ambiente + Swap environment URLs	Política rolling

Cenário / requisito	Resposta correta	Distratores clássicos
Rodar migração de banco em uma instância só	<code>container_commands</code> com <code>leader_only: true</code>	<code>commands</code>
Banco não pode ser apagado com o ambiente Beanstalk	RDS criado fora do ambiente	RDS dentro do ambiente
Pacotes npm internos e cache do npmjs	CodeArtifact com upstream	Bucket S3
Evitar sobrescrever a tag de uma imagem publicada	ECR tag immutability	Lifecycle policy
ECS volta sozinho se as novas tasks não ficam saudáveis	Deployment circuit breaker com rollback	Recriar o service

12. Observabilidade e Análise de Causa Raiz

Regra de ouro da prova: **logs** contam o que aconteceu (CloudWatch Logs, consultados com **Logs Insights**); **métricas** mostram tendências e disparam **alarmes** (CloudWatch, métricas personalizadas com **EMF**); **traces** mostram o caminho de uma requisição entre serviços (**X-Ray**, hoje instrumentado com **OpenTelemetry/ADOT**). **Annotations** do X-Ray são **indexadas** e filtráveis; **metadata** não. Para depurar, siga: métrica anormal → trace da requisição → log daquela requisição.

Logging, monitoramento e observabilidade

Diferenças

- **Logging:** registrar eventos e o estado da aplicação (erros, decisões, entradas relevantes). Responde "o que aconteceu nesta requisição?".
- **Monitoramento:** acompanhar **métricas e condições conhecidas** e alertar (CPU, taxa de erros, latência, fila crescendo). Responde "o sistema está saudável?".
- **Observabilidade:** capacidade de **entender o estado interno** do sistema a partir das saídas (logs, métricas e traces correlacionados), inclusive para problemas **não previstos**. Responde "por que está acontecendo?".
- **Os três pilares** devem compartilhar um **ID de correlação** (request ID, trace ID) para ligar métricas, traces e logs da mesma requisição.

Estratégia de logging e logs estruturados

- **Log estruturado:** cada linha em **JSON** com campos consistentes (`timestamp`, `level`, `service`, `requestId`, `traceId`, `userId` anonimizado, `action`, `durationMs`, `error`). Facilita filtros e agregações no Logs Insights e a extração de métricas.
- **Níveis** (`DEBUG`, `INFO`, `WARN`, `ERROR`) configuráveis por ambiente; `DEBUG` só quando necessário.
- **Eventos da aplicação e ações do usuário:** registre eventos de negócio ("pedido criado", "pagamento recusado") e ações relevantes com o contexto necessário para auditoria, **sem dados sensíveis**.
- **Lambda:**
 - Tudo que vai para stdout/stderr vai para o CloudWatch Logs (log group `/aws/lambda/<função>`).
 - **Controles avançados de log** permitem formato **JSON nativo**, nível mínimo de log da aplicação e do sistema e log group personalizado.
 - O **Powertools for AWS Lambda** (Logger) adiciona contexto (cold start, request ID, correlation ID) automaticamente.
- **Retenção:** defina a retenção de cada log group (o padrão é **nunca expirar**, o que gera custo).
- **Custos:** logs verbosos em alto volume custam caro; amostre logs de debug e use a classe de log **Infrequent Access** para logs pouco consultados.

Amazon CloudWatch

CloudWatch Logs e Logs Insights

- **Estrutura:** log groups → log streams → eventos.

- **Logs Insights:** linguagem de consulta para buscar e agregar logs. Exemplos:

```
fields @timestamp, @message
| filter level = "ERROR" and service = "pedidos"
| sort @timestamp desc
| limit 50
```

```
filter @type = "REPORT"
| stats avg(@duration), max(@duration), max(@maxMemoryUsed / 1024 / 1024) as maxMemMB by bin(5m)
```

```
fields @timestamp, requestId, @message
| filter requestId = "c0ffee-1234"
```

- **Comandos principais:** `fields`, `filter` (com `like` / `regex`), `stats` (`count`, `avg`, `sum`, `max`, `pct` por `bin(...)`), `sort`, `limit`, `parse` (extrair campos de texto), `dedup`.
- **Consultas em linguagem natural:** o Logs Insights pode gerar a consulta a partir de uma pergunta em linguagem natural.
- **Live Tail:** acompanhar logs em tempo real enquanto testa.
- **Metric filters:** transformam padrões nos logs em **métricas** (ex.: contar ocorrências de `"ERROR"`) para criar alarmes.
- **Subscription filters:** enviam logs em tempo real para **Lambda**, **Kinesis Data Streams**, **Amazon Data Firehose** ou **OpenSearch** (ex.: centralizar logs, alertas personalizados).
- **Data protection policies:** mascaram dados sensíveis nos logs (seção 8).
- **Exportação para o S3** para retenção longa e análise com Athena.

Métricas, métricas personalizadas e EMF

- **Métricas:** `namespace` (`AWS/Lambda`, `MinhaApp/Pedidos`) + **nome** + **dimensões** (pares chave-valor, ex.: `FunctionName`, `Ambiente`).
- **Resolução:** **padrão** (1 minuto) ou **alta resolução** (até **1 segundo**, `StorageResolution=1`); alarmes em métricas de alta resolução podem avaliar a cada 10 ou 30 segundos.
- **Métricas personalizadas:**
 - `PutMetricData` (API síncrona; cada chamada tem latência e custo; agrupe vários valores por chamada).
 - **Embedded Metric Format (EMF):** um **log JSON** com um bloco `_aws` que descreve as métricas. O CloudWatch **extrai as métricas do log de forma assíncrona**, sem chamadas de API na aplicação. É o jeito recomendado em Lambda (o Powertools Metrics gera EMF) e permite ter métricas e o log detalhado no mesmo evento.

```
{
  "_aws": {
    "Timestamp": 1790000000000,
    "CloudWatchMetrics": [{
      "Namespace": "MinhaApp",
      "Dimensions": [["Servico"]],
      "Metrics": [{"Name": "PedidosCriados", "Unit": "Count"}]
    }]
  },
  "Servico": "pedidos",
  "PedidosCriados": 1,
  "pedidoId": "123"
}
```

• Métricas importantes por serviço:

Serviço	Métricas a observar
Lambda	Invocations, Errors, Throttles, Duration, ConcurrentExecutions, IteratorAge (streams), DeadLetterErrors, AsyncEventAge, AsyncEventsDropped
API Gateway	Count, 4XXError, 5XXError, Latency, IntegrationLatency, CacheHitCount / CacheMissCount
SQS	ApproximateNumberOfMessagesVisible, ApproximateAgeOfOldestMessage, NumberOfMessagesSent/Received/Deleted
DynamoDB	ConsumedRead/WriteCapacityUnits, ThrottledRequests, ReadThrottleEvents, SystemErrors, SuccessfulRequestLatency
Kinesis	GetRecords.IteratorAgeMilliseconds, WriteProvisionedThroughputExceeded, ReadProvisionedThroughputExceeded
Step Functions	ExecutionsFailed, ExecutionsTimedOut, ExecutionThrottled
ALB	HTTPCode_Target_5XX_Count, TargetResponseTime, UnHealthyHostCount

Alarmes e notificações

- **Estados:** OK, ALARM, INSUFFICIENT_DATA. Configuração: **período, estatística** (Average, Sum, p99...), limite, **N de M períodos** (datapoints to alarm) e tratamento de dados ausentes.
- **Ações:** notificar via **SNS** (e-mail, chat, Lambda), escalar (Auto Scaling), agir no EC2 (parar, reiniciar, recuperar), abrir incidente no Systems Manager.
- **Alarmes compostos:** combinam alarmes com AND / OR para reduzir ruído.
- **Deteção de anomalias:** faixa esperada aprendida por ML em vez de limite fixo.
- **Notificações para ações específicas:**
 - **Limites de cota:** o **Service Quotas** publica métricas de uso (namespace **AWS/Usage**) → alarme ao chegar a 80% da cota; **Trusted Advisor** também avisa sobre limites.
 - **Conclusão ou falha de implantação:** eventos do **CodePipeline**, **CodeBuild** e **CodeDeploy** no **EventBridge** → regra → SNS ou chat; notificações de ferramentas de desenvolvedor configuradas no próprio pipeline.
 - **Mudanças de estado de recursos:** eventos do **AWS Health** e do EC2 no EventBridge.
 - **Chamadas de API sensíveis:** eventos do **CloudTrail** no EventBridge (ex.: alguém alterou uma política).

Dashboards e insights

- **Dashboards** do CloudWatch: widgets de métricas, logs (consultas Logs Insights), alarmes e texto; entre contas e Regiões.

- **Application Signals:** APM com painéis prontos de **latência, erros e volume** por serviço e operação, objetivos de nível de serviço (**SLOs**) e mapa de dependências, a partir de instrumentação OpenTelemetry.
- **Lambda Insights:** métricas do sistema por função (CPU, memória, rede, cold starts) via extensão.
- **Container Insights:** métricas e logs de ECS e EKS por cluster, serviço e task.
- **Contributor Insights:** os **principais contribuintes** (top N) de um padrão nos logs, como IPs que mais erram ou chaves mais acessadas no DynamoDB.
- **CloudWatch Synthetics (canaries):** scripts agendados que **simulam usuários** e testam endpoints e fluxos, alertando antes dos clientes.
- **CloudWatch RUM:** experiência de usuários reais no navegador.
- **Investigações do CloudWatch com IA:** analisam alarmes, métricas, logs, traces e mudanças recentes e sugerem hipóteses de causa raiz (seção 14).

Tracing distribuído

AWS X-Ray e OpenTelemetry

- **Trace:** o caminho completo de uma requisição, identificado pelo **trace ID** propagado no header `X-Amzn-Trace-Id` (ou `traceparent`, no padrão W3C do OpenTelemetry).
- **Segment:** o trabalho feito por um serviço. **Subsegment:** chamadas a dependências (DynamoDB, HTTP, SQL) e blocos de código instrumentados manualmente.
- **Annotations:** pares chave-valor **indexados** (string, número, booleano), usados em **filter expressions** para encontrar traces (ex.: `annotation.clientId = "123"`). Até 50 por trace.
- **Metadata:** qualquer objeto, **não indexado**; serve para guardar detalhes para depuração, não para busca.
- **Sampling rules:** por padrão, **a primeira requisição de cada segundo** (reservatório) **e 5% das demais**. Regras personalizadas por serviço, rota ou método reduzem custo ou aumentam a amostragem de rotas críticas.
- **Service map / trace map:** mapa visual dos serviços com latência, erros (4xx), falhas (5xx) e throttling em cada ligação.
- **Como habilitar:**
 - **Lambda:** ativar **active tracing**; a execution role precisa de `xray:PutTraceSegments` e `xray:PutTelemetryRecords` (política `AWSXRayDaemonWriteAccess`).
 - **API Gateway:** habilitar tracing no stage.
 - **ECS/EKS:** coletor **ADOT** (ou o daemon do X-Ray) como **sidecar** ou `DaemonSet`.
 - **EC2 e on-premises:** agente ou coletor instalado.
 - **Elastic Beanstalk:** opção de configuração do X-Ray.
 - O daemon e os coletores recebem segmentos pela porta **UDP 2000** (variável `AWS_XRAY_DAEMON_ADDRESS`).
- **Instrumentação:**
 - **Recomendada hoje: AWS Distro for OpenTelemetry (ADOT)** ou SDKs do OpenTelemetry, inclusive com **instrumentação automática** (zero code).
 - **Legada:** os **SDKs do X-Ray**, em **modo de manutenção desde 25/fev/2026**, com fim de suporte em 25/fev/2027. Simulados ainda os usam (`xray_recorder.begin_subsegment`, `patch_all()`, `putAnnotation`).
 - **CloudWatch Transaction Search** indexa todos os spans para busca.
- **X-Ray vs. CloudTrail vs. CloudWatch:** X-Ray mostra **o caminho e a latência** de uma requisição da aplicação; CloudTrail mostra **quem chamou qual API da AWS**; CloudWatch mostra **métricas e logs**.

Troubleshooting e causa raiz

Erros comuns e o que significam

Sintoma ou erro	Causa provável	Como resolver
<code>AccessDeniedException / 403 / is not authorized to perform</code>	Falta permissão na role, na política de recurso, na key policy do KMS ou uma SCP/permission boundary nega	Ler a ação e o recurso na mensagem, conferir política de identidade e de recurso, usar Policy Simulator e CloudTrail
<code>ThrottlingException, TooManyRequestsException, 429, Rate exceeded</code>	Cota de requisições estourada (API, Lambda, KMS, API Gateway)	Retries com backoff e jitter, cache, lotes, aumento de cota
<code>ProvisionedThroughputExceededException</code>	DynamoDB ou Kinesis acima da capacidade (ou chave quente)	Backoff, capacidade maior/on-demand, melhor partition key
<code>Task timed out after X seconds (Lambda)</code>	Função mais lenta que o timeout; dependência pendurada; função em VPC sem rota para o serviço	Aumentar timeout/memória, timeouts nas chamadas, VPC endpoint ou NAT
<code>Runtime exited with error: signal: killed / memória no limite</code>	Falta de memória (<code>Max Memory Used = Memory Size</code>)	Aumentar memória, corrigir vazamento
API Gateway 502	Resposta malformada da Lambda proxy ou exceção não tratada	Devolver <code>statusCode</code> e <code>body string</code> ; tratar erros
API Gateway 504	Integração passou do timeout	Assíncrono, streaming ou aumentar a cota de timeout
<code>ConditionalCheckFailedException</code>	Condição do DynamoDB falhou (item já existe, versão mudou)	Esperado em controle de concorrência; tratar no código
<code>KMS.ThrottlingException / AccessDenied</code> ao ler do S3	Cota do KMS ou falta de <code>kms:Decrypt</code>	Bucket Keys, cache de chaves, permissão na key policy
<code>SignatureDoesNotMatch / RequestTimeTooSkewed</code>	Relógio fora de sincronia ou credenciais erradas	Sincronizar o relógio (NTP) e conferir credenciais
<code>ResourceNotFoundException</code>	Nome ou ARN errado, Região errada	Conferir Região e ambiente
Mensagens do SQS processadas várias vezes	Visibility timeout curto ou falta de <code>DeleteMessage</code>	Aumentar visibility timeout, apagar após sucesso, idempotência
<code>IteratorAge</code> crescendo	Consumidor do stream mais lento que a produção	Mais memória, lotes maiores, parallelization factor, mais shards
Lambda não é invocada pela fonte	Falta a resource-based policy, regra/filtro errado, event source mapping desativado	Conferir permissões e o padrão do evento

Depurar código e falhas de implantação

- **Depuração local:** SAM CLI com depurador (`sam local invoke -d 5858` e o IDE conectado), **AWS Toolkit** para VS Code e JetBrains (depurar Lambda localmente e remotamente), testes unitários reproduzindo o evento que falhou.
- **Interpretar métricas, logs e traces:** comece pela métrica (quando começou, qual operação), abra traces com erro no período, identifique o **segmento lento ou com falha**, vá ao log com o mesmo request ID ou trace ID.
- **Linha REPORT do Lambda:** `Duration`, `Billed Duration`, `Memory Size`, `Max Memory Used` e `Init Duration` (só em cold start) revelam lentidão, memória no limite e custo de inicialização.
- **Falhas de implantação, onde olhar:**

Serviço	Onde encontrar a causa
---------	------------------------

Serviço	Onde encontrar a causa
CloudFormation / SAM / CDK	Aba Events da stack (primeiro evento <code>CREATE_FAILED</code> / <code>UPDATE_FAILED</code> e o motivo); status <code>ROLLBACK_COMPLETE</code>
CodeBuild	Logs do build no CloudWatch/S3, fase que falhou, relatórios de testes
CodeDeploy	Eventos de ciclo de vida da implantação no console; logs do agente e dos scripts nas instâncias (<code>/opt/codedeploy-agent/deployment-root/...</code> , <code>/var/log/aws/codedeploy-agent/</code>)
CodePipeline	Ação que falhou e o link para os detalhes do serviço executor
Elastic Beanstalk	Events do ambiente, <code>eb logs</code> (últimas 100 linhas ou pacote completo), enhanced health
ECS	Stopped reason da task (erro ao puxar imagem, health check, <code>OutOfMemory</code>), eventos do service, logs do contêiner
Lambda	Erros de upload (tamanho do pacote), permissões da execution role, logs da primeira invocação

Depurar integrações entre serviços

- **Permissões dos dois lados:** a origem precisa poder enviar (política de recurso da fila, do tópico ou da função, trust policy da role) e o destino precisa aceitar.
- **Formato do evento:** logar o evento recebido (sem dados sensíveis) e compará-lo com o que o código espera; usar eventos de exemplo (`sam local generate-event`).
- **Padrões e filtros:** regra do EventBridge, filter policy do SNS e filter criteria do event source mapping podem estar descartando eventos; teste com `TestEventPattern` .
- **Métricas de entrega:**
 - EventBridge: `FailedInvocations` , `InvocationsSentToDlq` .
 - SNS: `NumberOfNotificationsFailed` .
 - Lambda assíncrono: `AsyncEventsDropped` .
 - Examine também as DLQs e os destinos de falha.
- **Rede:** funções e contêineres em VPC precisam de rota (NAT ou VPC endpoint) e security groups que permitam a conexão.
- **Criptografia:** filas e tópicos com chave do KMS exigem que o serviço produtor tenha permissão na key policy.
- **Tracing** ponta a ponta mostra em qual salto a requisição parou.

Health checks e readiness probes

- **Health check** verifica se o componente está vivo; **readiness** verifica se está **pronto para receber tráfego** (dependências conectadas, cache aquecido).
- **Exemplos:**
 - **ALB/NLB: target group health checks** (caminho como `/health` , intervalo, limites de saudável e não saudável, códigos aceitos).
 - **ECS:** `healthCheck` do contêiner na task definition (comando, intervalo, `startPeriod` para dar tempo à inicialização), além do health check do load balancer e do grace period do service.
 - **Kubernetes (EKS):** `readinessProbe` (fora do balanceamento até ficar pronto), `livenessProbe` (reiniciar se travar) e `startupProbe` (inicialização lenta).
 - **Route 53:** health checks de endpoints para failover de DNS.
 - **Elastic Beanstalk:** enhanced health.
 - **CloudWatch Synthetics:** canaries testando o fluxo do usuário.
- **Boas práticas:** endpoint leve e rápido, que verifica **dependências críticas** com timeout curto e não gera efeitos colaterais; separar "vivo" de "pronto" para não reiniciar instâncias saudáveis quando um banco está lento.

Decisão rápida — Observabilidade e troubleshooting

Cenário / requisito	Resposta correta	Distratores clássicos
Métrica de negócio a partir da Lambda sem chamadas de API extras	Embedded Metric Format (EMF)	<code>PutMetricData</code> a cada invocação
Contar ocorrências de "ERROR" nos logs e alertar	Metric filter + alarme	Lambda lendo logs
Consultar e agregar logs de várias funções	CloudWatch Logs Insights	Baixar logs e usar grep
Enviar logs em tempo real para processamento	Subscription filter (Lambda/Kinesis/Firehose/OpenSearch)	Exportação para o S3
Encontrar traces de um cliente específico	Annotation do X-Ray + filter expression	Metadata
Guardar o payload completo no trace sem indexar	Metadata do X-Ray	Annotation
Rastrear requisições com o padrão atual recomendado	ADOT / OpenTelemetry enviando ao X-Ray	Novo código com X-Ray SDK
Lambda com tracing negado ao enviar segmentos	Permissões <code>xray:PutTraceSegments</code> na execution role	Abrir porta UDP 2000
Tracing em tasks do ECS	Coletor ADOT (ou daemon X-Ray) como sidecar	Ativar no ALB
Aumentar a amostragem de uma rota crítica	Regra de sampling personalizada	Instrumentar tudo a 100%
Alerta ao chegar a 80% de uma cota	Métrica de uso do Service Quotas + alarme	Verificar manualmente
Aviso quando a implantação termina ou falha	Evento do CodePipeline/CodeDeploy no EventBridge → SNS	Polling na API
Teste periódico do fluxo de login como um usuário	CloudWatch Synthetics canary	Health check do ALB
Quais IPs mais geram erros	Contributor Insights	Dashboard simples
Descobrir por que a stack falhou	Events da stack (primeiro <code>CREATE_FAILED</code>)	Logs da Lambda
Task do ECS para logo depois de iniciar	Stopped reason da task + logs do contêiner	Métricas do cluster
Tirar a task do balanceamento enquanto aquece	Readiness probe / health check com grace period	Liveness probe
Lambda sempre com <code>Max Memory Used</code> igual ao limite	Aumentar a memória	Aumentar o timeout

13. Otimização de Aplicações

Regra de ouro da prova: meça antes de otimizar. Em Lambda, **memória também é CPU**, e mais memória pode ficar **mais barata** se a função terminar mais rápido. **Cache** em cada camada (CloudFront, API Gateway, ElastiCache, DAX, memória da função) e **filtros na origem** (filter policies do SNS, regras do EventBridge, filter criteria do event source mapping) reduzem trabalho e custo. **Concorrência = taxa × duração**.

Concorrência e capacidade

Definir e dimensionar concorrência

- **Concorrência:** quantas requisições estão em processamento **ao mesmo tempo**. Fórmula: **concorrência = requisições por segundo × duração média (s)**.
 - 500 req/s com 200 ms → **100** execuções simultâneas.
 - Se a duração dobra (dependência lenta), a concorrência dobra com o mesmo tráfego e pode atingir o limite da conta.
- **Limites em cadeia:** API Gateway (10.000 req/s) pode receber mais do que o Lambda (1.000 execuções simultâneas por padrão) consegue atender; um banco pode aguentar menos conexões do que o Lambda abre. Use **reserved concurrency**, **RDS Proxy**, filas e `MaximumConcurrency` para proteger as dependências.
- **Concorrência em contêineres e EC2:** número de workers, threads, conexões por instância e tamanho do pool de conexões; Auto Scaling por métricas como requisições por alvo ou mensagens na fila por task.

Memória e computação mínimas

- **Lambda:**
 - Teste diferentes memórias com o **AWS Lambda Power Tuning** (ferramenta open source que executa a função com várias configurações via Step Functions e mostra custo × tempo).
 - Use as recomendações do **AWS Compute Optimizer** para funções Lambda.
 - Observe `Max Memory Used` na linha `REPORT`.
- **Contêineres:** defina CPU e memória a partir de testes de carga e das métricas do **Container Insights**; deixe margem para picos, mas evite reservar demais (custo no Fargate).
- **Arquitetura arm64 (Graviton):** geralmente melhor preço-desempenho para Lambda, contêineres e EC2, desde que as dependências sejam compatíveis.

Perfilamento e análise de desempenho

Encontrar gargalos

- **Profiling:** medir onde o tempo e a memória são gastos no código (funções mais lentas, alocações excessivas):
 - **Subsegmentos** do X-Ray/OpenTelemetry em blocos críticos.
 - **Application Signals** para latência por operação.
 - **Lambda Insights** para CPU e memória.
 - Profilers da linguagem em testes locais.

- (O Amazon CodeGuru Profiler saiu da lista de serviços do exame.)
- **Pelos traces:** o service map e a linha do tempo do trace mostram **qual dependência consome o tempo** (ex.: 800 ms em uma chamada HTTP externa, várias queries sequenciais ao DynamoDB).
- **Pelos logs:**
 - Registrar a **duração** das operações em logs estruturados (`durationMs`).
 - Consultar com Logs Insights: `stats avg(durationMs), pct(durationMs, 99) by operacao`.
 - Na Lambda, analisar `@duration`, `@initDuration` e `@maxMemoryUsed` das linhas `REPORT` para achar picos, cold starts e memória insuficiente.
- **Padrões de problema comuns:**
 - Chamadas sequenciais que poderiam ser **paralelas** ou em **lote**.
 - **Scan** no lugar de Query.
 - **N+1 consultas**.
 - Conexões recriadas a cada invocação.
 - Payloads grandes demais.
 - Logs síncronos excessivos.
 - Falta de cache.

Cache em cada camada

Cache com base em headers

- **CloudFront:**
 - A **cache policy** define a **chave de cache**: quais **headers**, **query strings** e **cookies** fazem parte dela. Incluir só o necessário (ex.: `Accept-Language` para conteúdo por idioma, `CloudFront-Viewer-Country` para conteúdo por país) aumenta a **taxa de acerto**. Incluir headers variáveis (como `User-Agent` completo ou `Authorization`) fragmenta o cache.
 - A **origin request policy** envia headers à origem **sem** incluí-los na chave.
 - TTLs mínimo, padrão e máximo; a origem controla com `Cache-Control` e `Expires`.
 - **Invalidações** removem objetos antes do TTL; prefira **nomes versionados** (`app.v2.js`).
- **API Gateway (REST):** cache por stage com **parâmetros de chave de cache** (headers, query strings, path) por método; clientes podem pedir `Cache-Control: max-age=0` se autorizados.
- **Headers HTTP de cache** na resposta da aplicação: `Cache-Control` (`max-age`, `no-store`, `private`), `ETag` / `If-None-Match` (resposta `304 Not Modified`), `Vary` (indicar quais headers mudam a resposta).

Cache na aplicação

- **Na memória do ambiente Lambda** (variáveis globais): configuração, segredos (com TTL curto), resultados de consultas de referência. Rápido e grátis, mas **não compartilhado** entre ambientes e perdido quando o ambiente é reciclado.
- **ElastiCache** (Valkey/Redis/Memcached): cache compartilhado entre instâncias e funções, com **lazy loading**, **write-through** e **TTL** (seção 5).
- **DAX** para leituras do DynamoDB em microssegundos.
- **Extensões com cache:** Parameters and Secrets Lambda Extension, AppConfig extension.
- **Cuidados:** invalidar ou expirar quando o dado muda, evitar cachear dados por usuário em cache compartilhado sem incluir o usuário na chave, e tratar a indisponibilidade do cache sem derrubar a aplicação.

Otimizar mensageria e uso de recursos

Filtros de assinatura e eficiência na mensageria

- **SNS filter policies:** cada assinante recebe só o que interessa; evita invocar funções que descartariam a mensagem.
- **EventBridge event patterns** específicos (por campo do `detail`).
- **Filter criteria no event source mapping** do Lambda (SQS, Kinesis, DynamoDB Streams): a função só é invocada para registros relevantes. Mensagens do SQS filtradas são **apagadas** da fila.
- **Lotes:** `SendMessageBatch`, `PutRecords`, `BatchWriteItem`, batch size e batching window do event source mapping reduzem chamadas e custo.
- **Long polling** no SQS.
- **Payloads menores:** enviar só o necessário (ou uma referência ao S3) reduz custo e latência.

Otimizar o uso de recursos

- **Lambda:**
 - Inicializar clientes do SDK e conexões **fora do handler**.
 - Reutilizar conexões HTTP (keep-alive, padrão nos SDKs atuais).
 - Carregar só os módulos necessários (imports seletivos, bundling com tree-shaking).
 - Usar arm64.
 - Ajustar memória com Power Tuning.
 - **SnapStart** ou **provisioned concurrency** para latência.
 - Evitar esperas ociosas (usar Step Functions para aguardar).
- **DynamoDB:** Query com índices adequados, projeções mínimas, `BatchGetItem`, on-demand vs. provisionado com Auto Scaling, TTL para remover dados antigos.
- **S3:** multipart upload e byte-range fetches em paralelo, prefixos para paralelismo, lifecycle.
- **API Gateway:** HTTP API quando os recursos da REST API não são necessários; cache; compressão de respostas.
- **Contêineres:** dimensionar tasks pelo uso real, Auto Scaling por métricas de aplicação, Fargate Spot para cargas tolerantes a interrupção.
- **Logs e métricas:** níveis de log adequados, retenção definida, EMF em vez de muitas chamadas `PutMetricData`.

Decisão rápida — Otimização

Cenário / requisito	Resposta correta	Distratores clássicos
Função limitada por CPU e lenta	Aumentar memória (aumenta a CPU)	Aumentar timeout
Encontrar a memória com melhor custo × tempo	AWS Lambda Power Tuning / Compute Optimizer	Tentativa e erro em produção
1.000 req/s com 300 ms de duração: concorrência?	300 execuções simultâneas	1.000
Reduzir latência de clientes do SDK a cada invocação	Inicializar clientes fora do handler	Criar a cada chamada
Conteúdo diferente por idioma no CloudFront com bom hit ratio	Incluir só <code>Accept-Language</code> na cache policy	Encaminhar todos os headers
Enviar header à origem sem fragmentar o cache	Origin request policy	Cache policy

Cenário / requisito	Resposta correta	Distratores clássicos
Cache de respostas por parâmetro de consulta na API	Cache do API Gateway com a query string na chave	ElastiCache no cliente
Função invocada para mensagens que ela descarta	Filter criteria no event source mapping / filter policy do SNS	if no código
Identificar a dependência que causa lentidão	Traces do X-Ray/OpenTelemetry (subsegmentos)	Métrica <code>Invocations</code>
Descobrir operações mais lentas pelos logs	Logs Insights com <code>stats pct(durationMs, 99) by operacao</code>	Dashboard padrão
Cold start com Java impactando a latência	SnapStart	Mais layers
Leituras repetidas do DynamoDB em microssegundos	DAX	Scan com cache local

14. Desenvolvimento Assistido por IA e o DVA-C03

Regra de ouro da prova: ferramentas de IA **aceleram** escrever, revisar, testar e depurar código, mas **não substituem a revisão humana**. Aplicações que **usam** IA precisam dos mesmos cuidados de qualquer integração (IAM de menor privilégio, segredos, logs sem dados sensíveis) e de controles próprios: **filtrar entradas e saídas do modelo**, defender contra **prompt injection** e **autorizar cada ação de um agente**.

Ferramentas de IA para desenvolvedores

Amazon Q Developer e Kiro

- O guia do DVA-C02 cobra o **Amazon Q Developer** em duas habilidades: **assistência ao desenvolvimento** (1.1.11) e **geração de testes automatizados** (3.3.6).
- **Recursos do Q Developer:**
 - Sugestões de código inline e chat no IDE.
 - **Geração de testes unitários** (`/test`).
 - **Revisão de código** com detecção de problemas de segurança e qualidade (`/review`).
 - **Documentação** (`/doc`).
 - **Desenvolvimento de funcionalidades** a partir de uma descrição (`/dev`).
 - **Transformação de código** (ex.: upgrade de versões do Java).
 - Ajuda no console para **explicar e diagnosticar erros**.
- **Situação atual:** o Q Developer **não aceita novas assinaturas desde 15/mai/2026** (suporte até 30/abr/2027). O sucessor é o **Kiro**.
- **Kiro:** plataforma de desenvolvimento **agêntico** da AWS (IDE, CLI e outras interfaces):
 - **Desenvolvimento orientado a especificações** (spec-driven): o pedido vira **requisitos, design e tarefas** antes do código.
 - **Steering files:** regras do projeto que orientam o agente.
 - **Agent hooks:** ações automáticas, como rodar testes ou atualizar a documentação quando um arquivo muda.
 - Suporte a **MCP** e agentes em paralelo.
- **Na prova:** "gerar testes unitários para uma função existente com IA" → **Q Developer** (ou Kiro); "revisar o código em busca de vulnerabilidades com IA" → recurso de revisão/scan de segurança do assistente.

IA no CI/CD, nos testes e no troubleshooting

- **Testes:** gerar casos de teste e dados de teste, analisar resultados e cobertura, automatizar regressão.
- **CI/CD:** apoiar aprovações de implantação com análise de risco, provisionar ambientes, validar depois da implantação (comparar métricas e logs antes e depois).
- **Troubleshooting:**
 - **Investigações do CloudWatch** (assistidas por IA) correlacionam alarmes, métricas, logs, traces e mudanças recentes e sugerem a causa raiz.
 - O assistente no console explica mensagens de erro.
 - Geração de consultas do Logs Insights em linguagem natural.

- **Otimização:** identificar gargalos de desempenho, uso excessivo de recursos e oportunidades de eficiência no código.
 - **Revisão humana obrigatória:** código gerado pode conter bugs, dependências inseguras ou licenças inadequadas; passe por testes, revisão e scanners como qualquer outro código.
-

Segurança em aplicações que usam IA

Controles ao integrar serviços de IA

- **Controle de acesso:** roles com **menor privilégio** para invocar modelos (`bedrock:InvokeModel` só nos modelos necessários) e acessar bases de conhecimento.
 - **Privacidade dos dados:**
 - Enviar ao modelo **só os dados necessários**.
 - Mascara PII antes (Comprehend, Guardrails).
 - Usar **AWS PrivateLink** para não trafegar pela internet.
 - No **Amazon Bedrock**, os prompts não são usados para treinar os modelos base.
 - **Controlar entradas e saídas do modelo:**
 - **Amazon Bedrock Guardrails:** filtros de conteúdo, tópicos negados, filtros de PII, detecção de **ataques de prompt** e verificação de fundamentação.
 - Validar o formato das respostas antes de usá-las (ex.: JSON esperado) e **nunca executar** diretamente código ou comandos gerados pelo modelo sem validação.
 - **Prompt injection:** tratar texto de usuários e de documentos recuperados como **dados, não instruções**; separar o system prompt; limitar o que o modelo pode fazer com as ferramentas disponíveis.
 - **Segurança de agentes:**
 - **Autorizar cada uso de ferramenta** (ex.: **Policy in AgentCore**, permissões mínimas por ferramenta).
 - **Isolar sessões** entre usuários (o AgentCore Runtime isola cada sessão).
 - **Identidade do agente** com credenciais delegadas (**AgentCore Identity**).
 - **Aprovação humana** (human-in-the-loop) antes de ações sensíveis.
 - **Logs sem conteúdo sensível:**
 - Prompts e respostas podem conter PII.
 - Mascara com **CloudWatch Logs data protection** e controle quem lê o **model invocation logging**.
 - Defina retenção curta.
 - **Segredos:** chaves de APIs de modelos de terceiros no Secrets Manager.
-

DVA-C02 ou DVA-C03

O que muda e como decidir

- **Datas:** DVA-C02 até **30/nov/2026**; DVA-C03 com inscrições a partir de **27/out/2026** e aplicação a partir de **1º/dez/2026**. Quem passar no C02 mantém a certificação até a data de expiração normal.
- **Formato do C03:** 65 questões (50 pontuadas), 130 minutos, nota mínima 720, US\$ 150.
- **Domínios do C03:** Development with AWS Services (30%), Security (26%), **Testing and Deployment** (22%) e Troubleshooting and Optimization (22%).
- **Novidades anunciadas no C03:**

- Desenvolvimento assistido por IA como competência central.
- Uma **tarefa dedicada à segurança de IA**: gestão de acesso, privacidade de dados, proteção contra prompt injection, segurança de interações de agentes (autorização de ferramentas, isolamento de sessão, fluxos de aprovação humana).
- Novos serviços no escopo: **Amazon Bedrock, Bedrock AgentCore, Amazon Q** e outros.
- Consolidação de habilidades em DynamoDB, segurança, testes, observabilidade e otimização.
- **Como usar este guia para o C03:**
 - Os capítulos 1 a 13 cobrem os fundamentos, que continuam valendo.
 - Esta seção e as tabelas de segurança de IA cobrem as novidades anunciadas.
 - Quando o guia oficial do C03 for publicado (27/out/2026), confira as tarefas e a lista de serviços no escopo.
 - Para Bedrock, Guardrails e AgentCore em profundidade, o guia do **AWS Certified AI Practitioner (AIF-C01)** deste repositório é um bom complemento.

Decisão rápida — IA no desenvolvimento

Cenário / requisito	Resposta correta	Distratores clássicos
Gerar testes unitários com IA no IDE (DVA-C02)	Amazon Q Developer /test	CodeBuild reports
Assistente de desenvolvimento para novos clientes em 2026	Kiro	Amazon Q Developer
Transformar um pedido em requisitos, design e tarefas antes do código	Kiro (spec-driven)	CodePipeline
Sugestões de causa raiz a partir de alarmes, logs e traces	Investigações do CloudWatch (IA)	CloudTrail
Impedir que o modelo exponha PII nas respostas	Bedrock Guardrails (filtro de informações sensíveis)	KMS
Detectar tentativas de prompt injection	Guardrails (filtro de ataques de prompt) + separar instruções de dados	Aumentar a temperatura
Agente só pode chamar ferramentas autorizadas	Policy in AgentCore / permissões mínimas por ferramenta	Instrução no prompt
Ação sensível do agente precisa de confirmação	Human-in-the-loop (aprovação humana)	Retry automático
Invocar o Bedrock sem trafegar pela internet	AWS PrivateLink	NAT Gateway
Prompts com PII aparecendo nos logs	Data protection policy do CloudWatch Logs e acesso restrito aos logs de invocação	Desligar todos os logs

15. Padrões recorrentes e palavras-chave

As questões do Developer - Associate giram em torno de poucas decisões: como invocar, como tratar falhas, como dar permissão sem credenciais fixas, como implantar com segurança e como encontrar a causa de um problema. Esta seção reúne as palavras do enunciado que apontam para cada resposta.

Palavras-chave do enunciado

Se o enunciado diz...	Pense em...
"Sem armazenar credenciais", "credenciais temporárias"	IAM roles, STS, Cognito Identity Pools
"Usuários do aplicativo", "login social", "JWT"	Cognito User Pools
"Acesso direto do app ao S3/DynamoDB"	Cognito Identity Pools
"Outra conta"	AssumeRole + trust policy (+ External ID)
"Upload direto do navegador"	Presigned URL
"Arquivo maior que 5 GB"	Multipart upload
"Mais de 4 KB com KMS"	Envelope encryption (GenerateDataKey)
"Rotação automática de senha"	Secrets Manager
"Configuração hierárquica gratuita"	Parameter Store Standard
"Feature flag sem reimplantar"	AppConfig
"Evento assíncrono falhou e preciso do erro"	Lambda Destination on-failure
"Mensagem processada duas vezes"	Visibility timeout + idempotência
"Ordem garantida"	SQS FIFO (MessageGroupId) ou Kinesis (partition key)
"Vários consumidores do mesmo stream com replay"	Kinesis Data Streams
"Mesma mensagem para vários sistemas"	SNS fan-out ou EventBridge
"Reprocessar eventos antigos"	EventBridge archive/replay ou Kinesis
"Fluxo com várias etapas e compensação"	Step Functions
"Aprovação humana no meio do fluxo"	Step Functions .waitForTaskToken
"Cold start"	Provisioned concurrency ou SnapStart
"Conexões demais no banco"	RDS Proxy
"Throttling esporádico"	Retry com backoff exponencial e jitter
"Consulta por outro atributo"	GSI
"Leitura em microssegundos do DynamoDB"	DAX
"Expirar itens automaticamente"	DynamoDB TTL
"Limite por cliente da API"	Usage plans + API keys
"Ambientes dev/test/prod na API"	Stages + stage variables
"10% do tráfego para a nova versão"	Canary (alias com pesos, CodeDeploy, stage canary)
"Rollback automático por alarme"	CodeDeploy / SAM DeploymentPreference / ECS circuit breaker
"Testar localmente"	SAM CLI (sam local invoke , start-api)

Se o enunciado diz...	Pense em...
"Métrica personalizada sem chamada de API"	EMF
"Buscar traces por um campo"	Annotation do X-Ray
"Agregar e consultar logs"	CloudWatch Logs Insights
"Mudanças manuais fora do template"	Drift detection
"Ver o que vai mudar antes de atualizar a stack"	Change set
"Gerar testes com IA"	Amazon Q Developer (ou Kiro)

Pares que mais se confundem

Par	Como separar
Execution role vs. resource-based policy (Lambda)	Execution role = o que a função pode fazer ; resource policy = quem pode invocar a função
Task role vs. task execution role (ECS)	Task role = permissões da aplicação ; execution role = agente do ECS (puxar imagem, logs, segredos)
DLQ vs. Destinations (Lambda)	DLQ guarda só o evento; Destination guarda evento + resposta/erro e aceita sucesso e falha
DLQ da fila vs. on-failure destination	SQS como fonte: DLQ na fila ; Kinesis/DynamoDB Streams: on-failure destination no mapping
Reserved vs. provisioned concurrency	Reserved garante e limita, sem custo; provisioned mantém ambientes aquecidos, com custo
Visibility timeout vs. delay queue	Visibility: esconde a mensagem durante o processamento ; delay: atrasa a primeira entrega
Short vs. long polling	Long polling (até 20 s) reduz respostas vazias e custo
SQS vs. Kinesis	SQS: um consumidor, apaga após processar; Kinesis: ordem por shard, vários consumidores, replay
SNS vs. EventBridge	SNS: fan-out simples, e-mail/SMS/push; EventBridge: regras ricas, SaaS, archive/replay, schema
Step Functions Standard vs. Express	Standard: até 1 ano, exatamente uma vez, auditável; Express: até 5 min, alto volume, mais barato
LSI vs. GSI	LSI: mesma partition key, só na criação, leitura forte; GSI: qualquer chave, a qualquer momento, eventual
Query vs. Scan	Query usa a chave; Scan lê tudo; <code>FilterExpression</code> não reduz a capacidade consumida
BatchWriteItem vs. TransactWriteItems	Lote: até 25, pode falhar parcialmente; transação: até 100, tudo ou nada, 2x capacidade
Lazy loading vs. write-through	Lazy: cache no miss (pode ficar desatualizado); write-through: atualiza na escrita (pode guardar o que ninguém lê)
DAX vs. ElastiCache	DAX: cache compatível com a API do DynamoDB; ElastiCache: cache genérico
SSE-S3 vs. SSE-KMS vs. SSE-C vs. client-side	Chave do S3; chave do KMS (auditoria); chave do cliente por requisição; criptografia antes do envio
AWS managed key vs. customer managed key	Managed: sem alterar a política, sem uso entre contas, rotação anual; customer: controle total, entre contas, rotação configurável
Secrets Manager vs. Parameter Store	Rotação automática e replicação vs. configuração hierárquica gratuita
Cognito User Pool vs. Identity Pool	Autenticação e JWT vs. credenciais AWS temporárias
ID token vs. access token	ID: quem é o usuário (claims); access: o que pode acessar (escopos)
REST API vs. HTTP API	REST: cache, usage plans, validação, WAF, privado; HTTP: mais simples, barato, JWT nativo
API Gateway 502 vs. 504	502: resposta malformada; 504: timeout de integração
Lambda proxy vs. não proxy	Proxy: a função trata tudo e devolve <code>statusCode</code> / <code>body</code> ; não proxy: mapping templates no gateway

Par	Como separar
CloudFront Functions vs. Lambda@Edge	Leve, só viewer, JS, submilissegundo vs. mais poderoso, origin e viewer, acesso à rede
Change set vs. drift detection	O que vai mudar vs. o que já mudou fora do CloudFormation
Export/ImportValue vs. nested stacks	Compartilhar valores entre stacks independentes vs. componentes implantados juntos
commands vs. container_commands (Beanstalk)	Antes de extrair a aplicação vs. depois, com leader_only possível
Rolling vs. rolling with additional batch vs. immutable	Capacidade reduzida vs. capacidade mantida (lote extra) vs. novas instâncias com rollback rápido
Canary vs. linear	Uma porcentagem por um tempo e depois 100% vs. passos iguais em intervalos
Annotation vs. metadata (X-Ray)	Indexada e filtrável vs. só para leitura
CloudTrail vs. X-Ray vs. CloudWatch	Quem chamou a API vs. caminho da requisição vs. métricas e logs
Metric filter vs. subscription filter	Gera métrica a partir de logs vs. envia os logs em tempo real para outro serviço
Readiness vs. liveness	Pronto para receber tráfego vs. vivo (reinicia se falhar)

Afirmações falsas que aparecem como alternativa

- "Guardar access keys em variáveis de ambiente da Lambda é seguro" — **falso**, use a execution role.
- "Aumentar o timeout da Lambda resolve 504 do API Gateway" — **falso**, o limite é o timeout de integração do gateway.
- "FilterExpression reduz o consumo de RCU" — **falso**, filtra depois da leitura.
- "LSI pode ser criado depois da tabela" — **falso**.
- "GSI suporta leitura fortemente consistente" — **falso**.
- "SQS Standard garante entrega única e ordem" — **falso**.
- "KMS Encrypt criptografa arquivos de qualquer tamanho" — **falso**, até 4 KB.
- "API keys autenticam usuários" — **falso**, identificam clientes para usage plans.
- "Lambda em sub-rede pública acessa a internet" — **falso**, precisa de NAT Gateway.
- "A DLQ do Lambda recebe o erro da invocação" — **falso**, só o evento; use Destinations.
- "Mudanças no API Gateway entram em vigor sem deployment" — **falso** para REST (exceto auto-deploy em HTTP API).
- "Rotação automática do KMS muda o ID da chave" — **falso**.

16. Mapa de Domínios do Exame

Cada domínio oficial tem um peso diferente na nota final (32/26/24/18% no DVA-C02). Esta seção cruza os tópicos das seções 1 a 14 com o domínio que eles testam, para priorizar a revisão pelo que realmente pesa na prova.

Domínio 1 — Development with AWS Services (32%)

Tópico	Onde revisar
Padrões de arquitetura, stateful/stateless, acoplamento, síncrono/assíncrono	Seção 1
Resiliência: retries, backoff, jitter, idempotência, circuit breaker	Seção 1
SDK, CLI, APIs e chamadas autenticadas	Seções 1 e 7
APIs: transformações, validação, códigos de status	Seção 3
Testes unitários com SAM	Seção 10
Mensageria, EventBridge e streaming	Seção 6
Amazon Q Developer	Seções 10 e 14
Lambda: VPC, configuração, erros, testes, integração, desempenho, tempo quase real	Seção 2
DynamoDB: chaves, consistência, Query vs. Scan, índices, serialização	Seção 4
Cache, ciclo de vida de dados, armazenamentos especializados	Seções 4 e 5

Domínio 2 — Security (26%)

Tópico	Onde revisar
Federação com Cognito e IAM, bearer tokens	Seção 7
Acesso programático, chamadas autenticadas, assumir roles	Seções 1 e 7
Permissões de principals, autorização de granularidade fina	Seção 7
Autenticação entre serviços, multi-tenant	Seção 7
Criptografia em repouso e em trânsito, client-side vs. server-side	Seções 5 e 8
KMS: chaves, envelope, entre contas, rotação	Seção 8
Certificados (ACM, Private CA) e chaves SSH	Seção 8
Classificação de dados, variáveis de ambiente criptografadas, segredos	Seção 8
Sanitização e mascaramento	Seção 8

Domínio 3 — Deployment (24%)

Tópico	Onde revisar
Dependências, estrutura de arquivos, repositórios, requisitos de recursos	Seção 9
AppConfig e configuração por ambiente	Seção 9
Testes com serviços AWS, integração, mocks, stages, eventos	Seção 10

Tópico	Onde revisar
Templates SAM e CloudFormation, ambientes por serviço	Seção 9
Empacotamento de Lambda, stages e custom domains do API Gateway	Seções 2 e 3
CodeBuild, CodeDeploy, CodePipeline, CodeArtifact	Seção 11
Estratégias de implantação e rollback, labels e branches	Seção 11
Elastic Beanstalk, ECS/ECR	Seção 11

Domínio 4 — Troubleshooting and Optimization (18%)

Tópico	Onde revisar
Depurar código, interpretar métricas, logs e traces	Seção 12
Consultar logs, métricas personalizadas (EMF), dashboards	Seção 12
Falhas de implantação e de integração	Seção 12
Logging estruturado, tracing, annotations, alertas	Seção 12
Health checks e readiness probes	Seção 12
Concorrência, profiling, memória e computação	Seções 2 e 13
Filter policies, cache por headers, cache na aplicação	Seção 13
Gargalos a partir de logs, uso de recursos	Seção 13

Observação sobre o modelo de pontuação

O DVA-C02 usa pontuação **compensatória**: não é preciso atingir a nota mínima em cada domínio separadamente, só na prova como um todo (720). **Desenvolvimento (32%)** e **Segurança (26%)** somam 58% da nota: as seções 1 a 8 têm o maior retorno por hora de estudo. Lambda, API Gateway, DynamoDB, IAM/Cognito e KMS aparecem em praticamente todos os domínios.

17. Autoteste — Flashcards de Revisão Rápida

Cada card abaixo esconde a resposta: clique para expandir só depois de tentar responder mentalmente. O objetivo é forçar recall ativo, não releitura passiva. Uma rodada de 20 a 25 cards por dia nos dias antes da prova cobre todo o banco.

Padrões e SDK

Qual é a diferença entre coreografia e orquestração?

RESPOSTA

Na **coreografia**, cada serviço reage a eventos e publica novos eventos, sem coordenador central (EventBridge, SNS). Na **orquestração**, um coordenador central controla a ordem, os erros e as compensações (Step Functions).

O que torna uma aplicação stateless e por que isso importa?

RESPOSTA

Guardar o estado fora da instância ou função (sessão no ElastiCache ou DynamoDB, arquivos no S3). Qualquer instância pode atender qualquer requisição, o que permite escalar horizontalmente e substituir instâncias sem perder dados.

Por que usar backoff exponencial com jitter?

RESPOSTA

Para espaçar as tentativas cada vez mais e acrescentar aleatoriedade, evitando que muitos clientes tentem ao mesmo tempo e sobrecarreguem o serviço de novo.

Quais são os modos de retry dos SDKs da AWS?

RESPOSTA

legacy, standard (padrão, 3 tentativas com backoff e jitter) e adaptive (acrescenta limitação de taxa no cliente).

Por que idempotência é obrigatória em sistemas com SQS Standard, SNS e Lambda assíncrono?

RESPOSTA

Porque a entrega é **ao menos uma vez**: a mesma mensagem ou evento pode ser processado mais de uma vez, e o resultado precisa ser o mesmo.

Como implementar idempotência com DynamoDB?

RESPOSTA

Gravar uma chave de idempotência (ex.: ID do pedido) com `ConditionExpression attribute_not_exists` e TTL; se a condição falhar, o evento já foi processado.

O que faz um circuit breaker?

RESPOSTA

Depois de várias falhas seguidas em uma dependência, para de chamá-la por um tempo e devolve erro rápido ou fallback, testando de novo depois.

Qual comando mostra a conta e a identidade das credenciais em uso?

RESPOSTA

```
aws sts get-caller-identity.
```

Como decodificar uma mensagem de erro de autorização codificada?

RESPOSTA

```
aws sts decode-authorization-message, com a permissão sts:DecodeAuthorizationMessage.
```

Qual é a diferença entre `--query` e `--filter` na CLI?

RESPOSTA

`--query` filtra a saída **no cliente** com JMESPath; `--filter` (quando disponível) filtra **no servidor**.

Em que ordem geral o SDK procura credenciais?

RESPOSTA

Parâmetros no código → variáveis de ambiente → arquivos `~/.aws/credentials` e `config` → credenciais do contêiner (task role) → metadados da instância EC2.

O que é o padrão claim check?

RESPOSTA

Guardar o payload grande no S3 e enviar só a referência na mensagem da fila ou do evento.

Lambda

Qual é o timeout máximo de uma função Lambda?

RESPOSTA

900 segundos (15 minutos). O padrão é 3 segundos.

Como a memória da Lambda afeta a CPU?

RESPOSTA

A CPU é proporcional à memória; com 1.769 MB a função tem o equivalente a 1 vCPU. Aumentar memória pode reduzir o tempo e até o custo.

Quantas vezes o Lambda tenta de novo uma invocação assíncrona que falhou?

RESPOSTA

2 vezes (3 execuções no total), configurável de 0 a 2, com idade máxima do evento de até 6 horas.

Qual é a diferença entre DLQ e Destinations na Lambda?

RESPOSTA

A **DLQ** (SQS/SNS) recebe só o evento que falhou. As **Destinations** recebem evento, resposta ou erro e metadados, para sucesso e falha, com destinos SQS, SNS, Lambda, EventBridge (e S3 para falha).

O que o `ReportBatchItemFailures` resolve?

RESPOSTA

Permite que a função informe **quais mensagens do lote falharam**, evitando reprocessar o lote inteiro (SQS, Kinesis, DynamoDB Streams).

Qual visibility timeout configurar na fila SQS usada por uma Lambda?

RESPOSTA

Pelo menos **6 vezes o timeout da função**.

Como evitar que um registro ruim bloqueie um shard do Kinesis?

RESPOSTA

`BisectBatchOnFunctionError`, `MaximumRetryAttempts`, `MaximumRecordAgeInSeconds` e um on-failure destination.

O que significa a métrica `IteratorAge` crescendo?

RESPOSTA

O consumidor do stream está ficando para trás em relação aos produtores.

Qual é a concorrência padrão do Lambda por conta e Região?

RESPOSTA

1.000 execuções simultâneas (aumentável).

Qual é a diferença entre reserved e provisioned concurrency?

RESPOSTA

Reserved garante e limita a concorrência de uma função, sem custo. **Provisioned** mantém ambientes pré-inicializados para eliminar cold starts, com custo, em uma versão ou alias.

Como desligar imediatamente uma função Lambda sem apagá-la?

RESPOSTA

Definir a **reserved concurrency** em 0.

O que é o SnapStart?

RESPOSTA

Um snapshot do ambiente já inicializado, restaurado nas novas execuções para reduzir cold starts (Java, Python e .NET).

Onde colocar a criação de clientes do SDK e conexões em uma Lambda?

RESPOSTA

Fora do handler, no código de inicialização, para reutilizá-los entre invocações.

Qual é o limite do pacote .zip e da imagem de contêiner da Lambda?

RESPOSTA

.zip: 50 MB compactado no upload direto e 250 MB descompactado (com layers). Imagem de contêiner: 10 GB.

Quantas layers uma função pode ter?

RESPOSTA

5.

Qual é o limite das variáveis de ambiente de uma função?

RESPOSTA

4 KB no total.

Qual é o payload máximo de uma invocação síncrona e de uma assíncrona?

RESPOSTA

Síncrona: **6 MB** (streaming até 200 MB). Assíncrona: **1 MB** (era 256 KB).

Uma função em VPC precisa chamar uma API na internet. O que é necessário?

RESPOSTA

A função em **sub-rede privada** com rota para um **NAT Gateway** (funções em VPC não têm IP público).

Qual política gerenciada dá a uma Lambda permissão para criar ENIs na VPC?

RESPOSTA

`AWSLambdaVPCLambdaAccessExecutionRole`.

Como evitar esgotar as conexões do RDS com muitas execuções da Lambda?

RESPOSTA

Usar o **RDS Proxy**.

Qual é a diferença entre uma versão e um alias da Lambda?

RESPOSTA

A **versão** é um snapshot imutável de código e configuração. O **alias** é um ponteiro com nome para uma versão (ou duas, com pesos), atualizável sem mudar quem chama.

O que o S3 precisa para invocar uma função Lambda?

RESPOSTA

Uma **resource-based policy** na função permitindo `lambda:InvokeFunction` para `s3.amazonaws.com` (com `SourceArn` do bucket).

Quais tipos de autenticação uma Lambda Function URL aceita?

RESPOSTA

`AWS_IAM` (SigV4) ou `NONE` (pública).

Quando usar CloudFront Functions em vez de Lambda@Edge?

RESPOSTA

Para lógica leve em viewer request/response (reescrita de URL, headers, redirecionamentos), com latência de submilissegundo e altíssima escala. Lambda@Edge serve para lógica mais pesada, eventos de origem e acesso à rede.

Como uma função de transformação do Firehose deve responder?

RESPOSTA

Com cada registro contendo `recordId`, `result` (`Ok`, `Dropped` ou `ProcessingFailed`) e `data` em base64.

API Gateway

Cite três recursos que existem na REST API e não na HTTP API.

RESPOSTA

Cache, usage plans e API keys, validação de requisição, mapping templates completos, AWS WAF, endpoint privado e canary release.

O que causa um erro 502 em uma integração Lambda proxy?

RESPOSTA

Resposta malformada da função: faltando `statusCode` ou com `body` que não é string, ou exceção não tratada.

O que causa um erro 504 no API Gateway?

RESPOSTA

Timeout de integração (padrão de 29 s em REST).

O timeout de 29 s do API Gateway pode ser aumentado?

RESPOSTA

Sim, para APIs REST **regionais e privadas**, por cota (possivelmente reduzindo o throttling). Com **response streaming**, integrações de até 15 minutos.

Qual é o limite padrão de throttling do API Gateway por conta e Região?

RESPOSTA

10.000 requisições por segundo, com burst de **5.000**.

Para que servem as stage variables?

RESPOSTA

Configuração por stage usada em integrações e templates, por exemplo para apontar cada stage para um alias diferente da Lambda.

O que é preciso fazer depois de alterar uma REST API para a mudança valer?

RESPOSTA

Criar um **deployment** em um stage.

Onde deve estar o certificado do ACM para um custom domain edge-optimized?

RESPOSTA

Na **us-east-1** (N. Virgínia). Para um domínio regional, na mesma Região da API.

Qual é a diferença entre um Lambda authorizer TOKEN e REQUEST?

RESPOSTA

TOKEN recebe só um token (ex.: header `Authorization`). **REQUEST** recebe headers, query strings, stage variables e contexto.

API keys são um mecanismo de autenticação?

RESPOSTA

Não. Identificam clientes para usage plans (throttling e cota); combine com outro mecanismo de autorização.

Qual é o TTL padrão e máximo do cache do API Gateway?

RESPOSTA

Padrão de **300 s**, máximo de **3.600 s**.

Como um cliente pode invalidar o cache do API Gateway?

RESPOSTA

Enviando `Cache-Control: max-age=0`, se tiver a permissão `execute-api:InvalidateCache`.

Como validar o corpo da requisição sem invocar o backend?

RESPOSTA

Com um **request validator** e um **model** (JSON Schema) no método.

Qual integração devolve uma resposta fixa sem backend?

RESPOSTA

Mock.

O que mostra a métrica `IntegrationLatency`?

RESPOSTA

O tempo gasto pelo **backend**; Latency é o tempo total no API Gateway.

Em uma Lambda proxy com CORS, quem precisa devolver o header `Access-Control-Allow-Origin`?

RESPOSTA

A **função Lambda**, na resposta.

DynamoDB

Quanto vale 1 RCU e 1 WCU?

RESPOSTA

1 RCU = 1 leitura fortemente consistente por segundo de até 4 KB (ou 2 eventualmente consistentes). **1 WCU** = 1 escrita por segundo de até 1 KB. Transações custam o dobro.

Quantas RCU para 10 leituras fortemente consistentes por segundo de itens de 6 KB?

RESPOSTA

6 KB → 2 blocos de 4 KB → $2 \times 10 =$ **20 RCU**.

Quantas RCU para 10 leituras eventualmente consistentes por segundo de itens de 6 KB?

RESPOSTA

10 RCU (metade das fortemente consistentes).

Quantas WCU para 5 escritas por segundo de itens de 2,5 KB?

RESPOSTA

2,5 KB → 3 blocos de 1 KB → **15 WCU**.

Qual é o tamanho máximo de um item no DynamoDB?

RESPOSTA

400 KB.

Por que usar partition keys de alta cardinalidade?

RESPOSTA

Para distribuir leituras e escritas de forma uniforme entre partições e evitar partições quentes e throttling.

Cite três diferenças entre LSI e GSI.

RESPOSTA

LSI usa a mesma partition key, só pode ser criado com a tabela, suporta leitura forte e há até 5. GSI usa qualquer chave, pode ser criado a qualquer momento, só tem consistência eventual, tem capacidade própria e há até 20.

Por que `FilterExpression` não reduz o custo de uma Query ou Scan?

RESPOSTA

Porque o filtro é aplicado **depois** da leitura; a capacidade é consumida pelos itens lidos.

Como continuar uma Query que retornou mais de 1 MB?

RESPOSTA

Usar o `LastEvaluatedKey` retornado como `ExclusiveStartKey` na próxima chamada.

Como acelerar um Scan em uma tabela grande?

RESPOSTA

Parallel scan, com `Segment` e `TotalSegments`.

Como impedir que um `PutItem` sobrescreva um item existente?

RESPOSTA

`ConditionExpression: attribute_not_exists(pk)`.

Como funciona o bloqueio otimista no DynamoDB?

RESPOSTA

Um atributo de versão é verificado em uma `ConditionExpression` a cada escrita e incrementado; se outro processo gravou antes, ocorre `ConditionalCheckFailedException`.

Quais são os limites de `BatchWriteItem` e `TransactWriteItems`?

RESPOSTA

`BatchWriteItem`: até **25 itens** e 16 MB, sem update nem condição. `TransactWriteItems`: até **100 ações** e 4 MB, tudo ou nada.

O que fazer com `UnprocessedItems` em um `BatchWriteItem`?

RESPOSTA

Reenviá-los com **backoff exponencial**.

Como funciona o TTL do DynamoDB?

RESPOSTA

Um atributo `Number` com o timestamp Unix em **segundos**; itens expirados são apagados em segundo plano, em poucos dias, sem consumir WCU.

Quanto tempo o DynamoDB Streams retém os registros e quais são as visões possíveis?

RESPOSTA

24 horas; `KEYS_ONLY`, `NEW_IMAGE`, `OLD_IMAGE` e `NEW_AND_OLD_IMAGES`.

O DAX serve leituras fortemente consistentes do cache?

RESPOSTA

Não. Só eventualmente consistentes; as fortes vão direto à tabela.

Qual é o período de retenção do PITR do DynamoDB?

RESPOSTA

Configurável de **1 a 35 dias**; a restauração cria uma nova tabela.

Qual modo de capacidade usar para tráfego imprevisível?

RESPOSTA

On-demand.

O que é um índice esperso?

RESPOSTA

Um GSI que contém só os itens que têm o atributo da chave do índice, útil para consultar um subconjunto.

Armazenamento e cache

Como permitir que um usuário sem credenciais AWS envie um arquivo direto ao S3?

RESPOSTA

Gerando uma **presigned URL** de PUT no backend.

Quando o multipart upload é obrigatório?

RESPOSTA

Acima de **5 GB** (limite do PUT único); é recomendado acima de 100 MB.

Qual é o desempenho mínimo do S3 por prefixo?

RESPOSTA

3.500 PUT/COPY/POST/DELETE e 5.500 GET/HEAD por segundo por prefixo.

Como reduzir as chamadas ao KMS em um bucket com SSE-KMS?

RESPOSTA

Habilitar **S3 Bucket Keys**.

O que o SSE-C exige?

RESPOSTA

Que o cliente envie a chave em cada requisição, por **HTTPS**.

Como forçar HTTPS em um bucket S3?

RESPOSTA

Bucket policy negando requisições com `aws:SecureTransport = false`.

Qual endpoint do Aurora usar para distribuir leituras?

RESPOSTA

O **reader endpoint**.

Como conectar ao RDS sem senha no código?

RESPOSTA

Autenticação IAM do RDS (token de 15 minutos) ou credenciais no Secrets Manager.

Qual é a diferença entre lazy loading e write-through?

RESPOSTA

Lazy loading grava no cache no miss (pode ficar desatualizado). **Write-through** atualiza o cache em cada escrita (pode guardar dados nunca lidos). Combine com TTL.

Onde guardar sessões de usuário para tornar a aplicação stateless?

RESPOSTA

ElastiCache ou **DynamoDB**.

Qual serviço usar para busca de texto completo com relevância?

RESPOSTA

Amazon OpenSearch Service.

Mensageria e orquestração

Qual é o visibility timeout padrão e o máximo do SQS?

RESPOSTA

Padrão de **30 s**, máximo de **12 horas**.

Qual é a retenção padrão e a máxima de mensagens no SQS?

RESPOSTA

Padrão de **4 dias**, máximo de **14 dias**.

Qual é o tamanho máximo de uma mensagem SQS e como enviar mais?

RESPOSTA

1 MiB (era 256 KB). Maior: **Extended Client Library** com o payload no S3 (até 2 GB).

O que é long polling e qual o tempo máximo de espera?

RESPOSTA

ReceiveMessage espera até chegar mensagem, por até **20 segundos**, reduzindo respostas vazias e custo.

Qual é o atraso máximo de uma delay queue?

RESPOSTA

15 minutos.

Como o SQS FIFO garante ordem com paralelismo?

RESPOSTA

Ordem garantida por MessageGroupId ; grupos diferentes são processados em paralelo.

Como o SQS FIFO deduplica mensagens?

RESPOSTA

Com MessageDeduplicationId ou deduplicação baseada no conteúdo, em uma janela de **5 minutos**.

O que o `maxReceiveCount` controla?

RESPOSTA

Quantas vezes uma mensagem pode ser recebida sem ser apagada antes de ir para a **DLQ**.

Como estender o tempo de processamento de uma mensagem já recebida?

RESPOSTA

`ChangeMessageVisibility`.

O que é uma filter policy do SNS?

RESPOSTA

Um filtro por assinatura, aplicado aos atributos ou ao corpo da mensagem, para que cada assinante receba só as mensagens relevantes.

O que a fila SQS precisa para receber mensagens de um tópico SNS?

RESPOSTA

Uma **política de acesso** permitindo `sqs:SendMessage` com o ARN do tópico como origem.

Por quanto tempo o EventBridge tenta entregar um evento por padrão?

RESPOSTA

24 horas, até **185 tentativas**; depois, DLQ se configurada.

O que é o EventBridge archive e replay?

RESPOSTA

Guardar eventos e reproduzi-los depois, para reprocessar ou testar.

O que é o EventBridge Pipes?

RESPOSTA

Integração ponto a ponto fonte → filtro → enriquecimento → destino, sem código de cola.

Qual é a capacidade de escrita de um shard do Kinesis?

RESPOSTA

1 MB/s ou **1.000 registros/s**; leitura de 2 MB/s.

O que o enhanced fan-out do Kinesis oferece?

RESPOSTA

2 MB/s por shard para cada consumidor, via push, com menor latência.

Qual é a retenção do Kinesis Data Streams?

RESPOSTA

Padrão de 24 horas, até **365 dias**.

Quando escolher Kinesis em vez de SQS?

RESPOSTA

Quando é preciso ordem, vários consumidores do mesmo dado e replay.

Qual é a diferença entre Step Functions Standard e Express?

RESPOSTA

Standard: até 1 ano, exatamente uma vez, histórico auditável, preço por transição. **Express:** até 5 minutos, alto volume, preço por execução e duração.

Como implementar aprovação humana no Step Functions?

RESPOSTA

Padrão `.waitForTaskToken`: a execução pausa até `SendTaskSuccess` ou `SendTaskFailure` com o token.

Qual é a diferença entre `Retry` e `Catch` no Step Functions?

RESPOSTA

`Retry` tenta de novo com backoff; `Catch` desvia para um estado de tratamento ou compensação quando os retries se esgotam.

Qual é o payload máximo entre estados do Step Functions?

RESPOSTA

256 KB; para mais, guarde no S3 e passe a referência.

Quando usar AppSync?

RESPOSTA

Para APIs GraphQL que combinam várias fontes de dados e oferecem atualizações em tempo real (subscriptions).

Autenticação e autorização

Qual é a diferença entre Cognito User Pools e Identity Pools?

RESPOSTA

User Pools autenticam usuários e emitem JWT. **Identity Pools** trocam identidades por **credenciais AWS temporárias**.

Quais tokens um User Pool emite e para que servem?

RESPOSTA

ID token (informações do usuário), **access token** (escopos e acesso a APIs) e **refresh token** (novos tokens sem novo login).

O que validar em um JWT recebido pelo backend?

RESPOSTA

Assinatura (chaves do JWKS), emissor (`iss`), público (`aud / client_id`), tipo (`token_use`) e expiração (`exp`).

Qual Lambda trigger do Cognito adiciona claims personalizados ao token?

RESPOSTA

Pre token generation.

Qual fluxo OAuth 2.0 usar para comunicação máquina a máquina no Cognito?

RESPOSTA

Client credentials.

Como limitar cada usuário ao próprio prefixo do S3 com Identity Pools?

RESPOSTA

Política com a variável `${cognito-identity.amazonaws.com:sub}` no caminho do recurso.

Qual condição limita o usuário aos próprios itens no DynamoDB?

RESPOSTA

`dynamodb:LeadingKeys` .

Qual é a diferença entre task role e task execution role no ECS?

RESPOSTA

Task role: permissões da aplicação. **Task execution role:** permissões do agente do ECS para puxar imagens, enviar logs e buscar segredos.

Como acessar recursos de outra conta sem criar usuários lá?

RESPOSTA

Assumir uma role na outra conta com `sts:AssumeRole` , confiada pela trust policy (e com External ID para terceiros).

Qual API do STS usar para chamar APIs que exigem MFA com um usuário IAM?

RESPOSTA

GetSessionToken com MFA.

Como um workflow do GitHub Actions pode implantar na AWS sem access keys?

RESPOSTA

Com **OIDC** e AssumeRoleWithWebIdentity.

Qual é a duração máxima de uma sessão de role e o limite de role chaining?

RESPOSTA

Até **12 horas** (conforme o MaxSessionDuration da role); role chaining é limitado a **1 hora**.

O que é o Amazon Verified Permissions?

RESPOSTA

Serviço de autorização da aplicação com políticas **Cedar**, para permissões de granularidade fina fora do código.

Qual é a forma recomendada de autenticação entre microsserviços na AWS?

RESPOSTA

Roles do IAM com SigV4 e políticas de recurso (ou OAuth 2.0 client credentials e mTLS quando necessário).

Por que não confiar em um `tenantId` enviado no corpo da requisição?

RESPOSTA

Porque o cliente pode forjá-lo; o tenant deve vir do **token validado**.

Criptografia e segredos

Qual é o limite de dados da operação `kms:Encrypt` ?

RESPOSTA

4 KB.

Explique a envelope encryption.

RESPOSTA

`GenerateDataKey` devolve uma chave de dados em claro e criptografada; a chave em claro criptografa os dados localmente e é descartada; a criptografada é guardada com os dados e descriptografada pelo KMS na leitura.

Para que serve `ReEncrypt` ?

RESPOSTA

Trocar a chave que protege um texto cifrado sem expor o texto claro.

O que é o `encryption context`?

RESPOSTA

Pares chave-valor não secretos ligados ao texto cifrado, exigidos no `Decrypt` e registrados no `CloudTrail`.

O que é necessário para outra conta usar uma chave do KMS?

RESPOSTA

Uma **customer managed key** com key policy permitindo a outra conta e uma política do IAM na outra conta.

Qual é o período da rotação automática de `customer managed keys`?

RESPOSTA

Configurável de **90 a 2.560 dias** (padrão 365); também há rotação sob demanda. O ID da chave não muda.

Como rotacionar uma chave com material importado?

RESPOSTA

Manualmente: criar uma nova chave e apontar o **alias** para ela.

Como resolver `ThrottlingException` do KMS em alto volume?

RESPOSTA

Cache de chaves de dados (AWS Encryption SDK), S3 Bucket Keys, retries com backoff ou aumento de cota.

Qual é a diferença entre criptografia do lado do cliente e do lado do servidor?

RESPOSTA

Client-side: a aplicação criptografa antes de enviar; o serviço só vê texto cifrado. **Server-side:** o serviço criptografa ao gravar.

Para que serve a AWS Private CA?

RESPOSTA

Emitir certificados privados para uso interno, como mTLS entre microsserviços e dispositivos IoT.

Certificados importados no ACM são renovados automaticamente?

RESPOSTA

Não; só os emitidos pelo ACM.

Qual é a diferença entre Secrets Manager e Parameter Store?

RESPOSTA

Secrets Manager: rotação automática, replicação, compartilhamento entre contas, pago. **Parameter Store**: configuração hierárquica, `SecureString`, sem rotação nativa, camada Standard gratuita.

Quais são os labels de versão de um segredo durante a rotação?

RESPOSTA

`AWSPENDING` (nova), `AWSCURRENT` (atual) e `AWSPREVIOUS` (anterior).

Como compartilhar um segredo do Secrets Manager com outra conta?

RESPOSTA

Resource policy no segredo + customer managed key do KMS com permissão para a outra conta.

Como ler segredos em Lambda com menor latência e custo?

RESPOSTA

Com a **AWS Parameters and Secrets Lambda Extension** (cache local).

Como proteger uma variável de ambiente sensível para que não apareça em claro no console?

RESPOSTA

Helpers de criptografia em trânsito com uma chave do KMS, e `kms:Decrypt` no código.

O que são PII e PHI?

RESPOSTA

PII: dados que identificam uma pessoa. **PHI**: informações de saúde ligadas a uma pessoa.

Como mascarar dados sensíveis nos logs do CloudWatch?

RESPOSTA

Com uma **data protection policy** no log group.

Empacotamento, IaC e testes

Qual linha transforma um template CloudFormation em SAM?

RESPOSTA

Transform: AWS::Serverless-2016-10-31.

Como testar uma função localmente com um evento do S3?

RESPOSTA

`sam local generate-event s3 put` e `sam local invoke -e evento.json`.

Como implantar o mesmo template SAM em outro ambiente?

RESPOSTA

`sam deploy --config-env <ambiente>` (seção do `samconfig.toml`), criando outra stack com outros parâmetros.

Como o SAM faz implantação canary de uma Lambda?

RESPOSTA

AutoPublishAlias + DeploymentPreference (ex.: `Canary10Percent5Minutes`), com hooks e alarmes, usando o CodeDeploy.

Qual é a única seção obrigatória de um template CloudFormation?

RESPOSTA

`Resources`.

Qual é a diferença entre change set e drift detection?

RESPOSTA

Change set mostra o que vai mudar antes de atualizar; **drift detection** mostra o que mudou fora do CloudFormation.

Como manter um recurso ao apagar a stack?

RESPOSTA

`DeletionPolicy: Retain` (ou `Snapshot` quando suportado).

Como usar um valor de uma stack em outra?

RESPOSTA

Outputs com `Export` e `Fn::ImportValue` na outra stack.

Como referenciar um segredo no template sem expô-lo?

RESPOSTA

Referência dinâmica `{{resolve:secretsmanager:...}}` (ou `{{resolve:ssm-secure:...}}`).

Qual capacidade é exigida por templates que criam roles com nome personalizado?

RESPOSTA

`CAPABILITY_NAMED_IAM`.

O que um custom resource com Lambda precisa fazer ao terminar?

RESPOSTA

Responder `SUCCESS` ou `FAILED` para a URL pré-assinada enviada pelo CloudFormation.

Para que servem `cfn-init` e `cfn-signal`?

RESPOSTA

`cfn-init` aplica a configuração do `AWS::CloudFormation::Init`; `cfn-signal` avisa o CloudFormation (com `CreationPolicy`) que a instância terminou.

O que o `cdk bootstrap` faz?

RESPOSTA

Cria os recursos de apoio do CDK (bucket, repositório, roles) uma vez por conta e Região.

Qual serviço implanta feature flags gradualmente com rollback por alarme?

RESPOSTA

AWS AppConfig.

Como testar um padrão de regra do EventBridge?

RESPOSTA

Com a API `TestEventPattern`.

Como testar um estado do Step Functions isoladamente?

RESPOSTA

Com a **TestState API**.

Como o frontend pode testar a API antes do backend existir?

RESPOSTA

Com a **integração mock** do API Gateway.

Como capturar mensagens publicadas em um tópico durante um teste?

RESPOSTA

Assinar uma fila SQS de teste no tópico e verificar as mensagens.

CI/CD e implantação

Quais são as fases do `buildspec.yml`?

RESPOSTA

`install`, `pre_build`, `build` e `post_build`.

Como usar um segredo no CodeBuild sem colocá-lo em texto?

RESPOSTA

Seções `parameter-store` ou `secrets-manager` em `env` no `buildspec.yml`.

Como acelerar builds que baixam as mesmas dependências?

RESPOSTA

Cache do CodeBuild (S3 ou local).

Qual é a ordem dos hooks do CodeDeploy em uma implantação in-place no EC2?

RESPOSTA

`ApplicationStop` → `DownloadBundle` → `BeforeInstall` → `Install` → `AfterInstall` → `ApplicationStart` → `ValidateService`.

Quais hooks o CodeDeploy usa em implantações de Lambda?

RESPOSTA

`BeforeAllowTraffic` e `AfterAllowTraffic`.

Qual hook do CodeDeploy no ECS permite testar a nova versão no listener de teste?

RESPOSTA

`AfterAllowTestTraffic`.

Qual é a diferença entre `Canary10Percent5Minutes` e `Linear10PercentEvery1Minute` ?

RESPOSTA

Canary: 10% por 5 minutos e depois 100%. Linear: mais 10% a cada minuto até 100%.

Como o CodeDeploy faz rollback?

RESPOSTA

Reimplanta a última revisão boa como uma **nova implantação**, automaticamente em falha ou alarme.

O CodeDeploy precisa de agente em Lambda e ECS?

RESPOSTA

Não; só em EC2 e on-premises.

Como adicionar aprovação humana antes de produção no CodePipeline?

RESPOSTA

Ação de **aprovação manual** (com notificação SNS).

Onde o CodePipeline guarda os artefatos entre ações?

RESPOSTA

Em um **bucket S3** (artifact store), criptografado com KMS.

Qual política do Beanstalk mantém a capacidade total com menor custo extra?

RESPOSTA

Rolling with additional batch.

Qual política do Beanstalk tem o rollback mais rápido e seguro?

RESPOSTA

Immutable.

Como fazer blue/green no Beanstalk?

RESPOSTA

Clonar o ambiente, implantar no novo e **trocar as URLs** (Swap environment URLs).

Para que serve `leader_only: true` em `container_commands` ?

RESPOSTA

Executar o comando (ex.: migração de banco) em uma única instância.

Por que não criar o RDS dentro do ambiente Beanstalk em produção?

RESPOSTA

Porque ele é apagado junto com o ambiente.

O que o ECS deployment circuit breaker faz?

RESPOSTA

Detecta implantações que não ficam saudáveis e faz rollback automático.

Como impedir que tags de imagem sejam sobrescritas no ECR?

RESPOSTA

Ativar **tag immutability**.

Para que serve o CodeArtifact?

RESPOSTA

Repositório gerenciado de pacotes (npm, PyPI, Maven...) com upstreams e cópias de repositórios públicos.

Observabilidade e otimização

Qual é a diferença entre logging, monitoramento e observabilidade?

RESPOSTA

Logging registra eventos; monitoramento acompanha métricas conhecidas e alerta; observabilidade permite entender o estado interno e problemas não previstos a partir de logs, métricas e traces correlacionados.

O que é o Embedded Metric Format (EMF)?

RESPOSTA

Um formato de log JSON do qual o CloudWatch extrai métricas de forma assíncrona, sem chamadas `PutMetricData`.

Qual é a diferença entre metric filter e subscription filter?

RESPOSTA

Metric filter cria métricas a partir de padrões nos logs; **subscription filter** envia os logs em tempo real para Lambda, Kinesis, Firehose ou OpenSearch.

Qual é a diferença entre annotations e metadata no X-Ray?

RESPOSTA

Annotations são indexadas e usadas em filter expressions; **metadata** não é indexada.

Qual é a regra de sampling padrão do X-Ray?

RESPOSTA

A primeira requisição de cada segundo e **5%** das demais.

Em que porta o daemon do X-Ray escuta?

RESPOSTA

UDP 2000.

Qual é a recomendação atual para instrumentar traces enviados ao X-Ray?

RESPOSTA

AWS Distro for OpenTelemetry (ADOT) ou OpenTelemetry; os SDKs do X-Ray estão em modo de manutenção desde fev/2026.

Qual permissão a execution role precisa para o active tracing da Lambda?

RESPOSTA

xray:PutTraceSegments e xray:PutTelemetryRecords (política AWSXRayDaemonWriteAccess).

Quais campos da linha `REPORT` da Lambda ajudam no troubleshooting?

RESPOSTA

Duration, Billed Duration, Memory Size, Max Memory Used e Init Duration (cold start).

Como ser alertado ao chegar perto de uma cota de serviço?

RESPOSTA

Métrica de uso do **Service Quotas** (AWS/Usage) com alarme do CloudWatch.

Onde procurar a causa de uma falha de implantação do CloudFormation?

RESPOSTA

Na aba **Events** da stack, a partir do primeiro evento `CREATE_FAILED` ou `UPDATE_FAILED`.

Qual é a diferença entre readiness e liveness probe?

RESPOSTA

Readiness: se o contêiner está pronto para receber tráfego. **Liveness:** se está vivo; se falhar, é reiniciado.

Como calcular a concorrência necessária?

RESPOSTA

Requisições por segundo $\times$ duração média em segundos.

Como encontrar a memória ideal de uma Lambda?

RESPOSTA

Com o **AWS Lambda Power Tuning** ou as recomendações do **Compute Optimizer**.

Como melhorar a taxa de acerto do cache do CloudFront com conteúdo por idioma?

RESPOSTA

Incluir **apenas** o header necessário (ex.: Accept-Language) na cache policy.

Como evitar invocar a Lambda para mensagens que ela descartaria?

RESPOSTA

Filter criteria no event source mapping, filter policies do SNS ou padrões do EventBridge.

Qual serviço do CloudWatch simula usuários testando endpoints periodicamente?

RESPOSTA

CloudWatch Synthetics (canaries).

IA no desenvolvimento

Quais habilidades do DVA-C02 citam o Amazon Q Developer?

RESPOSTA

Usar o Q Developer para assistência ao desenvolvimento (1.1.11) e para gerar testes automatizados (3.3.6).

Qual é o sucessor do Amazon Q Developer?

RESPOSTA

Kiro (o Q Developer não aceita novas assinaturas desde 15/mai/2026).

Cite três controles de segurança ao integrar um modelo de IA na aplicação.

RESPOSTA

IAM de menor privilégio para invocar modelos, Bedrock Guardrails nas entradas e saídas, PrivateLink, mascaramento de PII e logs sem conteúdo sensível.

Como proteger ações de um agente de IA?

RESPOSTA

Autorizar cada uso de ferramenta (Policy in AgentCore, permissões mínimas), isolar sessões e exigir aprovação humana para ações sensíveis.

Quando o DVA-C02 deixa de ser aplicado e o DVA-C03 começa?

RESPOSTA

O último dia do DVA-C02 é **30/nov/2026**; o DVA-C03 começa em **1º/dez/2026**.

Pegadinhas Duplas (confusões mais recorrentes)

Execution role ou resource-based policy: qual responde "a função não consegue ler o bucket" e "o bucket não consegue invocar a função"?

RESPOSTA

Não consegue ler → **execution role**. Não consegue ser invocada → **resource-based policy** da função.

Visibility timeout, delay queue ou retenção: qual responde "mensagem reprocessada durante o processamento", "atrasar a primeira entrega" e "mensagens sumindo depois de dias"?

RESPOSTA

Reprocessada → **visibility timeout**. Atrasar → **delay queue**. Sumindo → **retenção**.

502, 504 ou 429 no API Gateway: qual responde "resposta malformada", "timeout" e "limite excedido"?

RESPOSTA

Malformada → **502**. Timeout → **504**. Limite → **429**.

Change set, drift detection ou stack policy: qual responde "o que vai mudar", "o que mudou fora" e "impedir atualização de um recurso crítico"?

RESPOSTA

O que vai mudar → **change set**. O que mudou fora → **drift detection**. Impedir atualização → **stack policy**.

Reserved, provisioned ou unreserved concurrency: qual responde "limitar a 50", "sem cold start" e "o restante compartilhado da conta"?

RESPOSTA

Limitar → **reserved**. Sem cold start → **provisioned**. Restante → **unreserved**.

18. Mapa de cobertura do guia oficial e questões de múltipla resposta

O [guia oficial do DVA-C02](#) (versão 2.1) divide a prova em **13 tarefas**. A tabela localiza a revisão de cada tarefa neste material. Cobrir as tarefas publicadas **não garante** conhecer todas as questões: o guia não é uma lista exaustiva, e a [lista de serviços no escopo](#) está sujeita a alterações. Para o DVA-C03, confira o novo guia quando for publicado (27/out/2026).

Tarefa oficial	Decisões e conceitos a dominar	Onde revisar
1.1 Código para aplicações na AWS	Padrões de arquitetura, stateless, acoplamento, assíncrono, resiliência, APIs, testes unitários, mensageria, SDK, streaming, Q Developer, EventBridge, integrações com terceiros	Seções 1, 3, 6, 10 e 14
1.2 Código para AWS Lambda	VPC, configuração, Destinations e DLQ, testes, integrações, desempenho, tempo quase real	Seção 2
1.3 Armazenamentos de dados	Partition keys, consistência, Query vs. Scan, chaves e índices, serialização, ciclo de vida, cache, armazenamentos especializados	Seções 4 e 5
2.1 Autenticação e autorização	Federação (Cognito, IAM), bearer tokens, acesso programático, assumir roles, permissões, autorização fina, entre serviços	Seção 7
2.2 Criptografia	Em repouso e em trânsito, certificados, client-side vs. server-side, chaves, entre contas, rotação	Seções 5 e 8
2.3 Dados sensíveis	Classificação, variáveis de ambiente, segredos, sanitização, mascaramento, multi-tenant	Seções 7 e 8
3.1 Preparar artefatos	Dependências, estrutura, repositórios, requisitos de recursos, AppConfig	Seção 9
3.2 Testar em desenvolvimento	Testes com serviços, integração e mocks, stages, atualizar stacks em outros ambientes, eventos	Seções 9 e 10
3.3 Automatizar testes de implantação	Eventos de teste, APIs em vários ambientes, versões aprovadas, IaC, ambientes por serviço, testes com Q Developer	Seções 9, 10 e 11
3.4 CI/CD	Empacotamento de Lambda, stages e domínios, atualizar IaC, ambientes, estratégias, commits, fluxos, rollback, labels, stage variables	Seções 2, 3, 9 e 11
4.1 Análise de causa raiz	Depurar, interpretar métricas/logs/traces, consultar logs, EMF, dashboards, falhas de implantação e integração	Seção 12
4.2 Instrumentar para observabilidade	Logging vs. monitoramento vs. observabilidade, logs estruturados, métricas, annotations, alertas, tracing, health checks	Seção 12
4.3 Otimizar aplicações	Concorrência, profiling, memória, filter policies, cache por headers, cache na aplicação, recursos, gargalos	Seções 2 e 13

Como resolver múltipla resposta

O exame inclui questões de **múltiplas respostas**, com duas ou mais opções corretas entre cinco ou mais alternativas. Não há crédito parcial: marque **exatamente a quantidade pedida** e avalie cada alternativa isoladamente.

1. Uma função Lambda processa mensagens de uma fila SQS e algumas mensagens estão sendo processadas mais de uma vez. Escolha DUAS ações: (A) aumentar o visibility timeout da fila para pelo menos 6 vezes o timeout da função; (B) tornar o processamento idempotente; (C) reduzir a retenção da fila; (D) trocar para short polling; (E) aumentar a reserved concurrency.

RESPOSTA

A e B. O visibility timeout curto faz a mensagem reaparecer durante o processamento, e a entrega ao menos uma vez exige idempotência. C, D e E não resolvem. Revise as seções 2 e 6.

2. Um app mobile precisa que usuários façam login com Google e enviem fotos direto para o próprio prefixo no S3. Escolha DUAS: (A) Cognito User Pool com federação do Google; (B) Cognito Identity Pool com role usando `$_cognito-identity.amazonaws.com:sub`; (C) access keys de um usuário IAM no app; (D) bucket público; (E) um usuário IAM por cliente.

RESPOSTA

A e B. O User Pool autentica e o Identity Pool entrega credenciais temporárias limitadas ao prefixo do usuário. C, D e E são inseguras ou não escalam. Revise a seção 7.

3. Uma aplicação precisa criptografar arquivos de 200 MB com uma chave do KMS. Escolha DUAS: (A) chamar `GenerateDataKey` e criptografar localmente com a chave de dados; (B) guardar a chave de dados criptografada junto com o arquivo; (C) chamar `Encrypt` com o arquivo inteiro; (D) guardar a chave de dados em claro no S3; (E) usar a AWS managed key para compartilhar com outra conta.

RESPOSTA

A e B. Envelope encryption. `Encrypt` aceita até 4 KB, guardar a chave em claro anula a proteção e AWS managed keys não servem para outra conta. Revise a seção 8.

4. Uma equipe quer implantar uma nova versão da Lambda com 10% do tráfego por 10 minutos e rollback automático se os erros subirem. Escolha DUAS: (A) `AutoPublishAlias` com `DeploymentPreference: Canary10Percent10Minutes` no SAM; (B) alarme do CloudWatch nos erros associado à implantação; (C) atualizar `$LATEST` diretamente; (D) aumentar o timeout; (E) apagar a versão anterior antes da implantação.

RESPOSTA

A e B. Canary com CodeDeploy via SAM e alarme para rollback. As demais não fazem deslocamento gradual nem rollback. Revise as seções 9 e 11.

5. Uma API REST no API Gateway deve oferecer planos com limites diferentes para clientes e reduzir chamadas repetidas ao backend. Escolha DUAS: (A) usage plans com API keys; (B) cache no stage; (C) HTTP API; (D) Lambda Function URL; (E) aumentar a memória da Lambda.

RESPOSTA

A e B. Usage plans controlam throttling e cota por cliente; o cache reduz chamadas. HTTP API não tem esses recursos. Revise a seção 3.

6. Uma tabela DynamoDB sofre throttling em uma chave muito acessada, embora a capacidade total esteja sobrando. Escolha DUAS: (A) usar uma partition key de maior cardinalidade ou write sharding; (B) fazer retries com backoff exponencial; (C) criar um LSI na tabela existente; (D) trocar Query por Scan; (E) aumentar o tamanho dos itens.

RESPOSTA

A e B. O problema é uma partição quente: melhore a distribuição da chave e trate o throttling com backoff. LSI só na criação, e Scan piora. Revise a seção 4.

7. Uma função precisa ler a senha do banco com rotação automática e baixa latência. Escolha DUAS: (A) guardar a senha no Secrets Manager com rotação; (B) ler o segredo com a Parameters and Secrets Lambda Extension (cache); (C) colocar a senha em texto na variável de ambiente; (D) guardar a senha no código; (E) usar Parameter Store Standard com rotação nativa.

RESPOSTA

A e B. Secrets Manager rotaciona e a extensão faz cache. Parameter Store não tem rotação nativa. Revise a seção 8.

8. Um pedido criado deve ser processado por faturamento, estoque e e-mail, cada um com retries e DLQ próprios. Escolha DUAS: (A) publicar em um tópico SNS; (B) assinar uma fila SQS por serviço no tópico; (C) uma única fila SQS com três consumidores; (D) Kinesis com um shard; (E) chamar os três serviços sincronicamente.

RESPOSTA

A e B. Fan-out SNS → SQS dá a cada consumidor sua cópia, retries e DLQ. Uma fila com três consumidores divide as mensagens em vez de copiá-las. Revise a seção 6.

9. Uma equipe precisa encontrar rapidamente os traces de um cliente específico e medir o tempo de uma dependência externa. Escolha DUAS: (A) adicionar uma annotation com o ID do cliente; (B) criar um subsegmento para a chamada externa; (C) guardar o ID do cliente em metadata para busca; (D) desativar o sampling; (E) usar CloudTrail.

RESPOSTA

A e B. Annotations são indexadas e subsegmentos medem dependências. Metadata não é indexada e CloudTrail registra APIs da AWS. Revise a seção 12.

10. Uma aplicação no Elastic Beanstalk precisa ser atualizada sem reduzir a capacidade e com rollback mais rápido possível, e o banco não pode ser perdido. Escolha DUAS: (A) política de implantação immutable; (B) RDS criado fora do ambiente; (C) política all at once; (D) RDS dentro do ambiente; (E) política rolling.

RESPOSTA

A e B. Immutable mantém a capacidade e tem o rollback mais rápido; o RDS fora do ambiente não é apagado com ele. Revise a seção 11.

11. Uma função Lambda em VPC precisa gravar no DynamoDB e chamar uma API pública de terceiros. Escolha DUAS: (A) gateway VPC endpoint para o DynamoDB; (B) NAT Gateway com a função em sub-rede privada; (C) atribuir IP público à função; (D) colocar a função em sub-rede pública sem NAT; (E) remover a função da VPC mesmo que ela precise acessar o RDS privado.

RESPOSTA

A e B. O endpoint dá acesso privado ao DynamoDB e o NAT dá saída à internet. Funções em VPC não recebem IP público. Revise a seção 2.

12. Uma equipe quer métricas de negócio emitidas pela Lambda e alertas quando os erros aumentarem, com o menor impacto na latência. Escolha TRÊS: (A) emitir métricas com Embedded Metric Format; (B) criar alarme do CloudWatch na métrica de erros; (C) notificar pelo SNS; (D) chamar `PutMetricData` várias vezes por invocação; (E) gravar métricas em um arquivo no `/tmp`; (F) usar o CloudTrail para métricas.

RESPOSTA

A, B e C. EMF gera métricas pelos logs sem chamadas de API, o alarme detecta o aumento e o SNS notifica. As demais adicionam latência ou não funcionam. Revise a seção 12.

Síntese de revisão

Este guia organiza a revisão pelos quatro domínios do exame e reúne formatos de consulta rápida e prática ativa.

- Os pesos oficiais do DVA-C02 são 32% para desenvolvimento, 26% para segurança, 24% para implantação e 18% para troubleshooting e otimização. O DVA-C02 pode ser feito até 30/nov/2026; o DVA-C03 começa em 1º/dez/2026 (seção 14).
- As tabelas de decisão rápida ao fim de cada capítulo e os padrões recorrentes (seção 15) concentram as palavras-chave, os pares que mais se confundem e as afirmações falsas clássicas.
- As tabelas de números (Lambda, API Gateway, DynamoDB, mensageria) foram conferidas em setembro de 2026 e mostram os limites que mudaram entre 2024 e 2026.
- O autoteste e as questões de múltipla resposta (seções 17 e 18) oferecem 216 perguntas para revisão ativa.
- O mapa da seção 18 relaciona as 13 tarefas publicadas no guia oficial às seções correspondentes.